

IT3212: Knowledge extraction from data

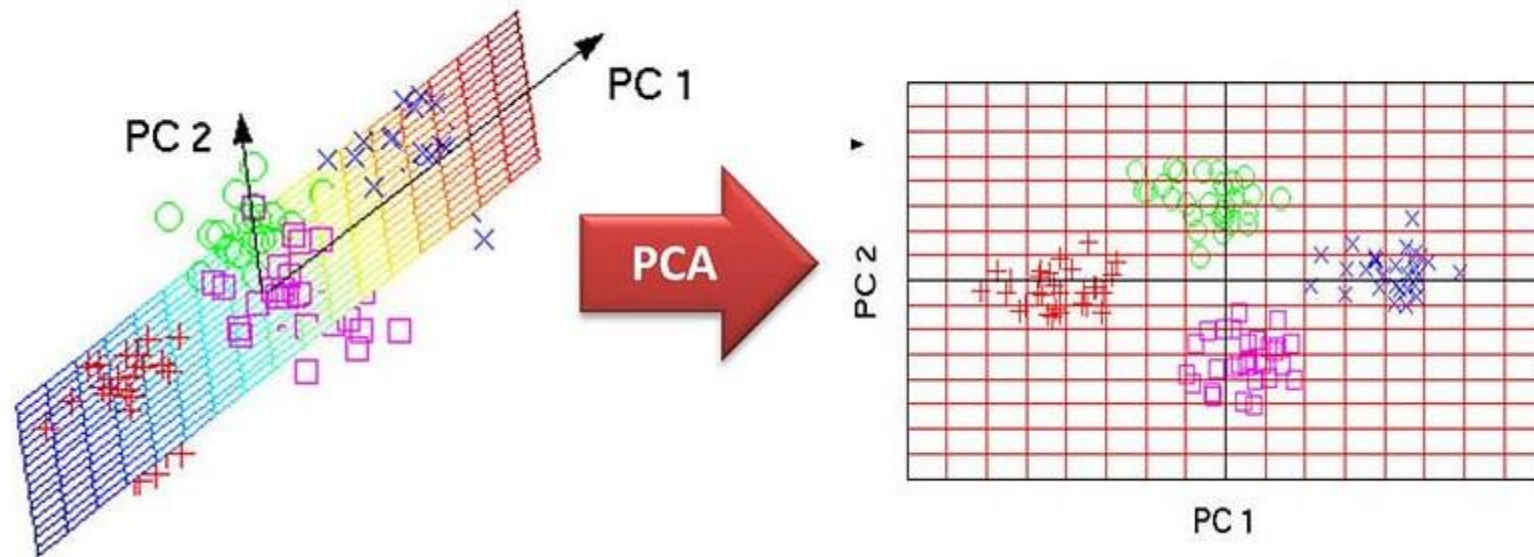
Kshitij (K) Sharma

Department of Computer and Information Science
Faculty of Information Technology and Electrical Engineering

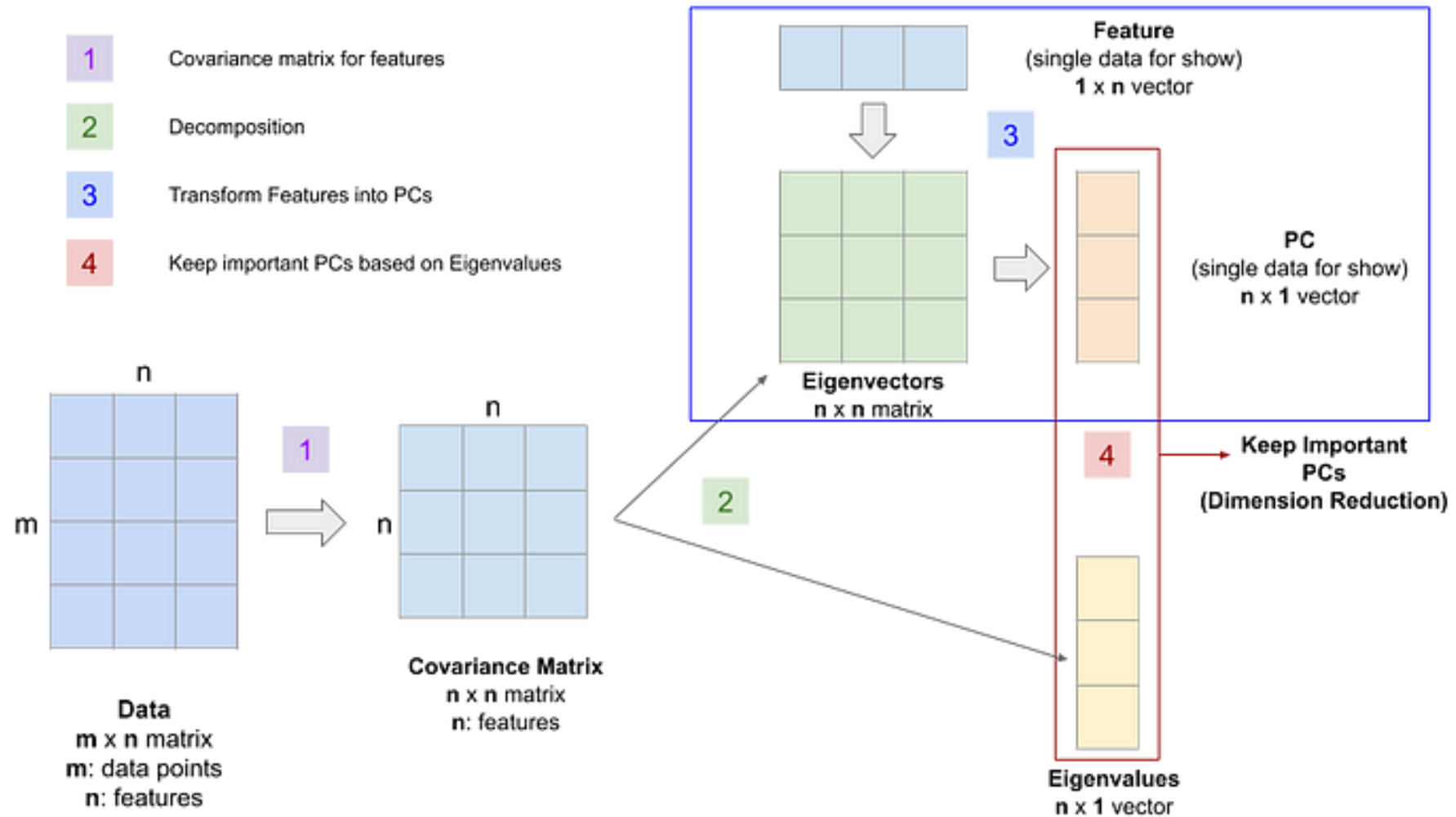
Most common techniques

- PCA
- LDA
- Fourier Transform
- Wavelet Transform
- Histogram based features
- Autoencoders (**Later in the course**)
- Text data processing
- Time series decomposition
- Image processing

PCA



PCA



PCA



```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

Perform one-hot encoding

```
categorical_columns = df.columns[df.dtypes == object] # Find all categorical columns
df = pd.get_dummies(df, columns = categorical_columns, drop_first=True)
```

Create training / test split

```
X_train, X_test, y_train, y_test = train_test_split(df[df.columns[df.columns != 'salary']],
                                                    df['salary'], test_size=0.25, random_state=1)
```

Standardize the dataset;

```
sc = StandardScaler()
sc.fit(X_train)
X_train_std = sc.transform(X_train)
X_test_std = sc.transform(X_test)
```

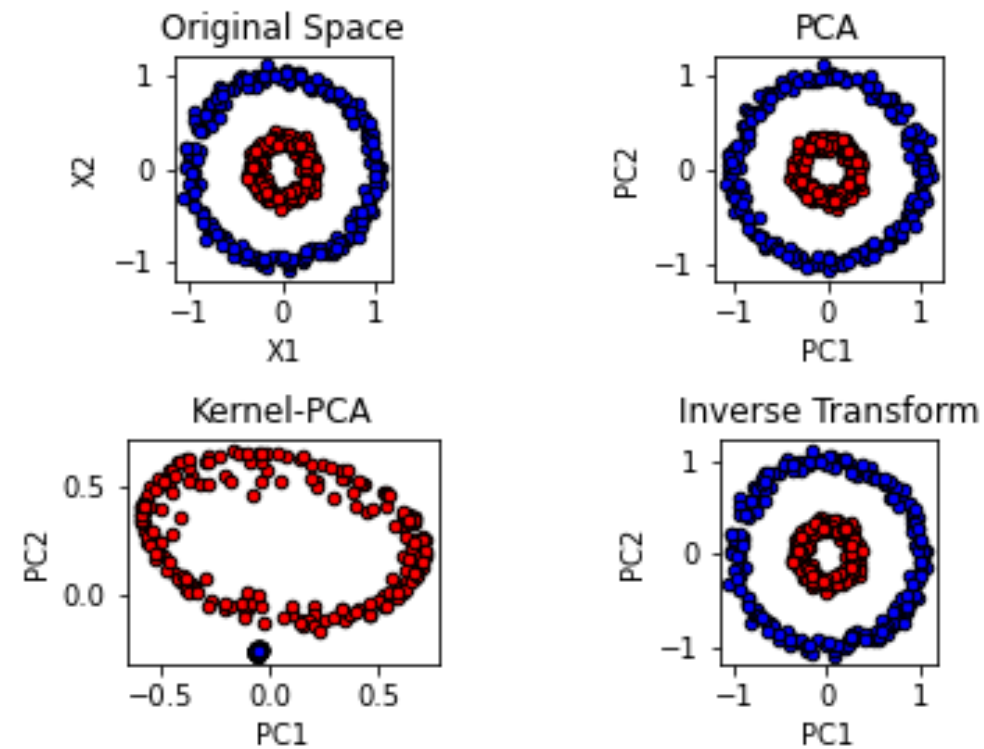
Perform PCA

```
pca = PCA()
```

Determine transformed features

```
X_train_pca = pca.fit_transform(X_train_std)
X_test_pca = pca.transform(X_test_std)
```

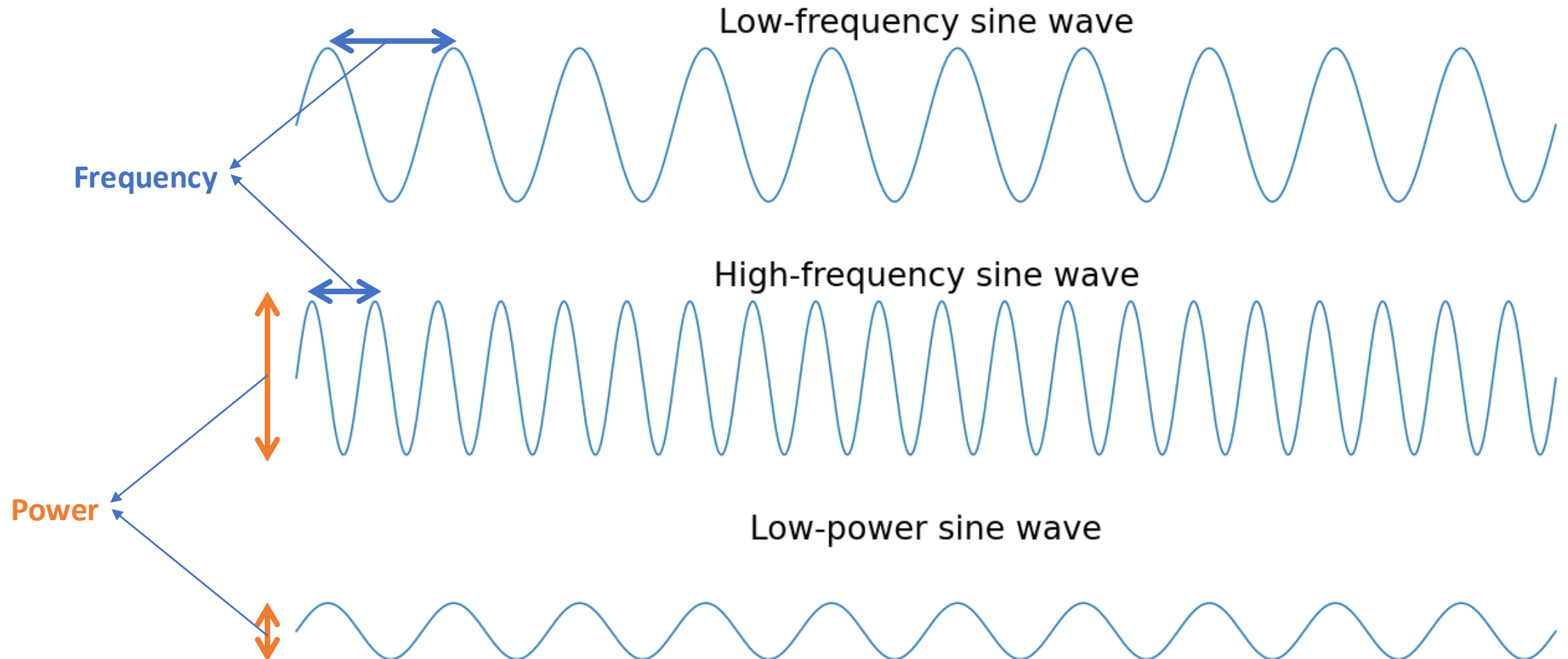
Kernel PCA



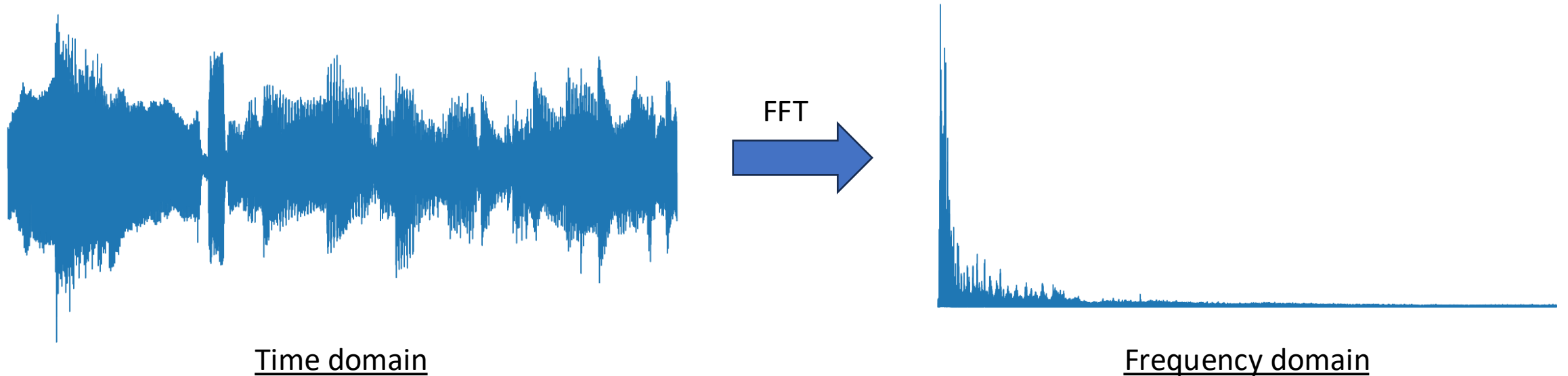
Fourier Transform

- A **signal** is information that changes over time. For example, audio, video, and voltage traces are all examples of signals.
- A **frequency** is the speed at which something repeats. For example, clocks tick at a frequency of one hertz (Hz), or one repetition per second.
- **Power**, in this case, just means the strength of each frequency.

Fourier Transform

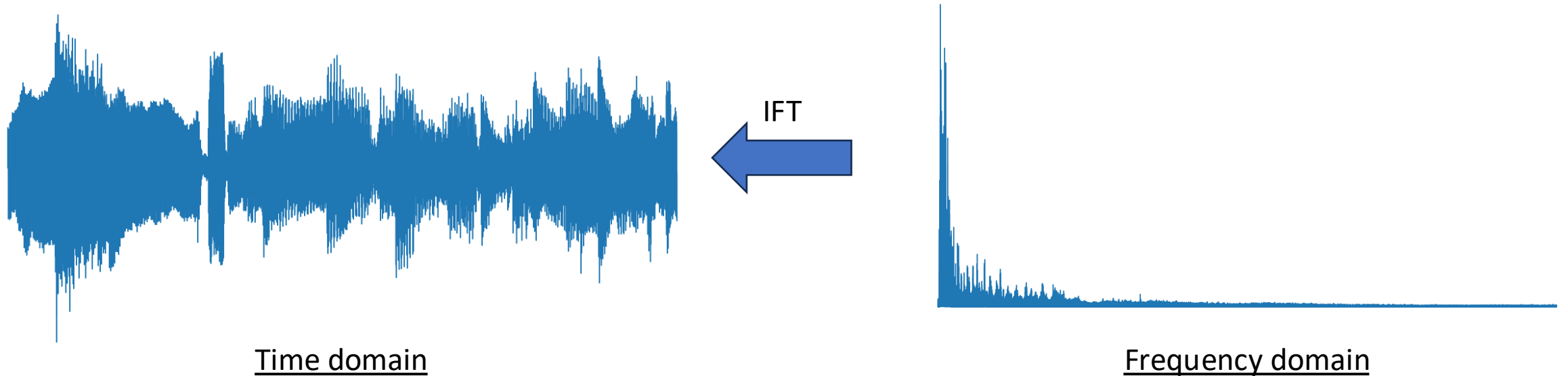


Fourier Transform



- Compare signal with sinusoids of various frequencies
- For each frequency we get a magnitude and a phase
- High magnitude indicates high similarity between the signal and a sinusoid

Fourier Transform



- Superimpose sinusoids
- Weight them by the relative magnitude
- Use relative phase
- Original signal and FT have same information

Fourier Transform

```
import numpy as np
from matplotlib import pyplot as plt
```

```
SAMPLE_RATE = 44100 # Hertz
DURATION = 5 # Seconds
```

```
def generate_sine_wave(freq, sample_rate, duration):
    x = np.linspace(0, duration, sample_rate * duration, endpoint=False)
    frequencies = x * freq
    # 2pi because np.sin takes radians
    y = np.sin((2 * np.pi) * frequencies)
    return x, y
```

```
# Generate a 2 hertz sine wave that lasts for 5 seconds
x, y = generate_sine_wave(2, SAMPLE_RATE, DURATION)
plt.plot(x, y)
plt.show()
```

Fourier Transform

```
_, nice_tone = generate_sine_wave(400, SAMPLE_RATE, DURATION)
_, noise_tone = generate_sine_wave(4000, SAMPLE_RATE, DURATION)
noise_tone = noise_tone * 0.3
```

```
mixed_tone = nice_tone + noise_tone
```

Generate two tones and mix them

```
normalized_tone = np.int16((mixed_tone / mixed_tone.max()) * 32767)
```

```
plt.plot(normalized_tone[:1000])
plt.show()
```

Plot them

```
from scipy.io.wavfile import write
```

```
# Remember SAMPLE_RATE = 44100 Hz is our playback rate
write("mysinewave.wav", SAMPLE_RATE, normalized_tone)
```

Write them

Fourier Transform

```
from scipy.fft import fft, fftfreq
```

```
# Number of samples in normalized_tone
```

```
N = SAMPLE_RATE * DURATION
```

```
yf = fft(normalized_tone)
```

```
xf = fftfreq(N, 1 / SAMPLE_RATE)
```

```
plt.plot(xf, np.abs(yf))
```

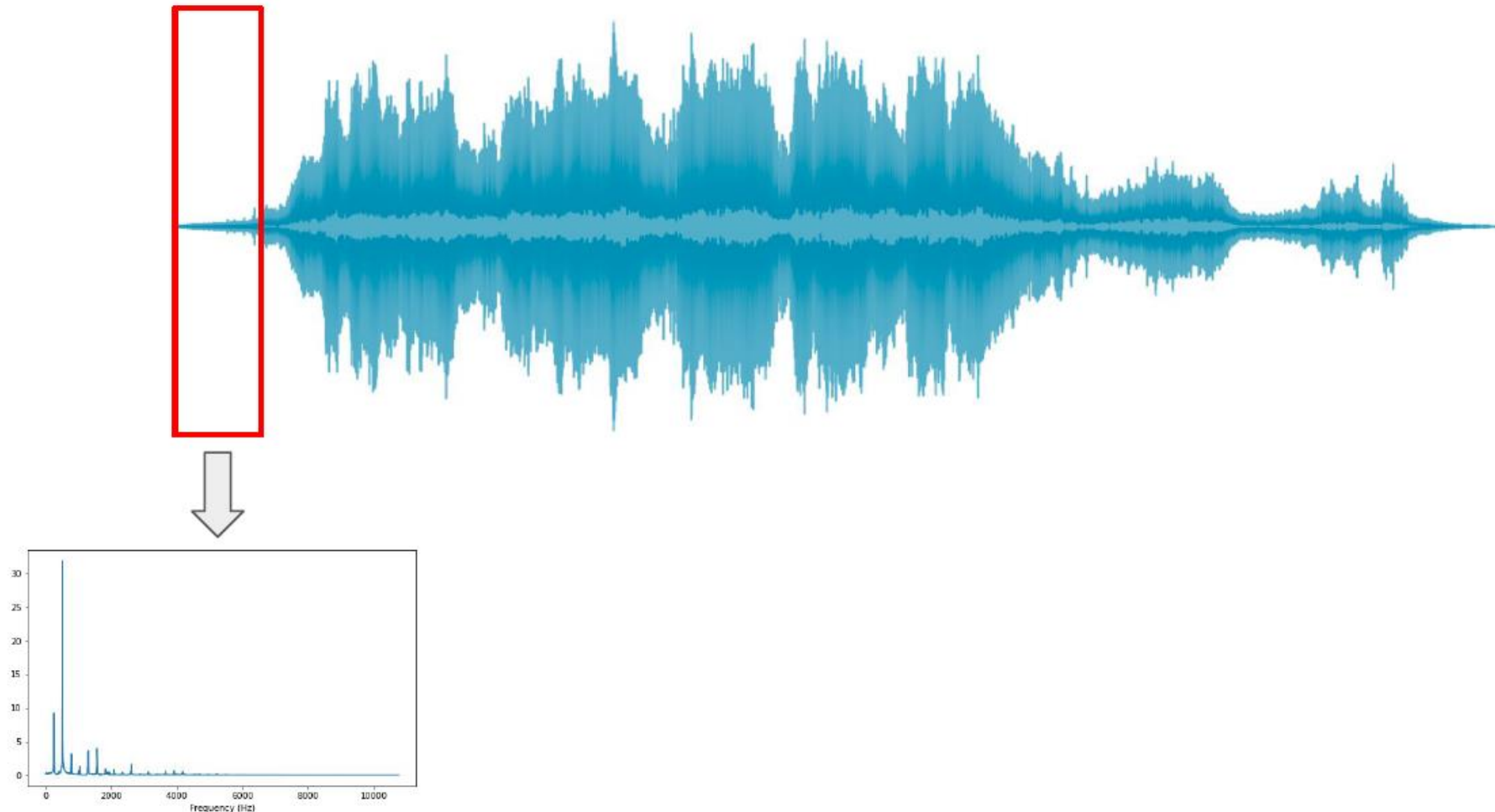
```
plt.show()
```

```
yf = fft(normalized_tone) # calculates the transform itself.
```

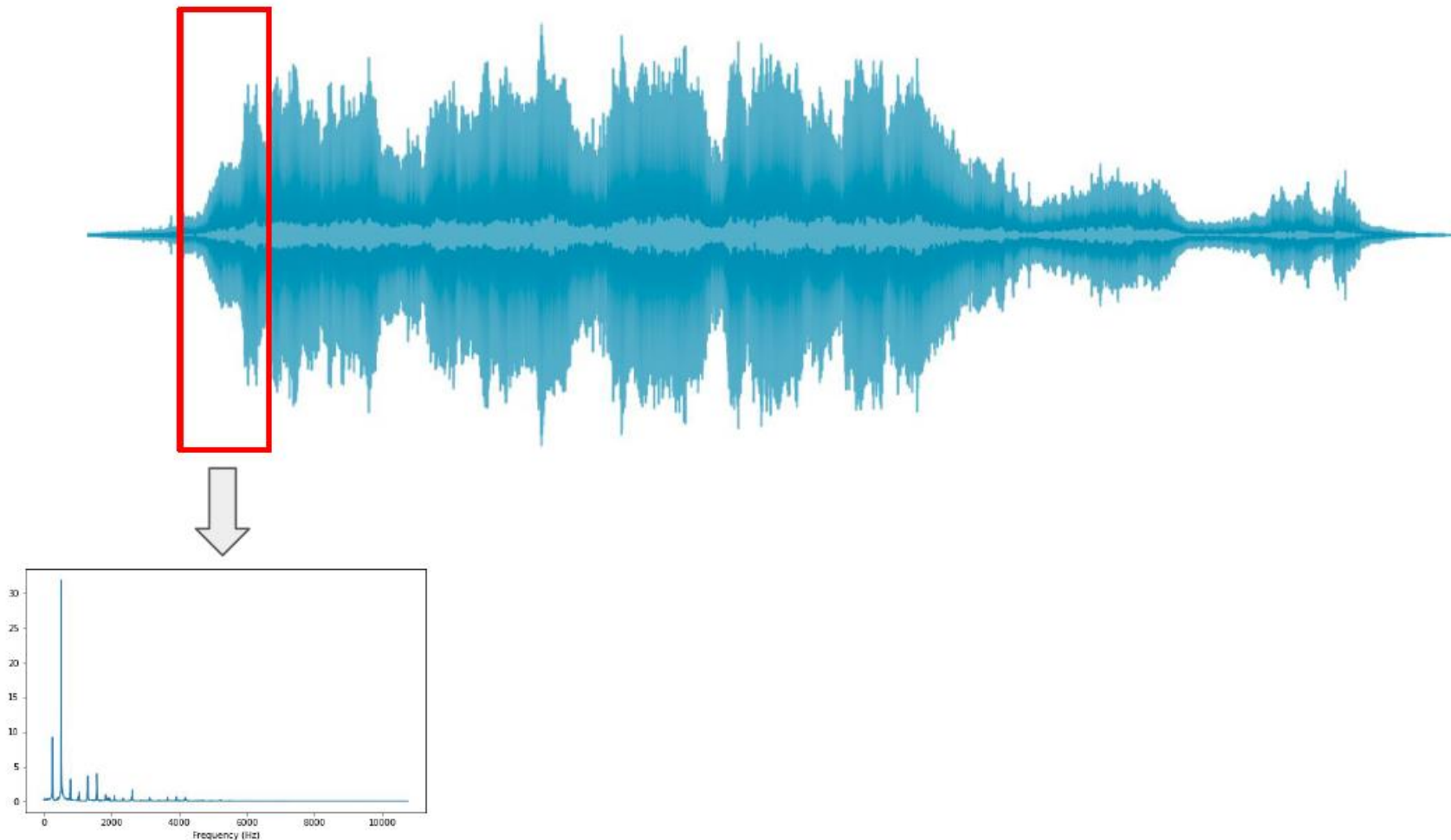
```
xf = fftfreq(N, 1 / SAMPLE_RATE)
```

```
#calculates the frequencies in the center of each bin in the output of fft(). Without this, there would be  
#no way to plot the x-axis on your frequency spectrum.
```

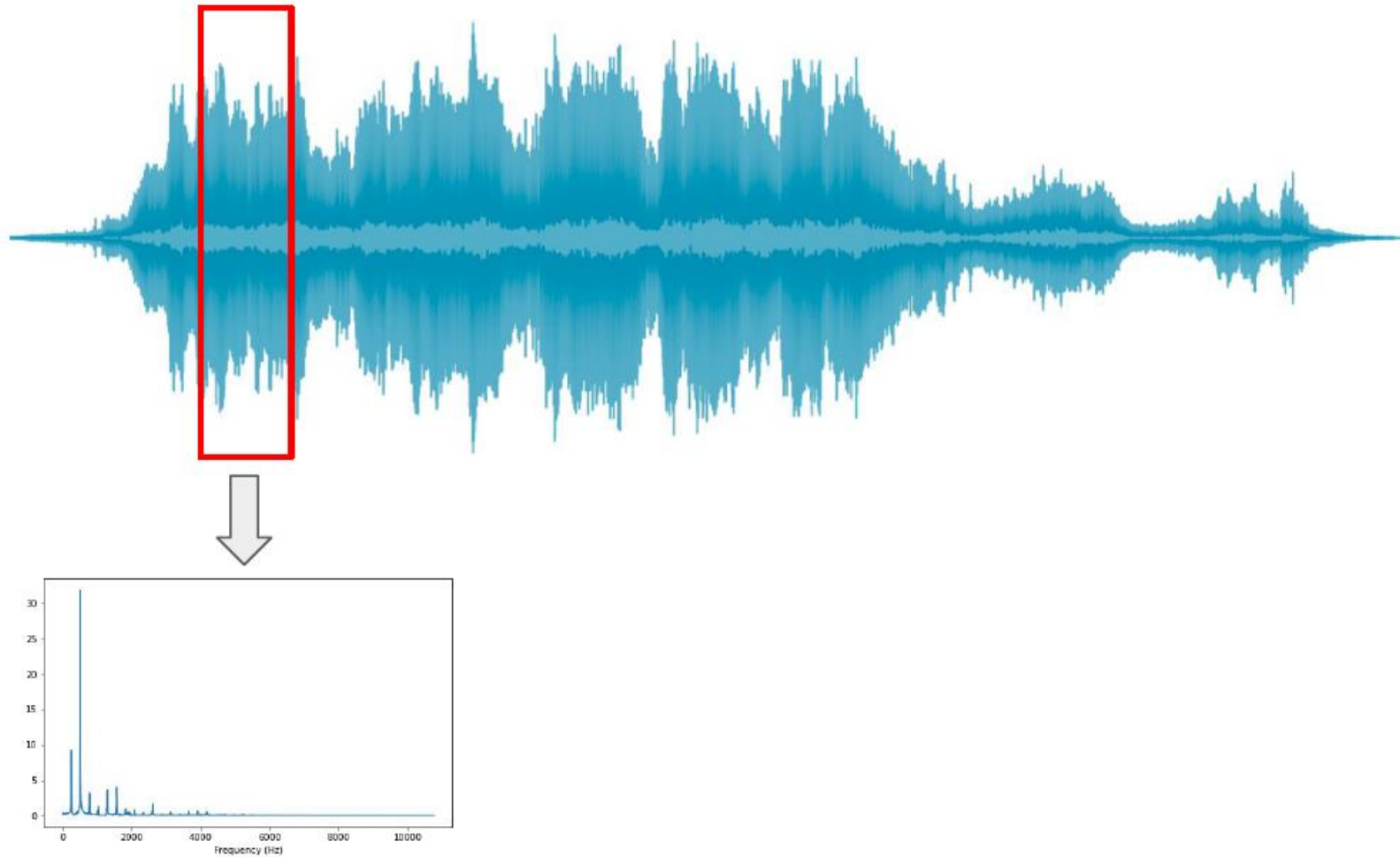
Short term Fourier Transform



Short term Fourier Transform

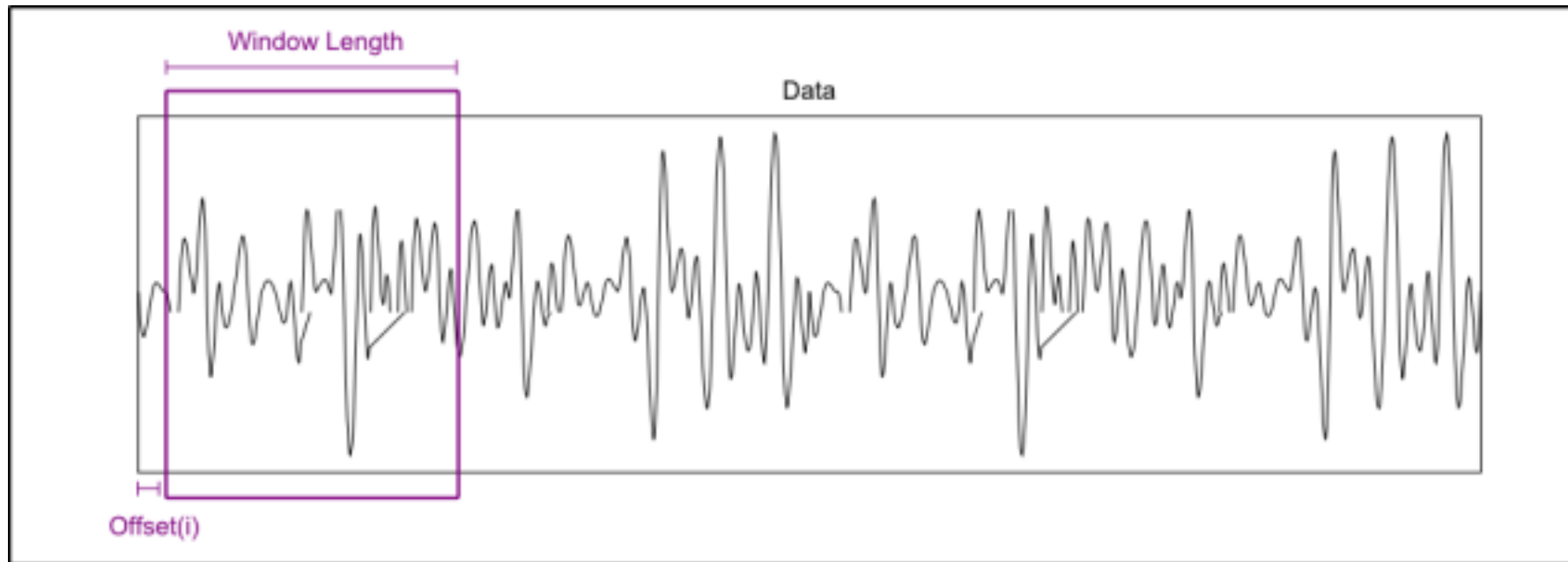


Short term Fourier Transform



Short term Fourier Transform

Step 1 – Pick segment: We need to find our current segment to process from the overall data set. We use the concept of a ‘sliding window’ to help us visualize what’s happening. The data inside the window is the current segment to be processed.



Short term Fourier Transform

Step 1 – Pick segment: We need to find our current segment to process from the overall data set. We use the concept of a ‘sliding window’ to help us visualize what’s happening. The data inside the window is the current segment to be processed.

```
# data = a numpy array containing the signal to be processed  
# fs = a scalar which is the sampling frequency of the data
```

```
hop_size = np.int32(np.floor(fft_size * (1-overlap_fac)))
```

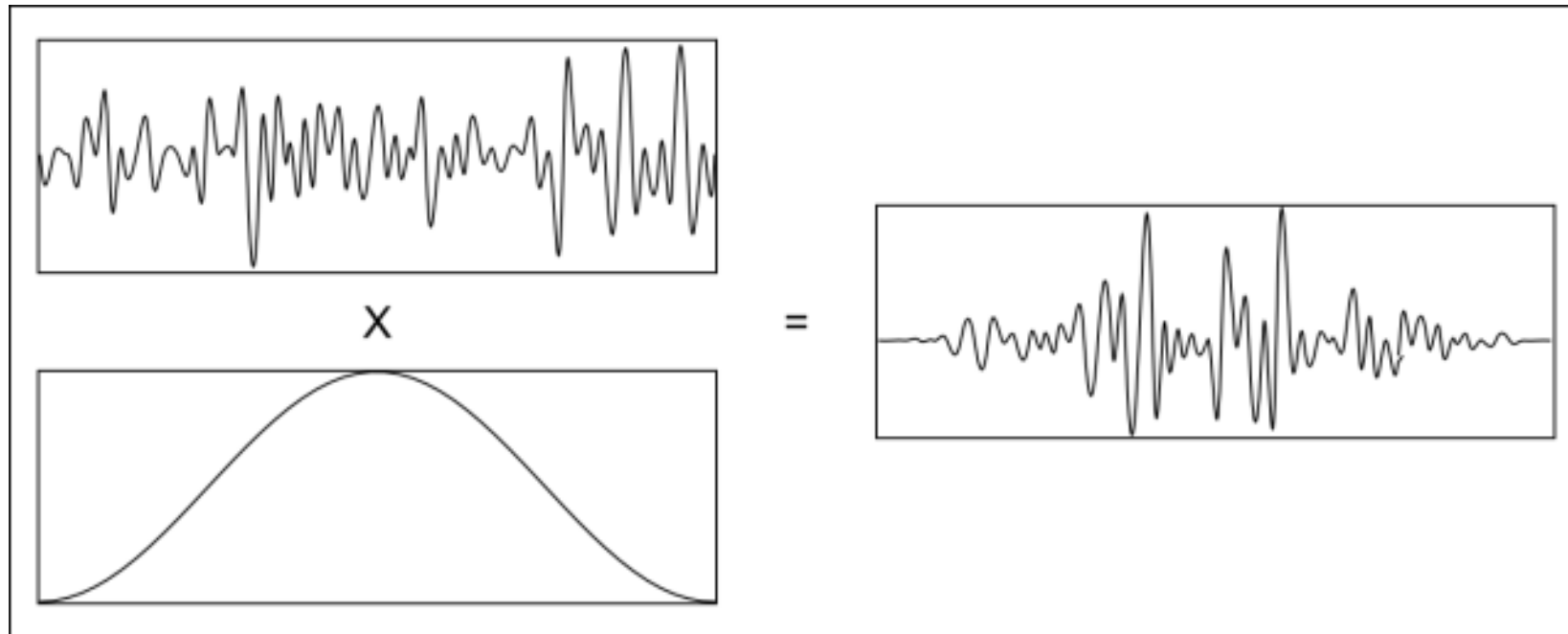
```
pad_end_size = fft_size # the last segment can overlap the end of the data array by no more than one  
window size
```

```
total_segments = np.int32(np.ceil(len(data) / np.float32(hop_size)))
```

```
t_max = len(data) / np.float32(fs)
```

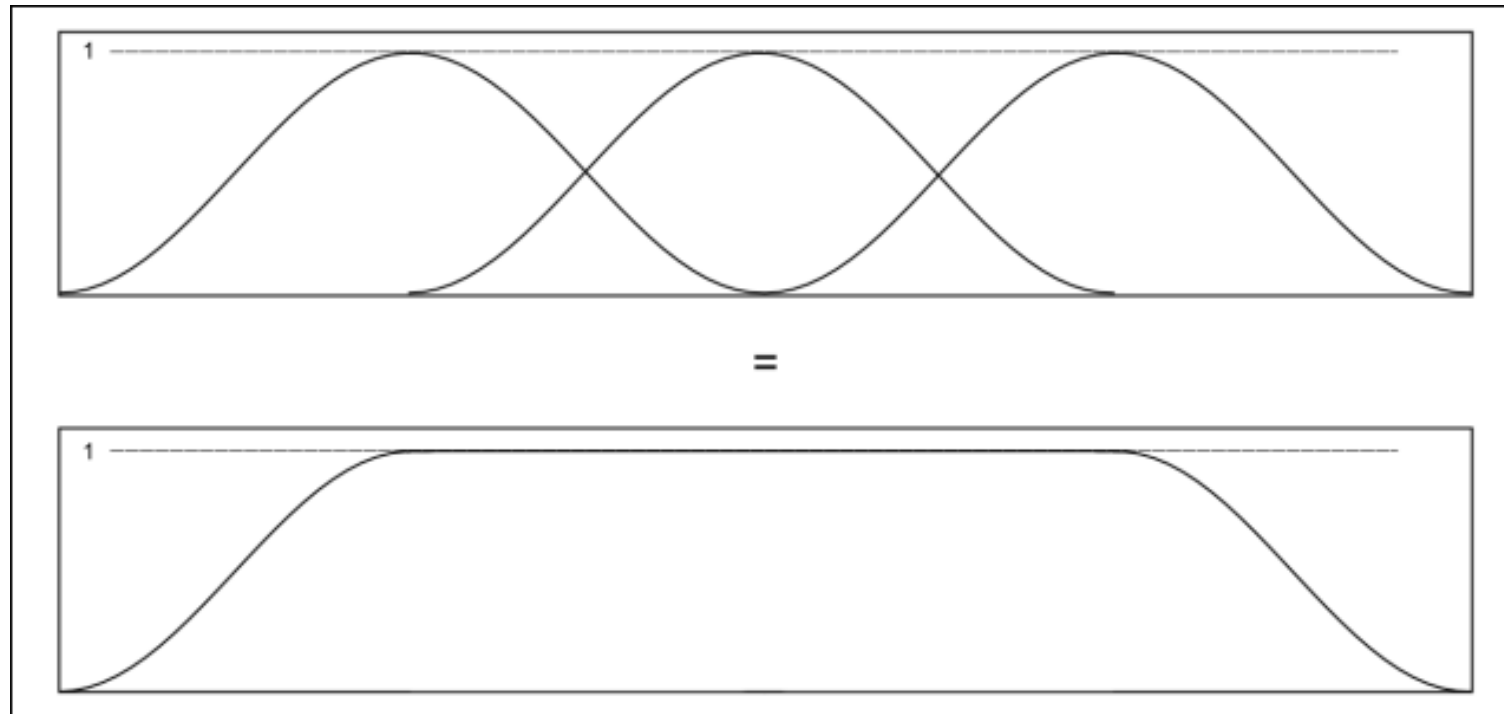
Short term Fourier Transform

Step 2 – Multiply by half-cosine: This helps to alleviate a problem created by segmenting the data. When the data is cut into pieces, the edges make a sharp transition that didn't exist before. Multiplying by a half cosine function helps to fade the signal in and out so that the transitions at the edges do not affect the Fourier transform of the data.



Short term Fourier Transform

Step 2 – Multiply by half-cosine: This helps to alleviate a problem created by segmenting the data. When the data is cut into pieces, the edges make a sharp transition that didn't exist before. Multiplying by a half cosine function helps to fade the signal in and out so that the transitions at the edges do not affect the Fourier transform of the data.



```
window = np.hanning(fft_size) # our half cosine window
```

Short term Fourier Transform

Step 3 – Pad Data: We pad the end of the current segment with a number of zeros equal to the length of the window.

Step 4 – Fourier Transform and Scale: This is where all the magic happens. Everything up to this point was just preparing the data, but this step is the beating heart of the algorithm. Here we finally take the Fourier transform of the data. I'm not going to explain the Fourier transform here except to say that it takes a signal and extracts the frequency content from that signal.

Short term Fourier Transform

```
inner_pad = np.zeros(fft_size) # the zeros which will be used to double each segment size
```

```
proc = np.concatenate((data, np.zeros(pad_end_size))) # the data to process
```

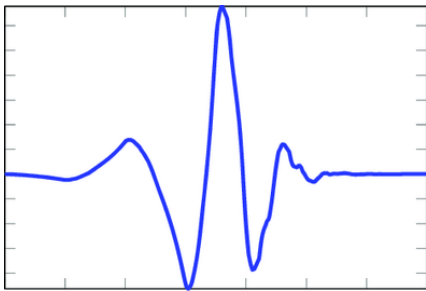
```
result = np.empty((total_segments, fft_size), dtype=np.float32) # space to hold the result
```

```
for i in xrange(total_segments): # for each segment
    current_hop = hop_size * i # figure out the current segment offset
    segment = proc[current_hop:current_hop+fft_size] # get the current segment
    windowed = segment * window # multiply by the half cosine function
    padded = np.append(windowed, inner_pad) # add 0s to double the length of the data
    spectrum = np.fft.fft(padded) / fft_size # take the Fourier Transform and scale by the number of samples
    autopower = np.abs(spectrum * np.conj(spectrum)) # find the autopower spectrum
    result[i, :] = autopower[:fft_size] # append to the results array
```

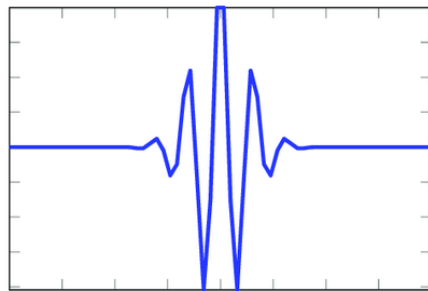
```
result = 20*np.log10(result) # scale to db
```

```
result = np.clip(result, -40, 200) # clip values
```

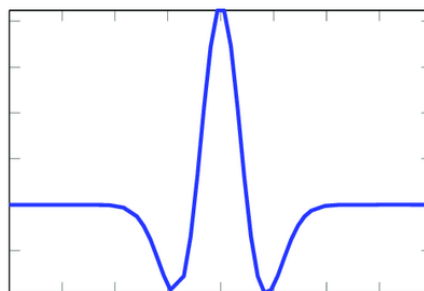
Wavelet Transform



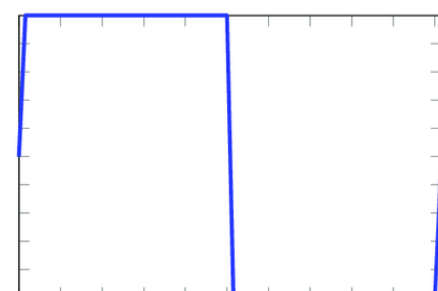
(a) db Wavelet



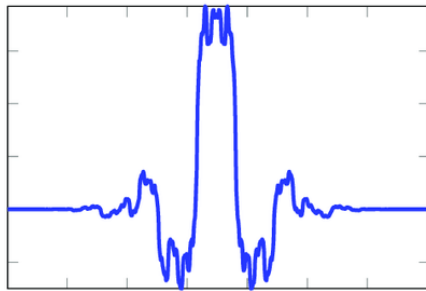
(b) Morlet Wavelet



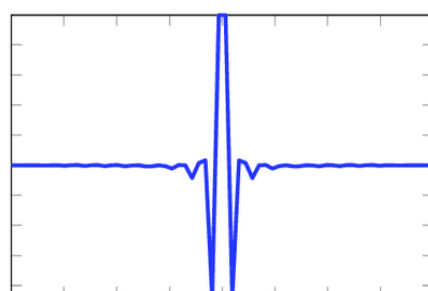
(e) Mexican Hat Wavelet



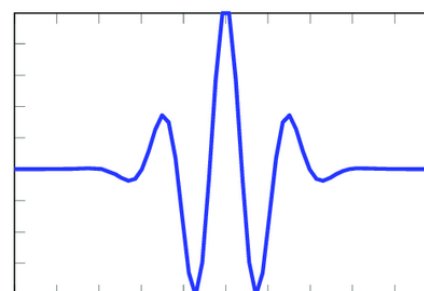
(f) Haar Wavelet



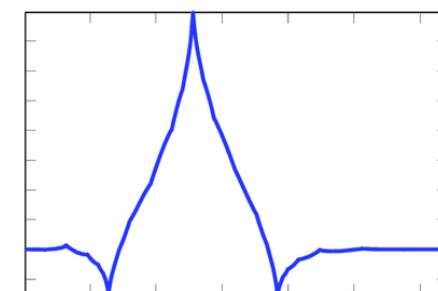
(c) Biorthogonal Wavelet



(d) Spline Wavelet

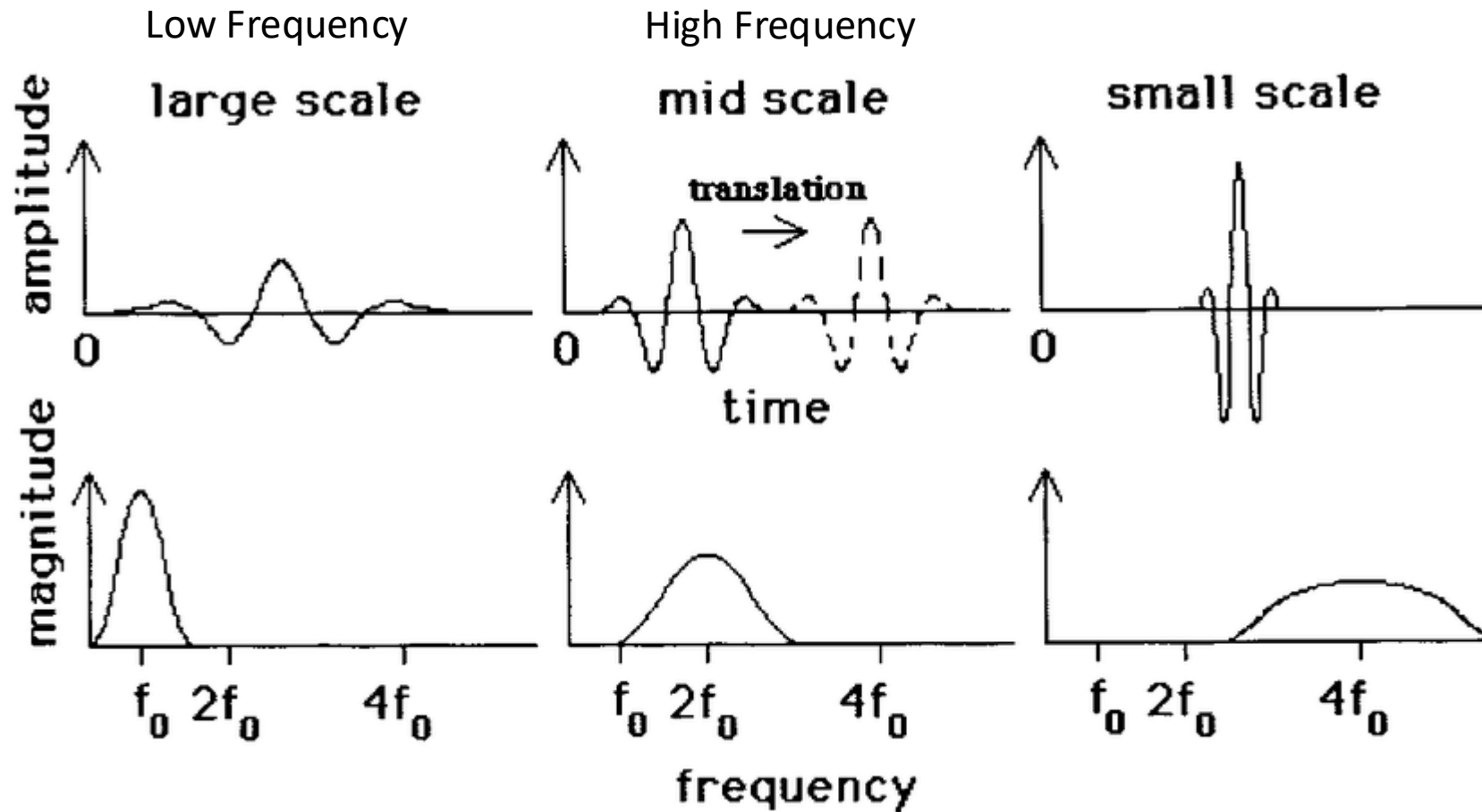


(g) Gaussian Wavelet



(h) Coiflet Wavelet

Wavelet Transform



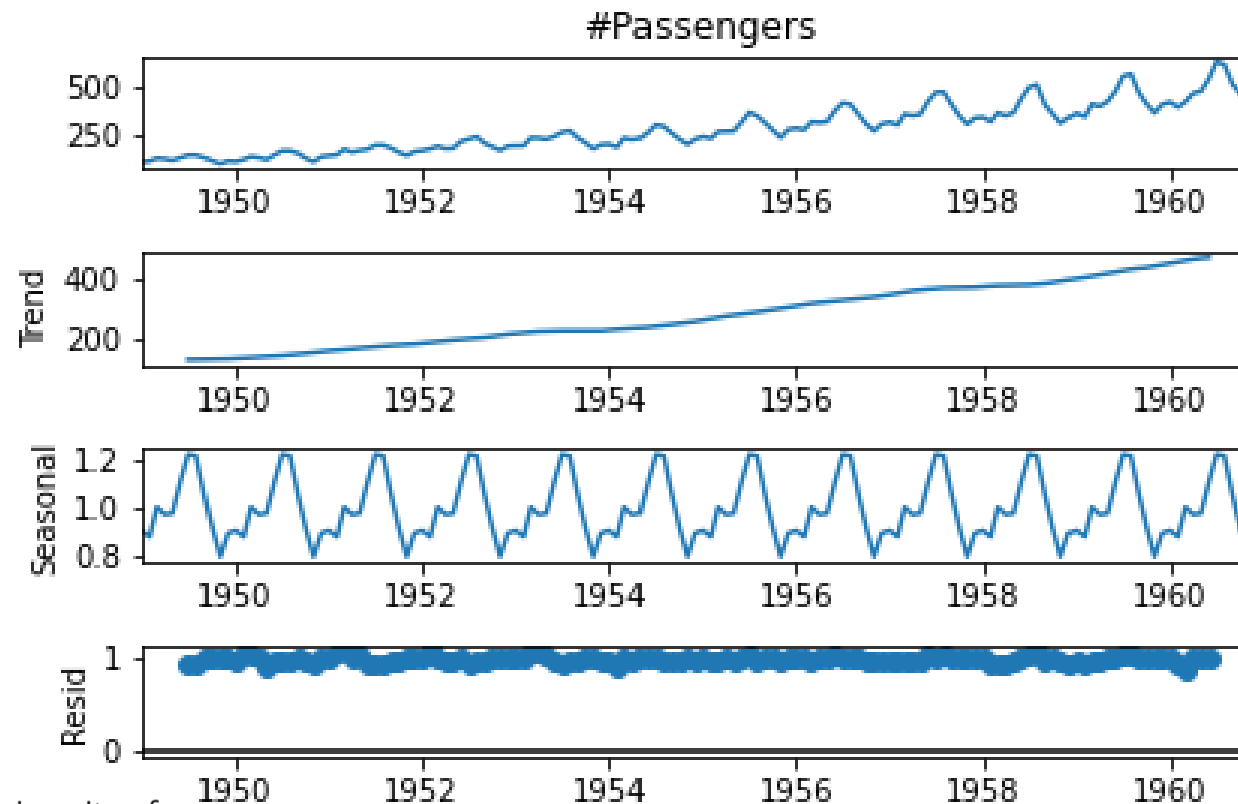
<https://ataspinar.com/2018/12/21/a-guide-for-using-the-wavelet-transform-in-machine-learning/>

Time series decomposition

Seasonality: describes the periodic signal in your time series.

Trend: describes whether the time series is decreasing, constant, or increasing over time.

Noise: describes what remains behind the separation of seasonality and trend from the time series.



Time series decomposition

```
import pandas as pd
import numpy as np
from statsmodels.tsa.seasonal import seasonal_decompose
df=pd.read_csv('AirPassengers.csv')
df.set_index('Month',inplace=True)
df.index=pd.to_datetime(df.index)
#drop null values
df.dropna(inplace=True)
df.plot()
result=seasonal_decompose(df['#Passengers'],model='multiplicable',period=12)
result.seasonal.plot()
result.trend.plot()
result.plot()
```

Time series decomposition

Three models when decomposing a time series:

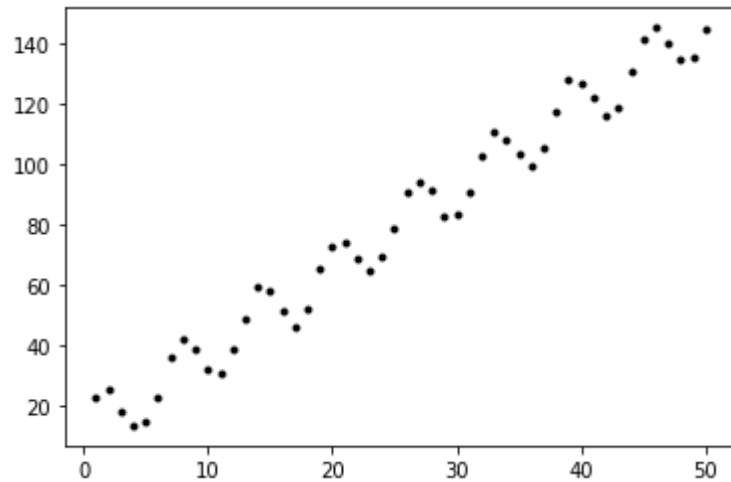
1. Additive time series model. **Time Series = Trend + Seasonality + Residual**
2. Multiplicative time series model. **Time Series = Trend x Seasonality x Residual**
3. Pseudo-Additive model. **Time Series = Trend + Seasonality*Trend + Residual*Trend**

If the seasonality and residuals are independent of the trend in the plotted time series, we use **an additive model** to decompose the data.

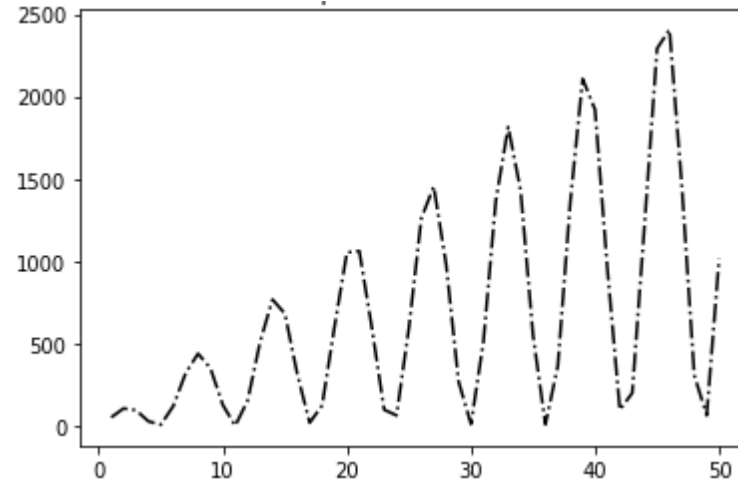
If the plotted series shows the seasonality and residuals to change with the change in trend, we use a **multiplicative model** to decompose our data

There are cases where we cannot either use an additive time series model or a multiplicative one to decompose our data. We use a special model known as the Pseudo additive model in such situations

Time series decomposition



If the seasonality and residuals are independent of the trend in the plotted time series, we use **an additive model** to decompose the data.



If the plotted series shows the seasonality and residuals to change with the change in trend, we use a **multiplicative model** to decompose our data

There are cases where we cannot either use an additive time series model or a multiplicative one to decompose our data. We use a special model known as the Pseudo additive model in such situations

Time series decomposition

frequency is the time a time series is taking to complete on cycle

```
ts_dicomposition = seasonal_decompose(x=additive_Tmodel, model='additive', freq=6)
```

```
trend_estimate = ts_dicomposition.trend
```

```
seasonal_estimate = ts_dicomposition.seasonal
```

```
residual_estimate = ts_dicomposition.resid
```

```
ts_decomposition = seasonal_decompose(x=multiplicative_Tmodel, model='multiplicative', freq=6)
```

```
trend_estimate = ts_decomposition.trend
```

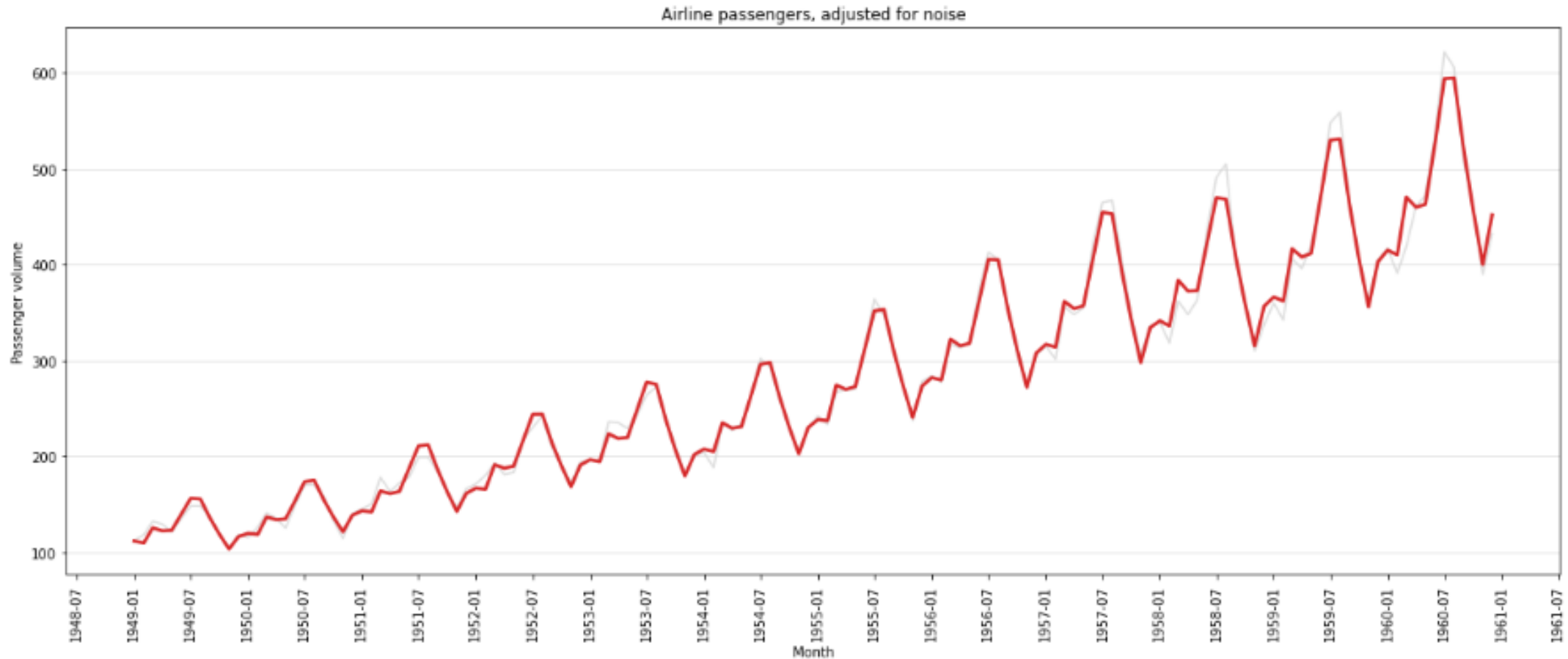
```
seasonal_estimate = ts_decomposition.seasonal
```

```
residual_estimate = ts_decomposition.resid
```

Time series decomposition: Moving average



Time series decomposition: Moving average



Time series decomposition: STL decomposition

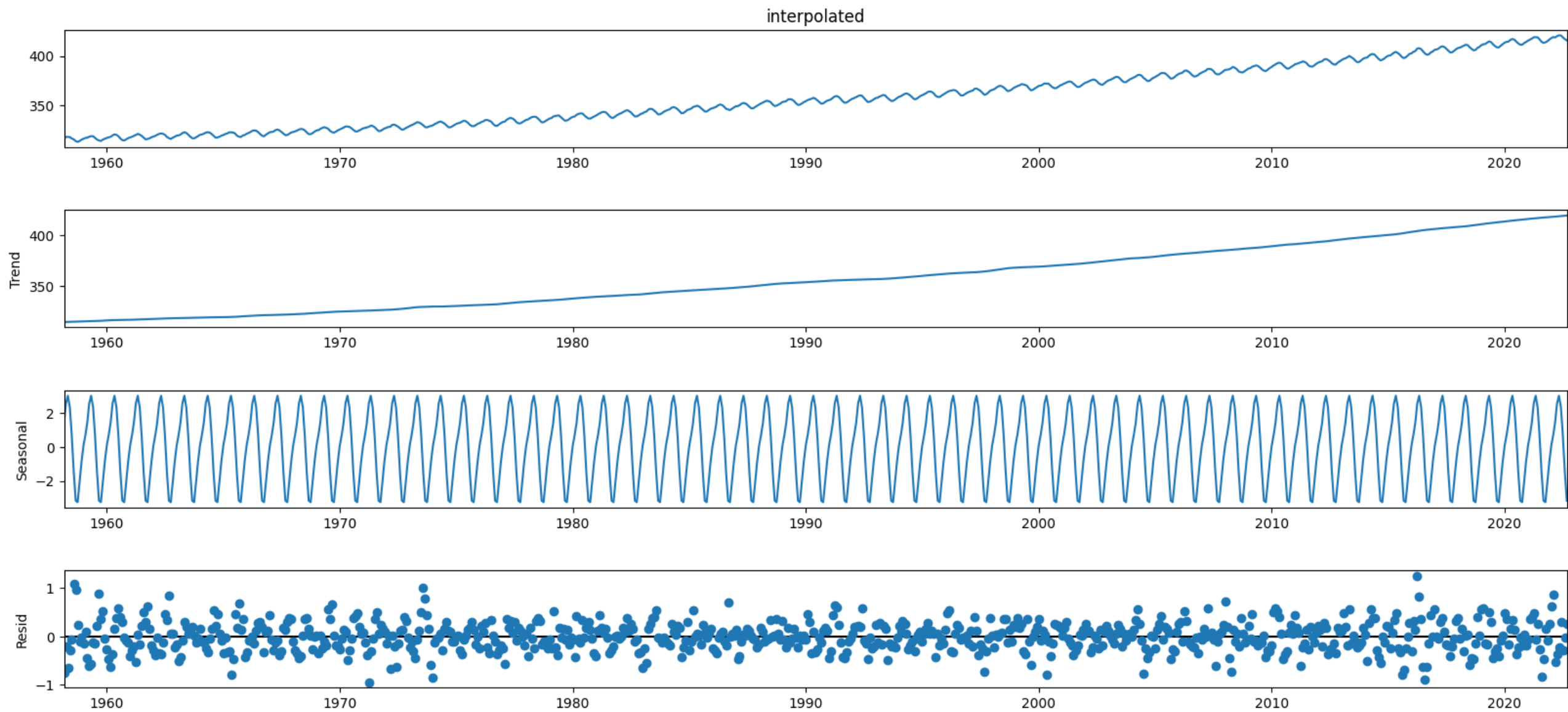
Seasonal and Trend decomposition using LOESS (locally estimated scatterplot smoothing)

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from datetime import datetime
from statsmodels.tsa.seasonal import seasonal_decompose
df_co2 = pd.read_json("KeelingCurve.json")
df_co2 = df_co2.set_index("Date") # set datetimeindex

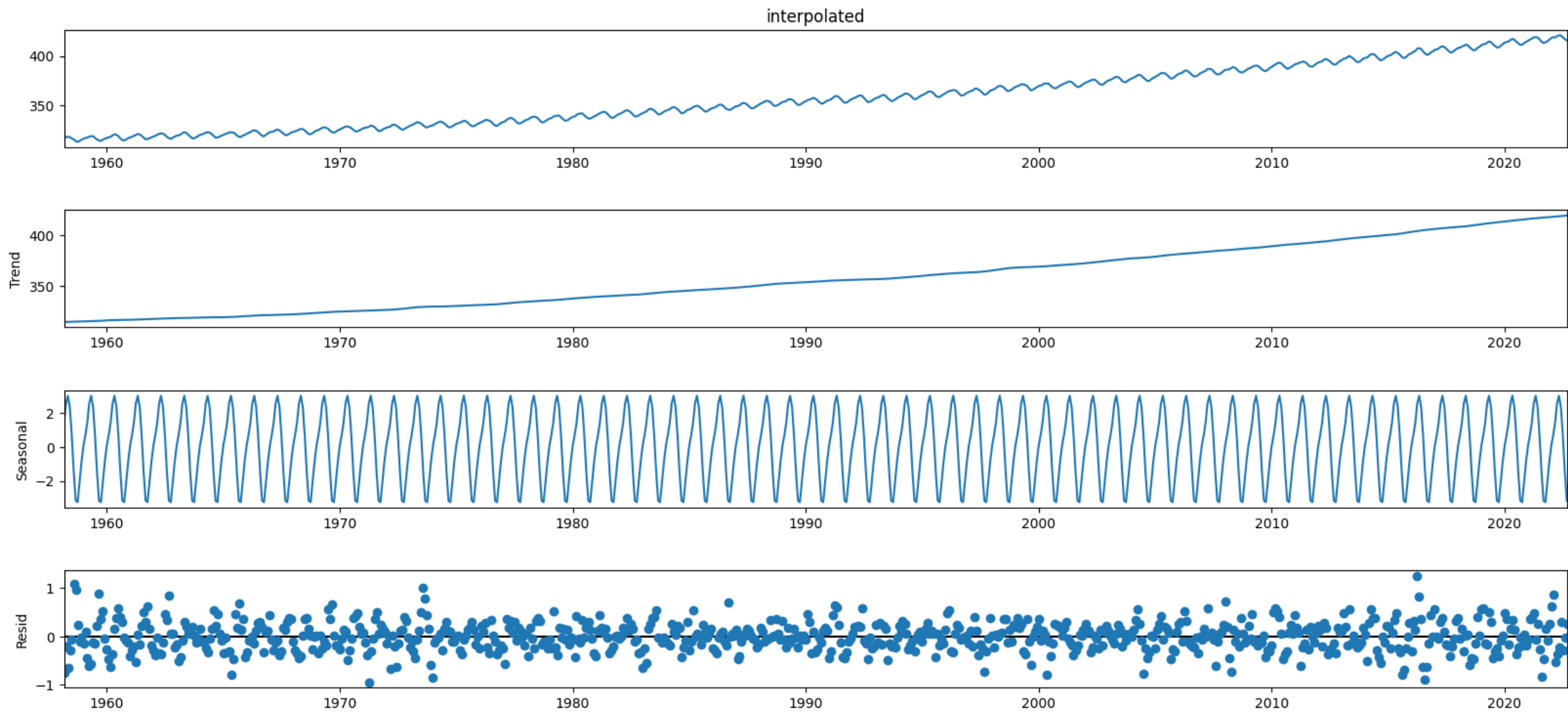
stl_co2 = seasonal_decompose(
    df_co2["interpolated"], model="additive", extrapolate_trend="freq"
)

fig = stl_co2.plot()
plt.show()
```

Time series decomposition: STL



Time series decomposition: STL



Time series decomposition: STL

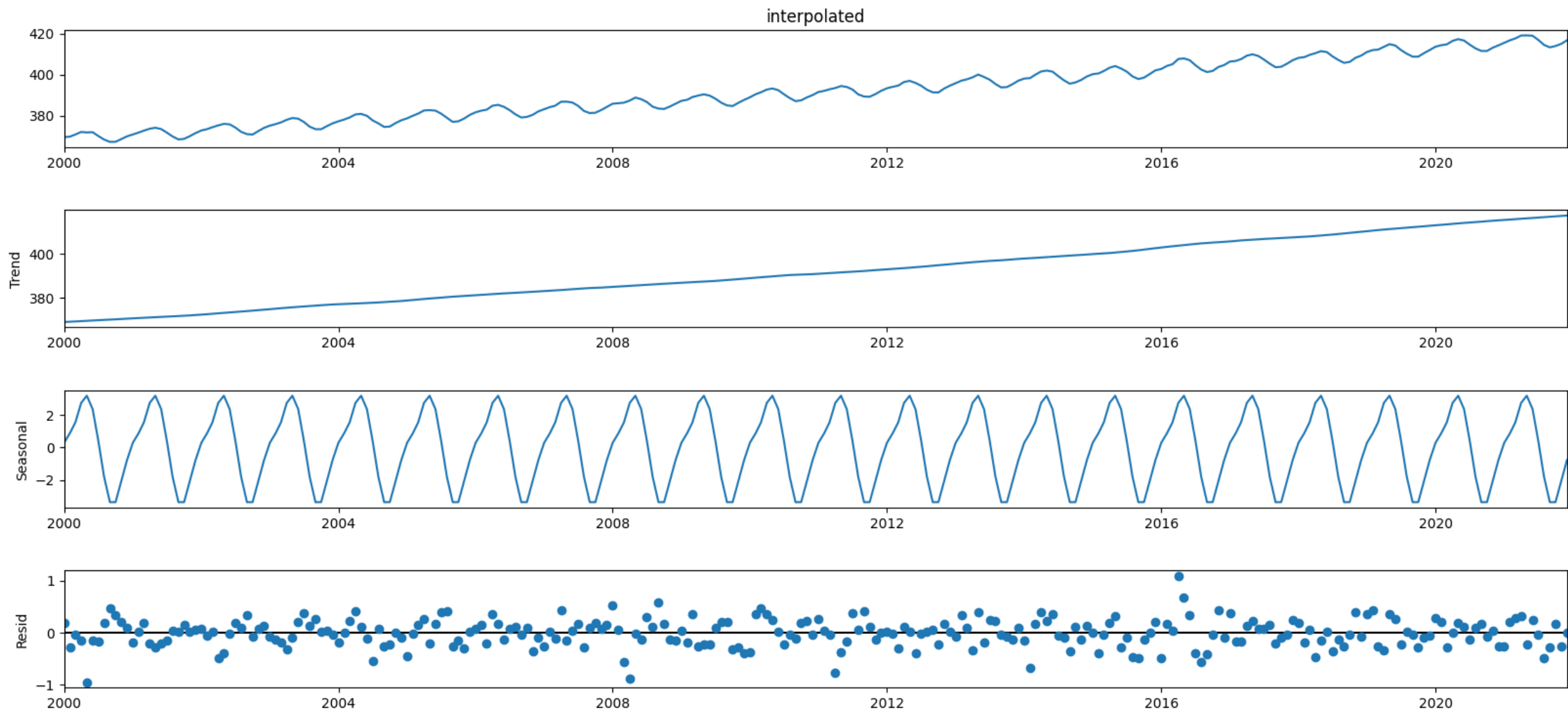
Focus on this century

```
df_co2_2000_2020 = df_co2["2000-01-01":"2021-12-31"]

stl_co2_21 = seasonal_decompose(
    df_co2_2000_2020["interpolated"], model="additive", extrapolate_trend="freq"
)

fig = stl_co2_21.plot()
plt.show()
```

Time series decomposition: STL

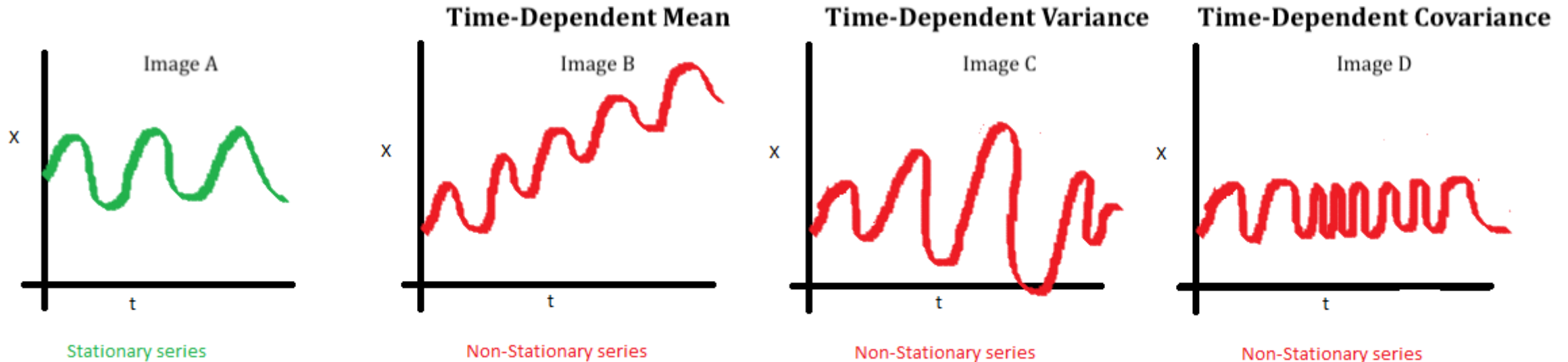


Time series decomposition: ARMA/ARIMA

- AR stands for autoregressive
- MA stands for moving average
- ARMA model is appropriate when time series is stationary (choppy, mean-reverting, no trend)
- When series is nonstationary (smooth, trending), we apply ARMA after taking difference. The model is called ARIMA
- ARMA model is based on idea of “history repeats itself.

Time series decomposition: ARMA/ARIMA

The Principles of Stationarity



Time series decomposition: ARMA

- The autoregressive component represents the relationship between a current value in the time series and its past values.
- The term "autoregressive" refers to the idea that a value in the time series is regressed on its own past values.
- The moving average component models the relationship between a current value and past white noise error terms.
- The term "moving average" refers to the idea that it's a weighted average of past error terms.

Time series decomposition: ARIMA

- The integrated component refers to the differencing of the time series data to make it stationary.
- Stationarity means that the statistical properties of the time series (e.g., mean, variance) do not change over time.
- The order of differencing is denoted by 'd.' If 'd' is 0, no differencing is needed. If 'd' is 1, it means differencing the series once, and so on.
- Differencing helps remove trends and make the data suitable for modeling with ARMA components.

Time series decomposition: ARMA/ARIMA

```
import numpy as np
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA
```

```
gdp = pd.read_csv("GDPC1.csv", index_col=0)
```

```
# Take 1st diff of log of gdp
d_ln_gdp = np.log(gdp).diff()
```

```
p = 3
d = 0
q = 1
```

```
mod = ARIMA(d_ln_gdp, order=(p, d, q))
res = mod.fit()
print(res.summary())
```

Text data feature extraction

Term	Review 1	Review 2	Review 3	TF (Review 1)	TF (Review 2)	TF (Review 3)
This	1	1	1	1/7	1/8	1/6
movie	1	1	1	1/7	1/8	1/6
is	1	2	1	1/7	1/4	1/6
very	1	0	0	1/7	0	0
scary	1	1	0	1/7	1/8	0
and	1	1	1	1/7	1/8	1/6
long	1	0	0	1/7	0	0
not	0	1	0	0	1/8	0
slow	0	1	0	0	1/8	0
spooky	0	0	1	0	0	1/6
good	0	0	1	0	0	1/6

Term	Review 1	Review 2	Review 3	IDF
This	1	1	1	0.00
movie	1	1	1	0.00
is	1	2	1	0.00
very	1	0	0	0.48
scary	1	1	0	0.18
and	1	1	1	0.00
long	1	0	0	0.48
not	0	1	0	0.48
slow	0	1	0	0.48
spooky	0	0	1	0.48
good	0	0	1	0.48

Text data feature extraction

Term	Review 1	Review 2	Review 3	IDF	TF-IDF (Review 1)	TF-IDF (Review 2)	TF-IDF (Review 3)
This	1	1	1	0.00	0.000	0.000	0.000
movie	1	1	1	0.00	0.000	0.000	0.000
is	1	2	1	0.00	0.000	0.000	0.000
very	1	0	0	0.48	0.068	0.000	0.000
scary	1	1	0	0.18	0.025	0.022	0.000
and	1	1	1	0.00	0.000	0.000	0.000
long	1	0	0	0.48	0.068	0.000	0.000
not	0	1	0	0.48	0.000	0.060	0.000
slow	0	1	0	0.48	0.000	0.060	0.000
spooky	0	0	1	0.48	0.000	0.000	0.080
good	0	0	1	0.48	0.000	0.000	0.080

Text data feature extraction

```
from sklearn.feature_extraction.text import CountVectorizer  
vectorizer = CountVectorizer(stop_words='english')  
print(vectorizer.get_feature_names())
```

```
X = vectorizer.fit_transform([doc1,doc2,doc3])  
df_ bow_sklearn = pd.DataFrame(X.toarray(),columns=vectorizer.get_feature_names())  
df_ bow_sklearn.head()
```

#Optional

```
vectorizer = CountVectorizer(stop_words='english',ngram_range=(2,2))
```

Text data feature extraction

```
tfidf_vectorizer = TfidfVectorizer(input='filename', stop_words='english')  
tfidf_vector = tfidf_vectorizer.fit_transform(text_files)
```

Make a DataFrame out of the resulting tf-idf vector

```
tfidf_df = pd.DataFrame(tfidf_vector.toarray(), index=text_titles,  
columns=tfidf_vectorizer.get_feature_names())
```

Add column for document frequency = number of times word appears in all documents

```
tfidf_df.loc['00_Document Frequency'] = (tfidf_df > 0).sum()
```

drop "00_Document Frequency"

```
tfidf_df = tfidf_df.drop('00_Document Frequency', errors='ignore')
```

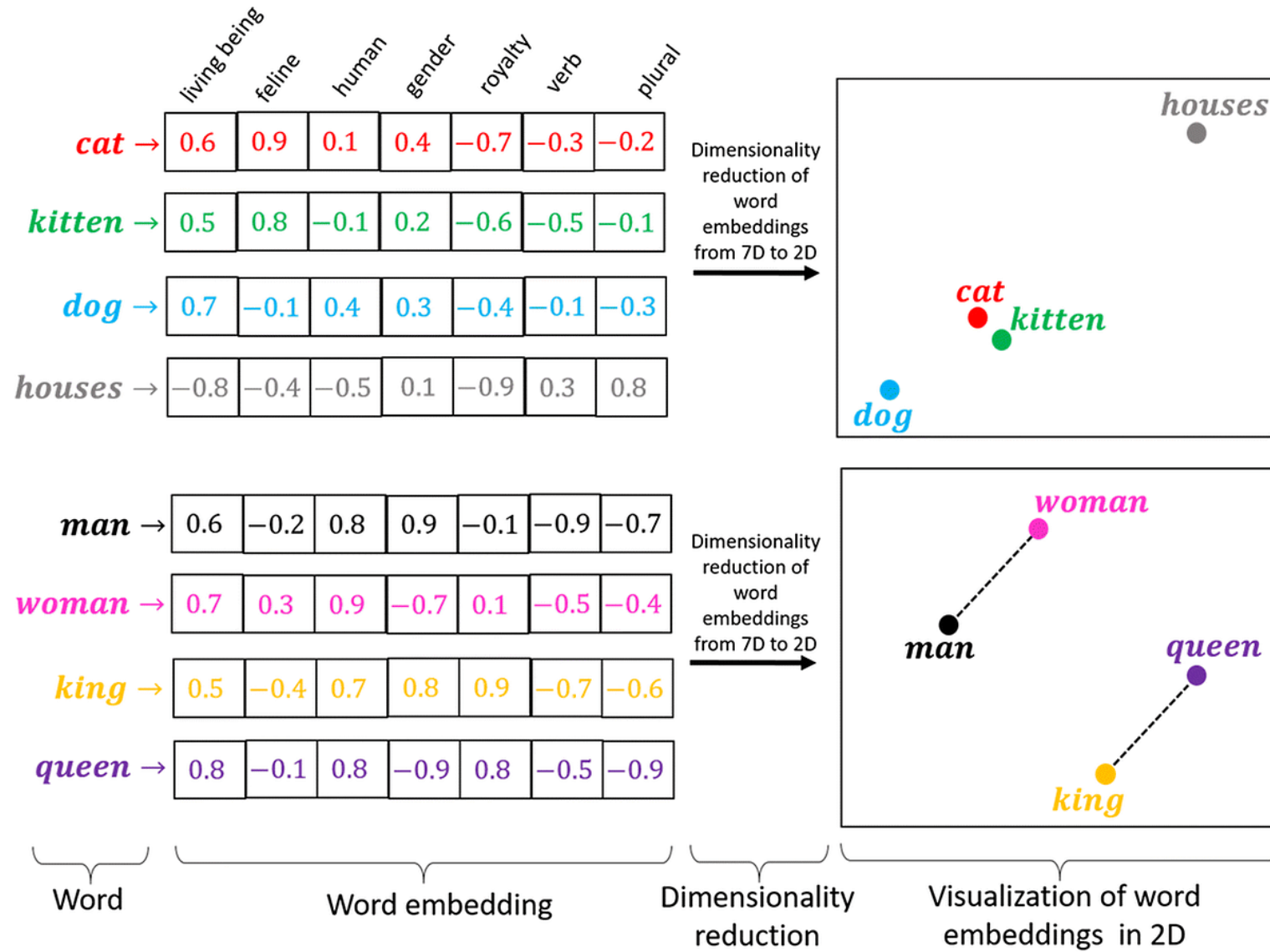
reorganize the DataFrame so that the words are in rows rather than columns.

```
tfidf_df.stack().reset_index()
```

#sort the matrix and search

```
tfidf_df.sort_values(by=['document','tfidf'], ascending=[True,False]).groupby(['document'])  
top_tfidf[top_tfidf['term'].str.contains('<example_term>')]  
top_tfidf[top_tfidf['document'].str.contains('<example_document>')]
```

Text data feature extraction

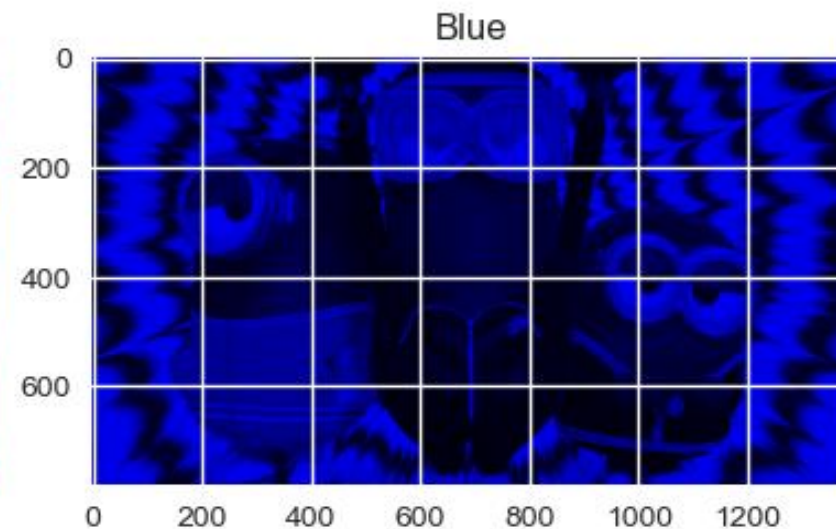
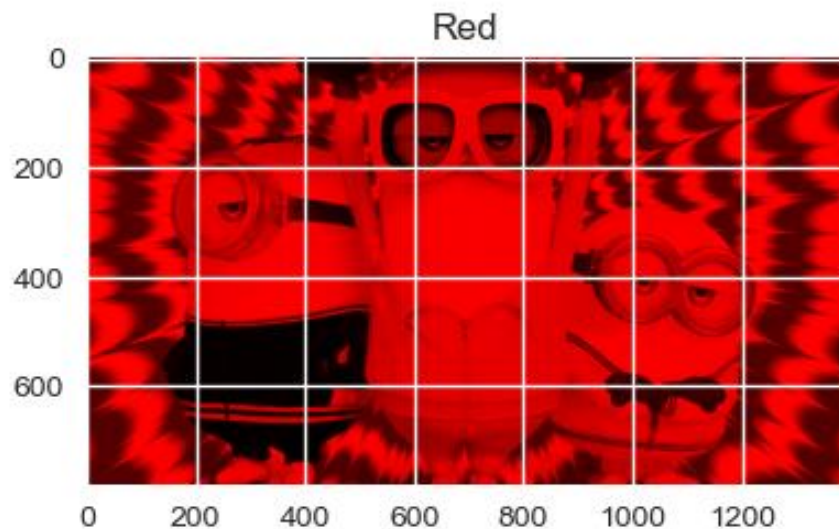
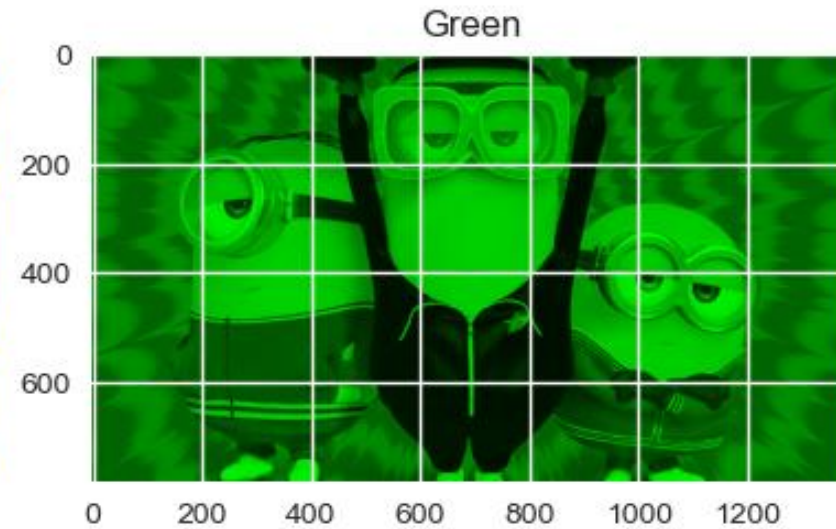
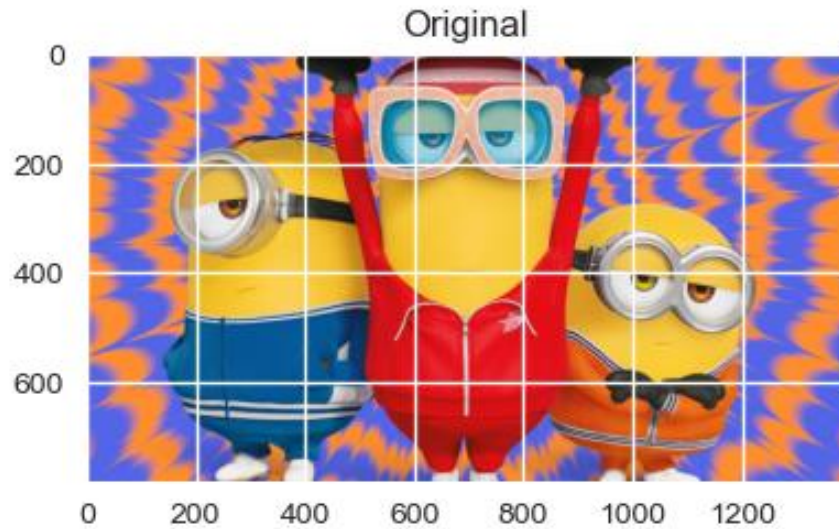


Histogram-based features and features from Images

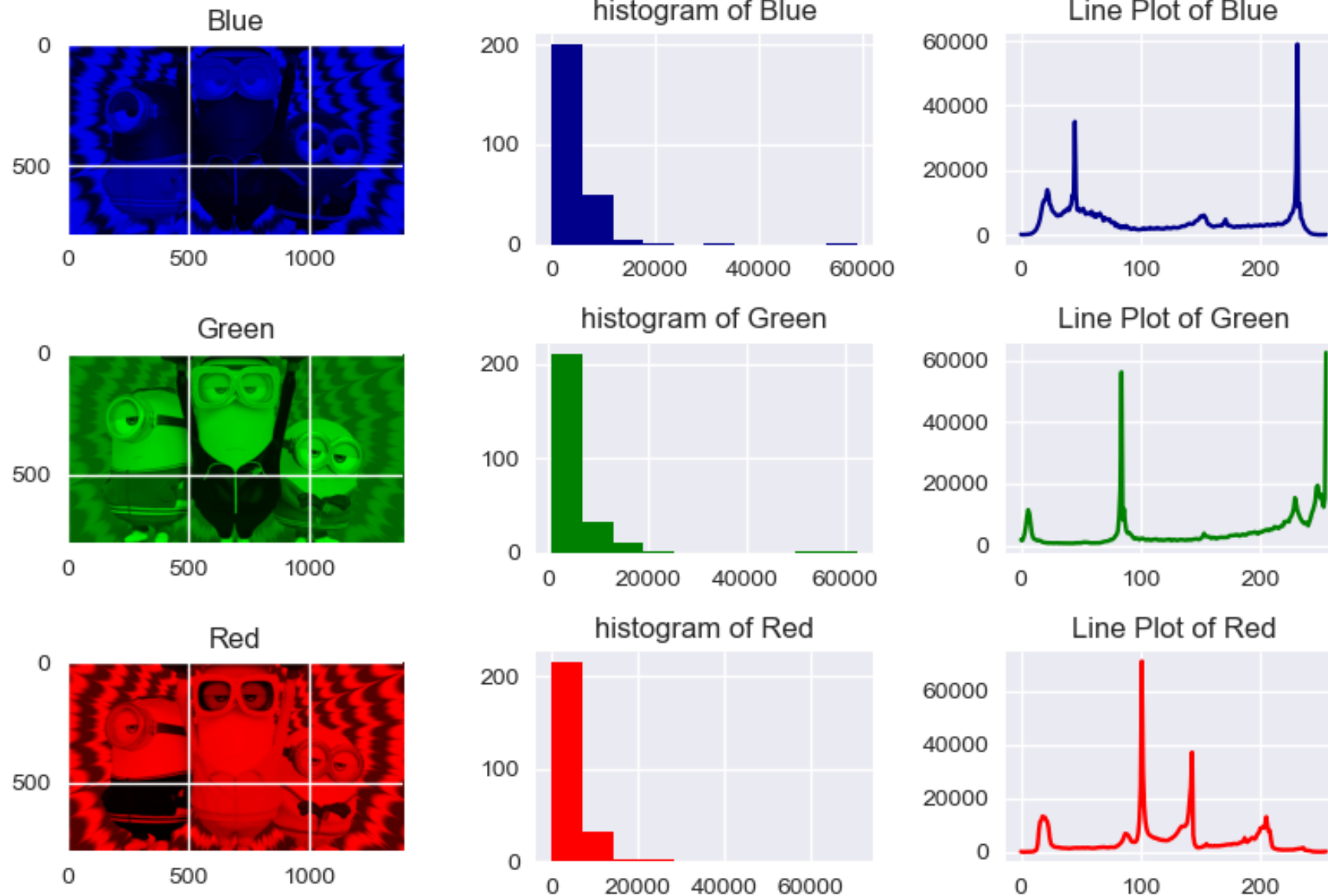
Histogram-based Features

- Colour Histogram
- Grayscale Histograms
- Local Binary Patterns
- Histogram of Oriented Gradients (HOG)
- Histogram of Local Binary Patterns (HLBP)

Histogram-based Features: Colour Histogram



Histogram-based Features: Colour Histogram



Histogram-based Features: Gray Histogram

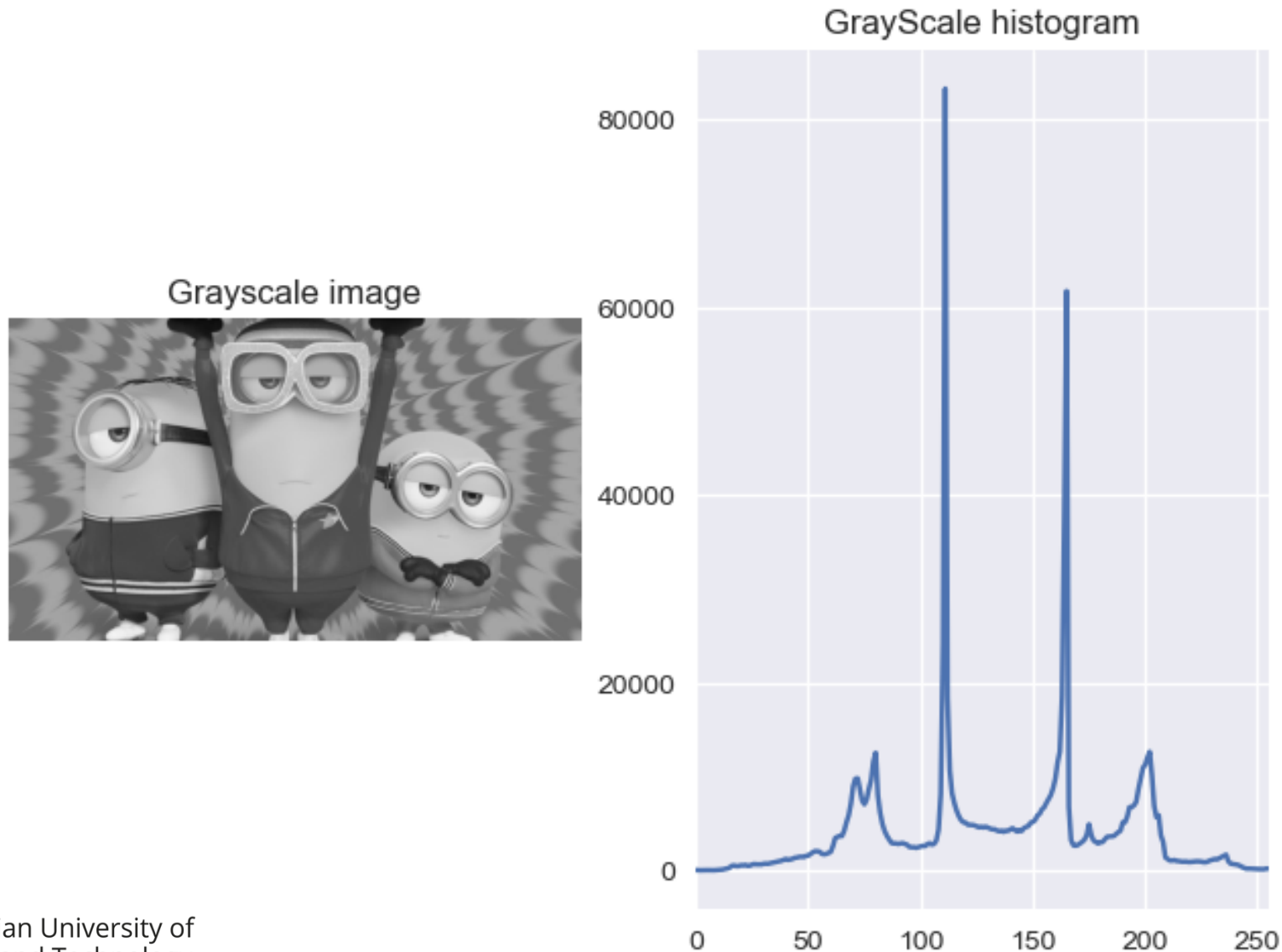
Original



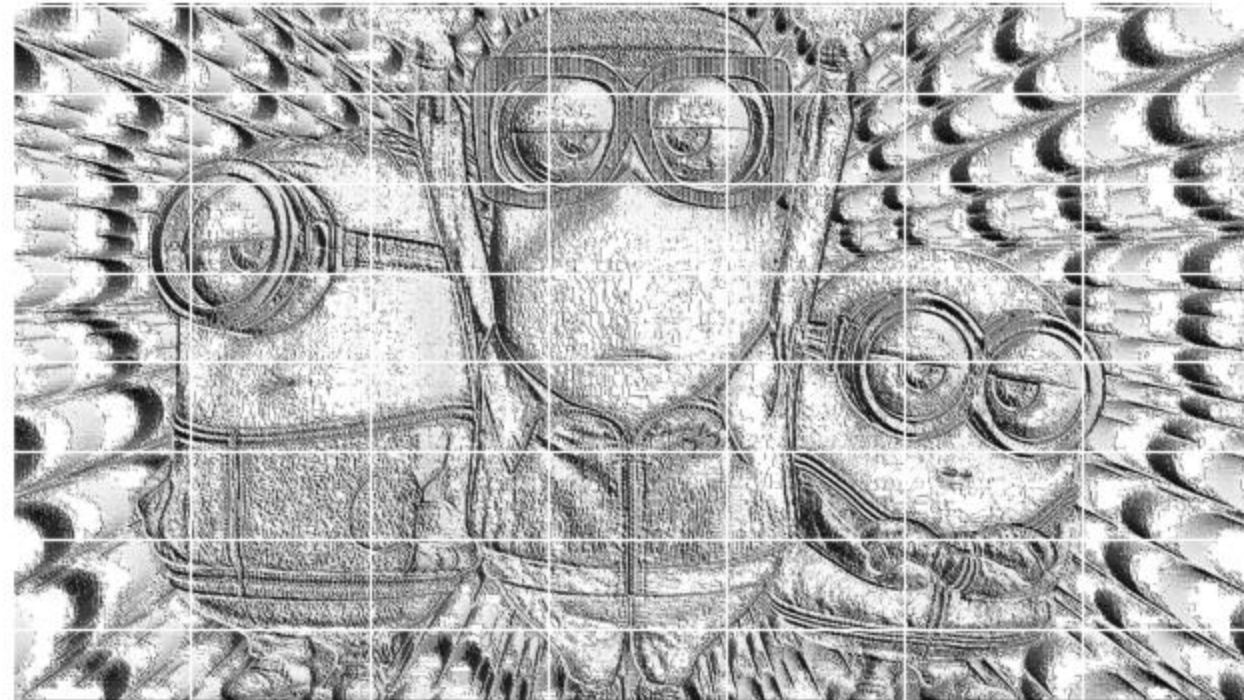
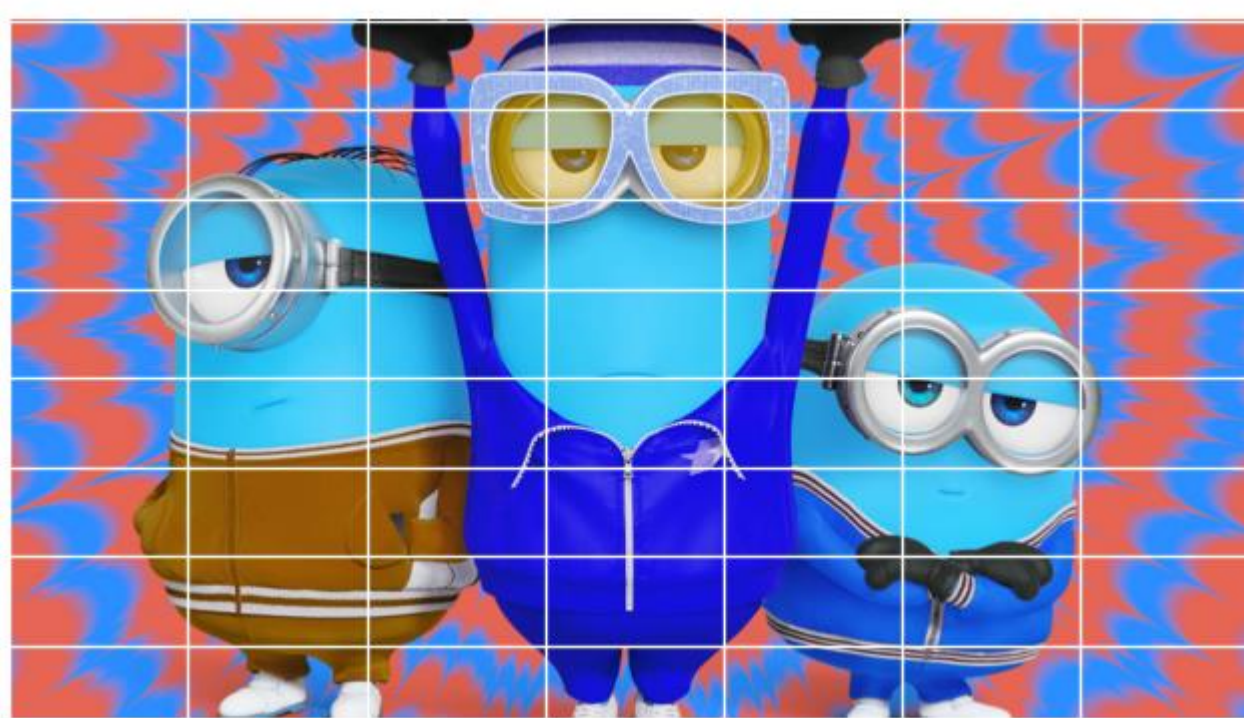
GrayScale



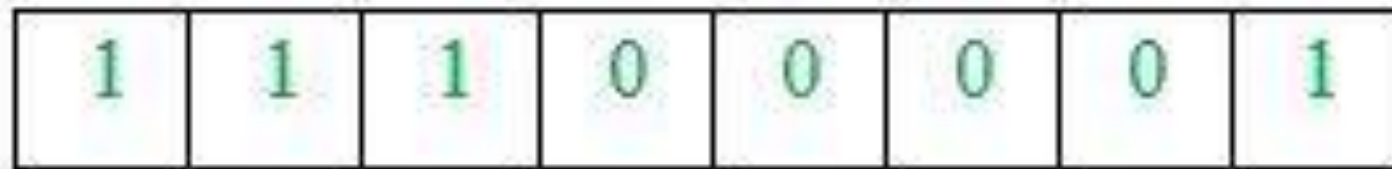
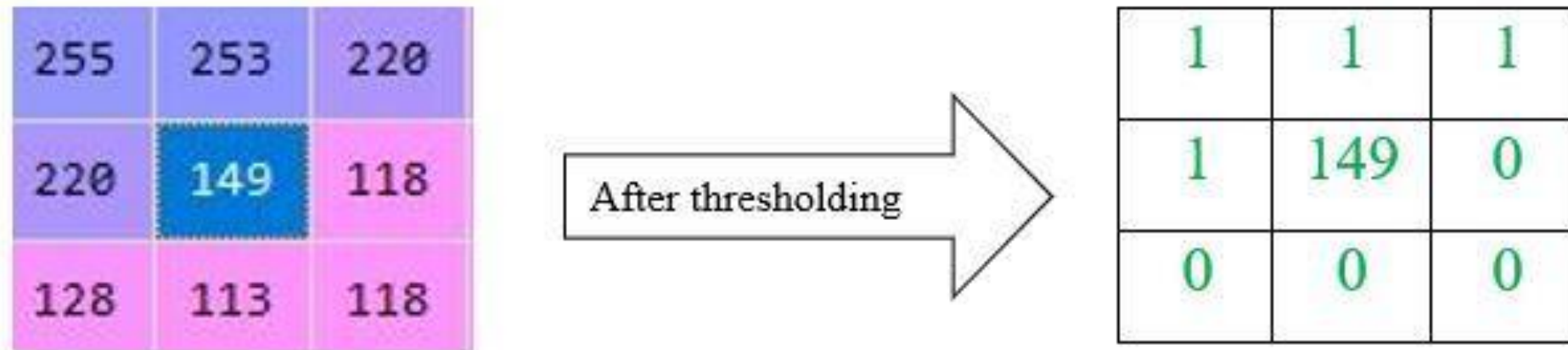
Histogram-based Features: Gray Histogram



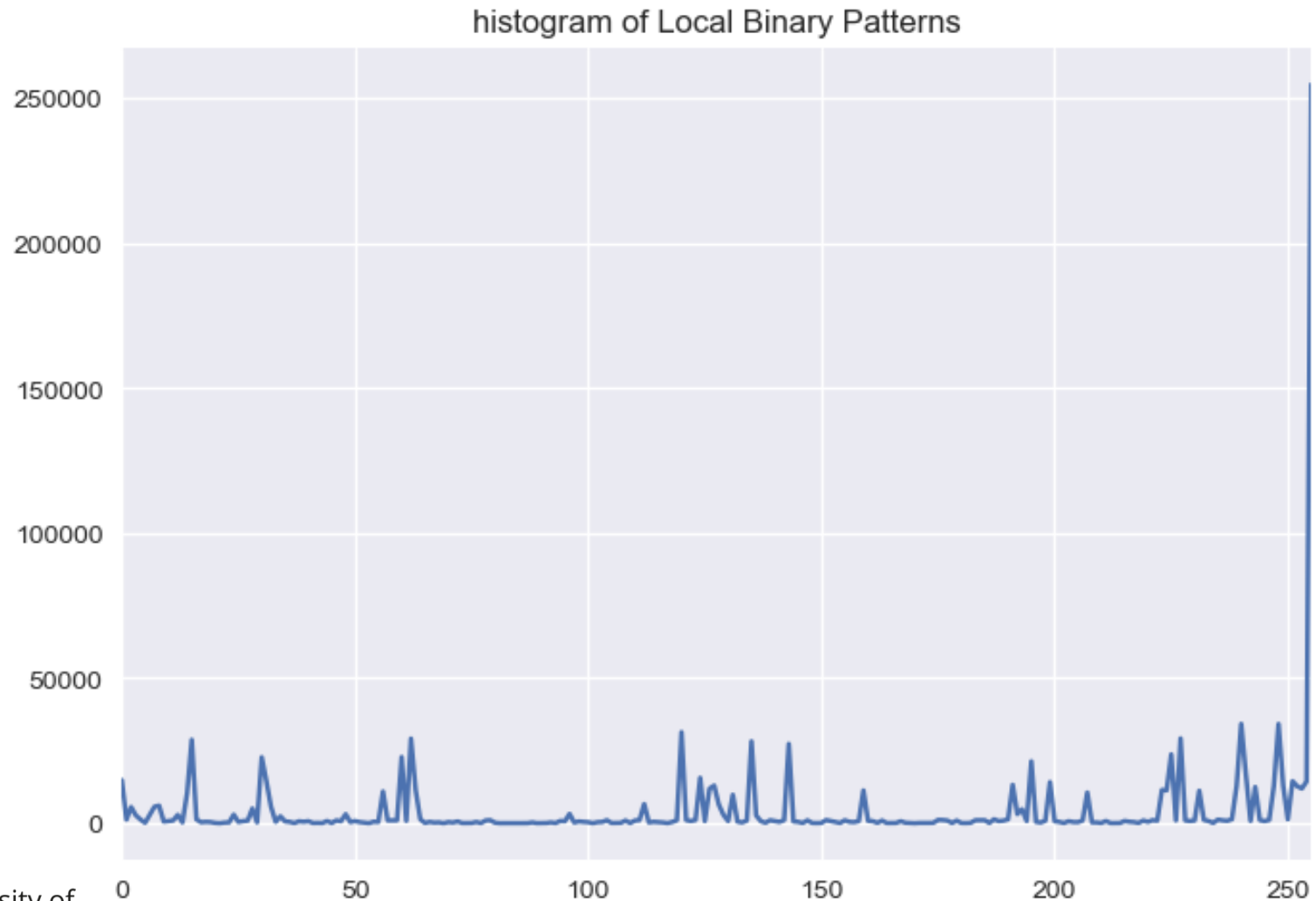
Histogram-based Features : Local Binary Patterns



Histogram-based Features: Local Binary Patterns



Histogram-based Features : Histogram of Local Binary Patterns

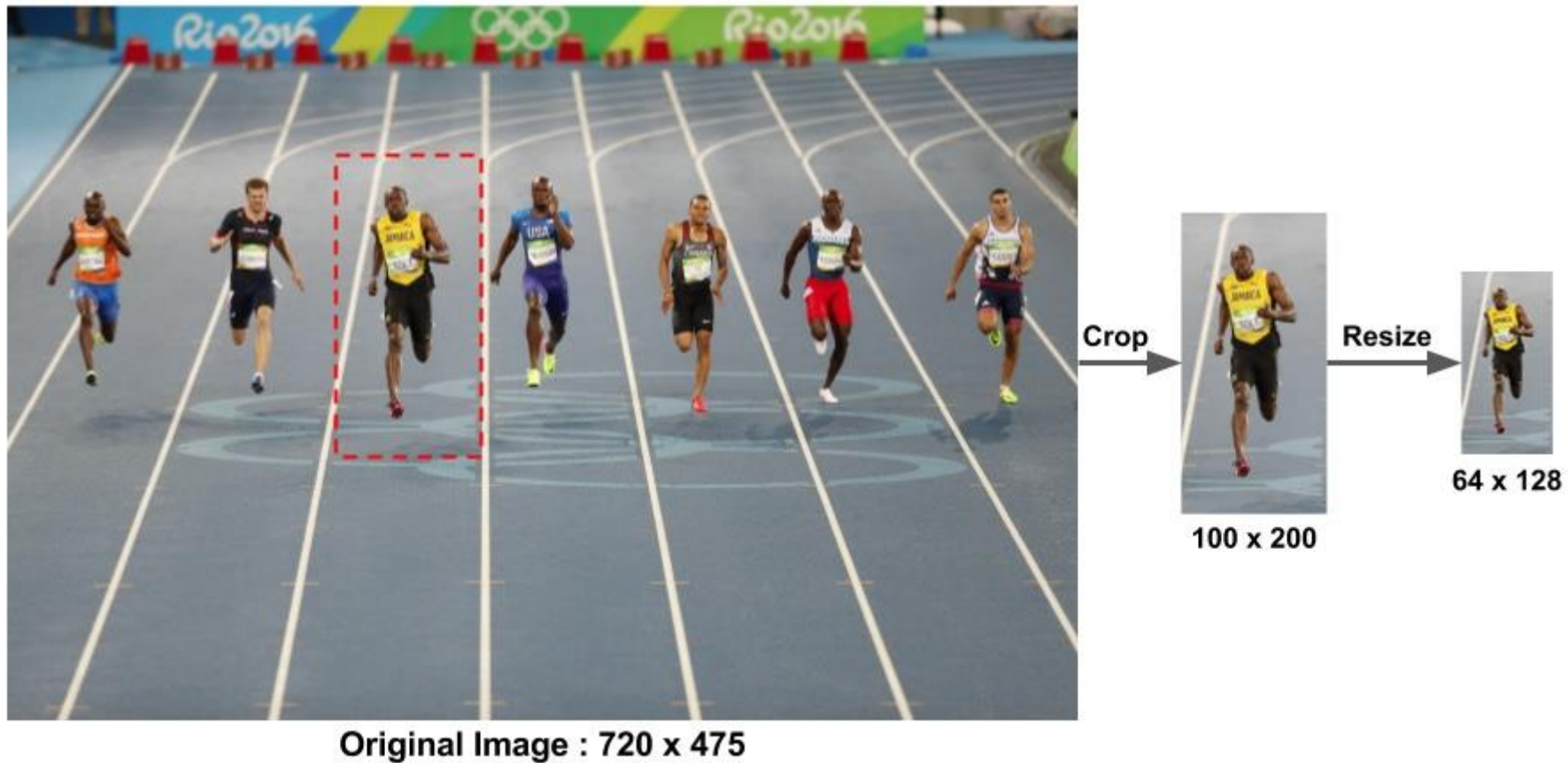


Histogram-based Features: HOG

- HOG focuses on the structure of the object. It extracts the information of the edges magnitude as well as the orientation of the edges.
- It uses a detection window of 64x128 pixels, so the image is first converted into (64, 128) shape.
- The image is then further divided into small parts, and then the gradient and orientation of each part is calculated. It is divided into 8x16 cells into blocks with 50% overlap, so there are going to be $7 \times 15 = 105$ blocks in total, and each block consists of 2x2 cells with 8x8 pixels.
- We take the 64 gradient vectors of each block (8x8 pixel cell) and put them into a 9-bin histogram.

Histogram-based Features: HOG

Step 1 : Preprocessing



Histogram-based Features: HOG

Step 2 : Calculate the Gradient Images



X-gradient



Y-gradient



Gradient
Magnitude

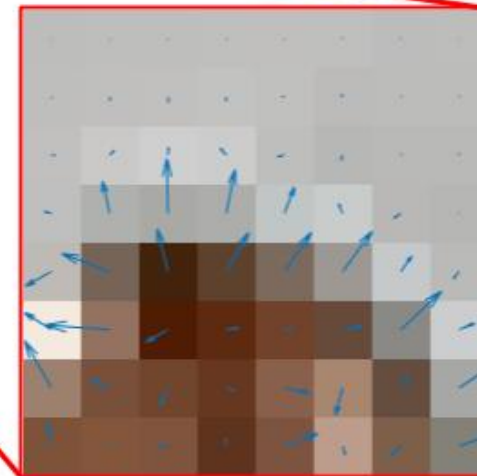
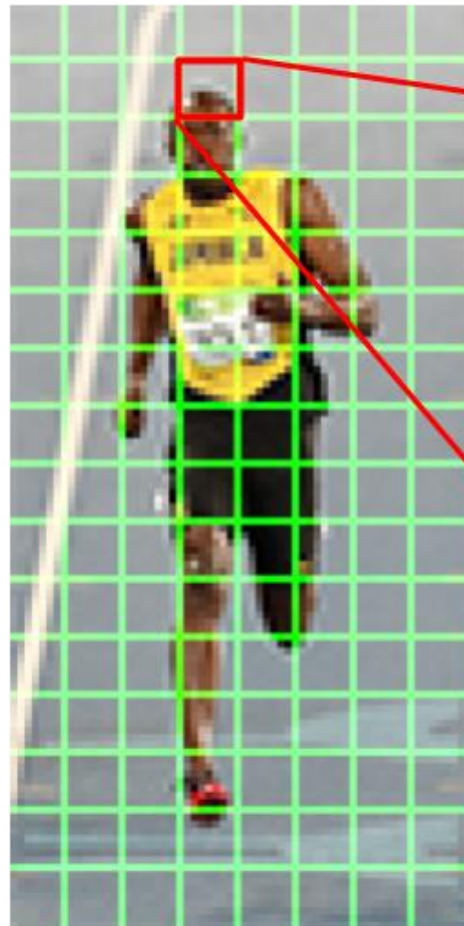
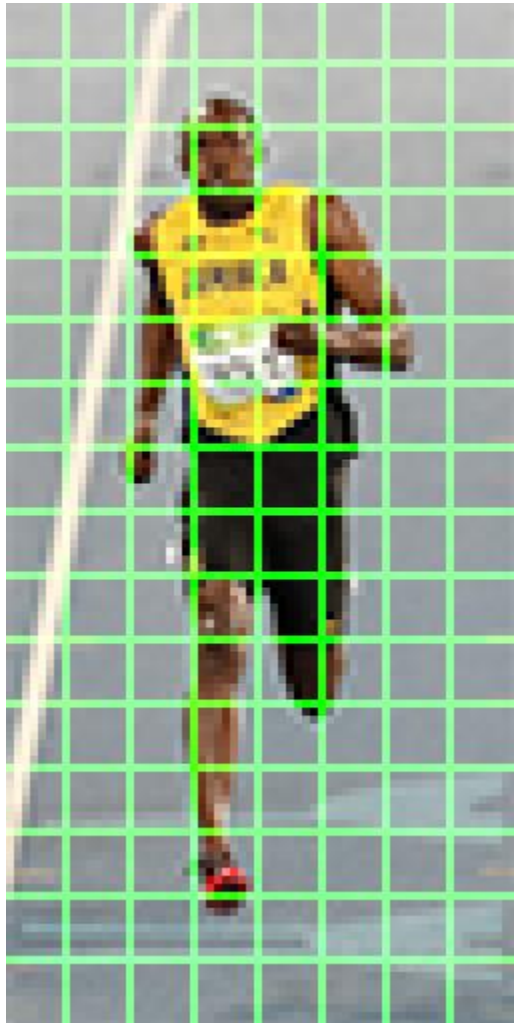
-1	0	1
----	---	---

Sobel Filter

-1
0
1

Histogram-based Features: HOG

Step 3 : Calculate Histogram of Gradients in 8×8 cells



2	3	4	4	3	4	2	2
5	11	17	13	7	9	3	4
11	21	23	27	22	17	4	6
23	99	165	135	85	32	26	2
91	155	133	136	144	152	57	28
98	196	76	38	26	60	170	51
165	60	60	27	77	85	43	136
71	13	34	23	108	27	48	110

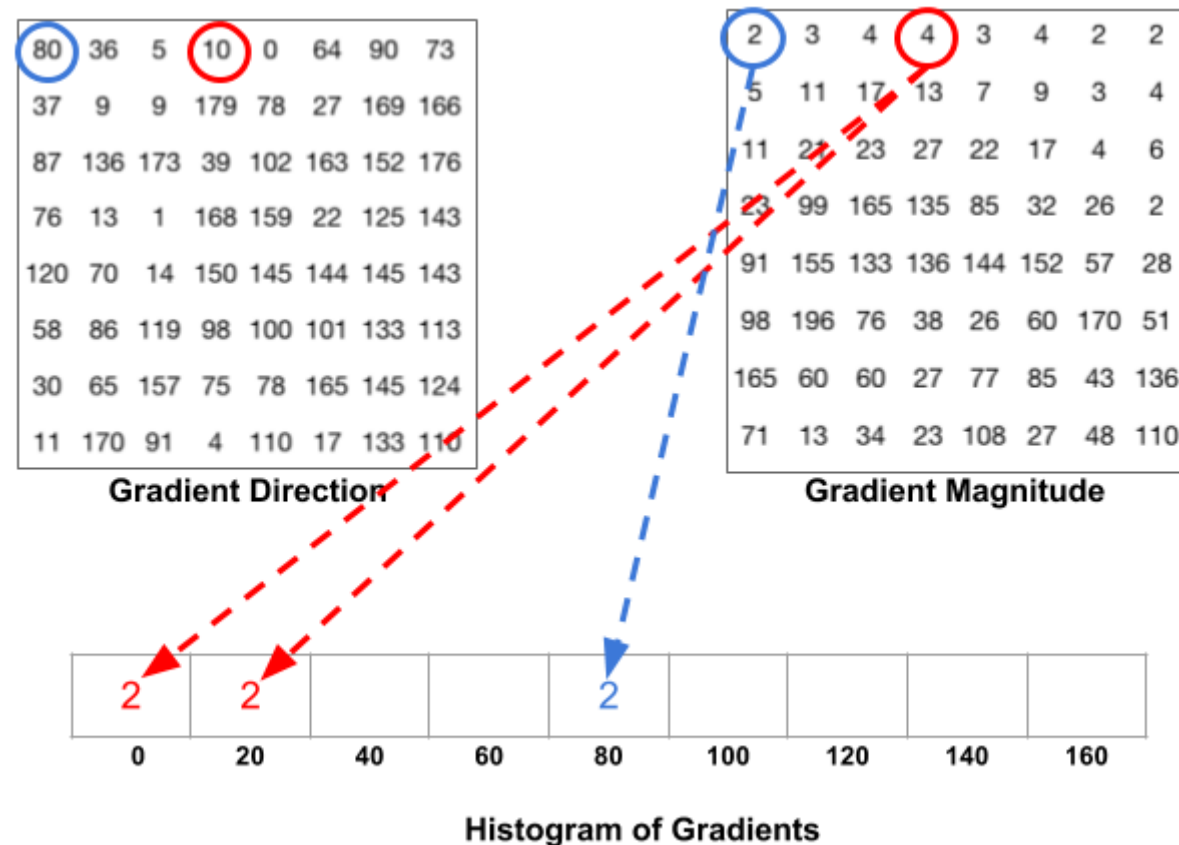
Gradient Magnitude

80	36	5	10	0	64	90	73
37	9	9	179	78	27	169	166
87	136	173	39	102	163	152	176
76	13	1	168	159	22	125	143
120	70	14	150	145	144	145	143
58	86	119	98	100	101	133	113
30	65	157	75	78	165	145	124
11	170	91	4	110	17	133	110

Gradient Direction

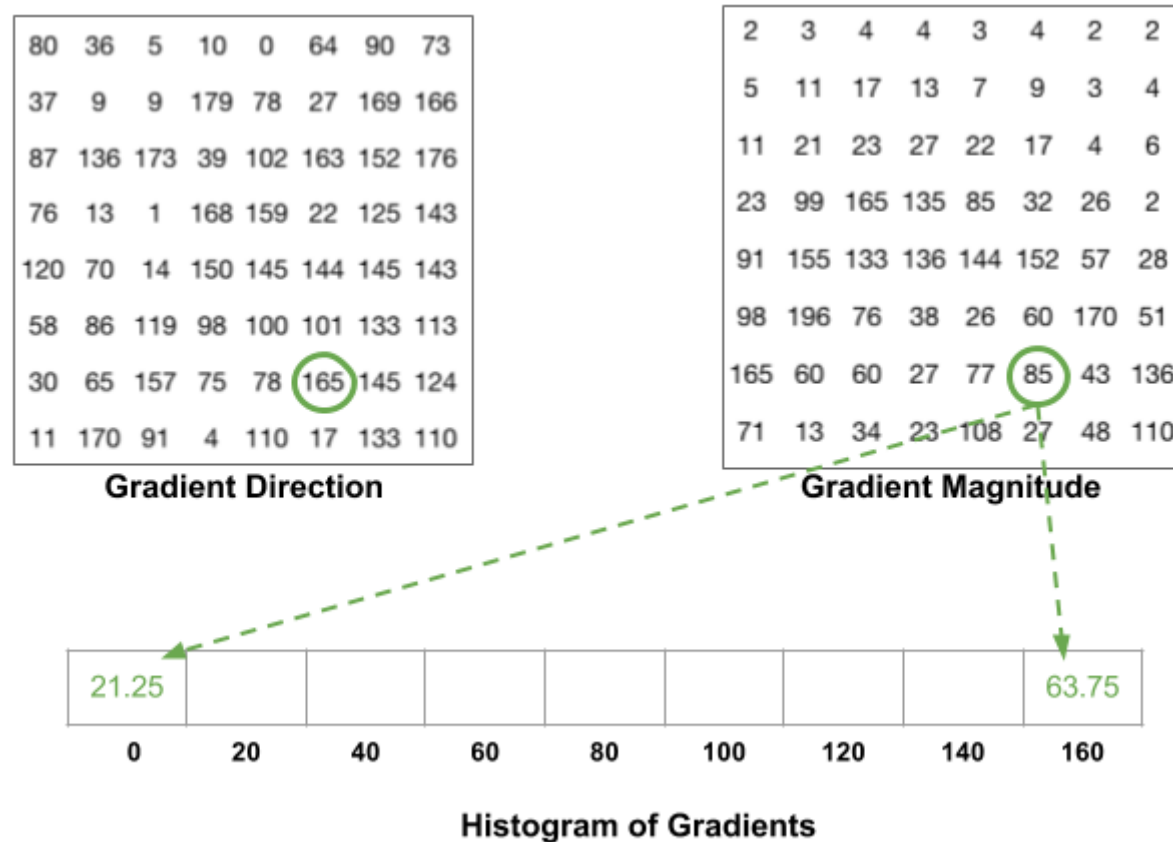
Histogram-based Features: HOG

Step 3 : Calculate Histogram of Gradients in 8×8 cells



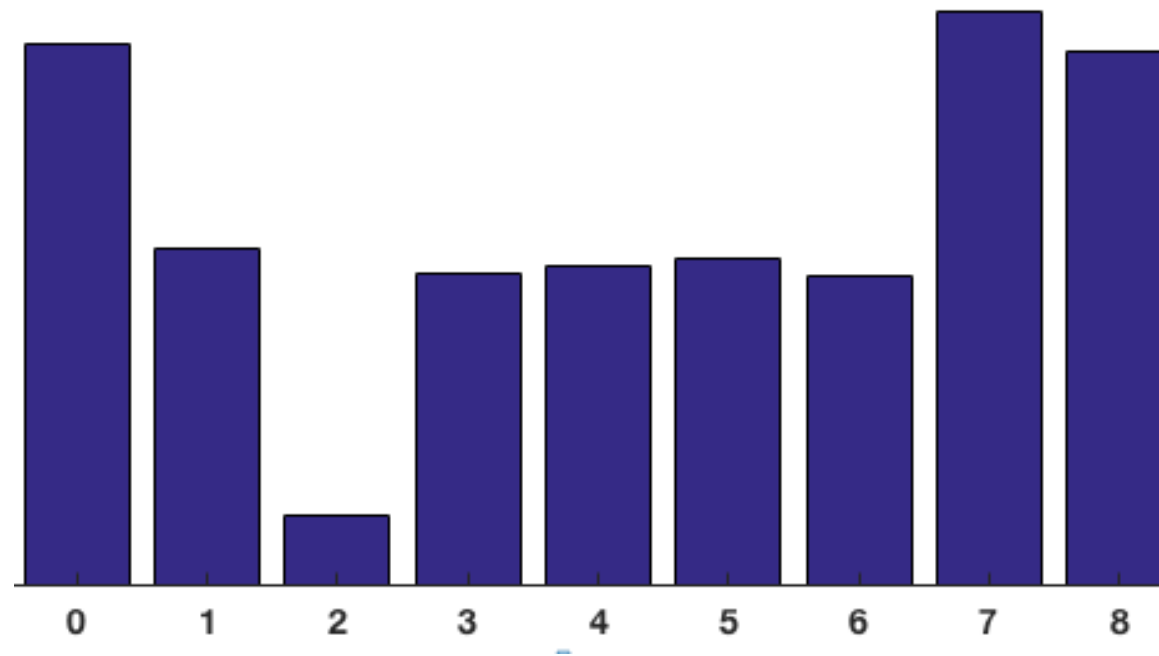
Histogram-based Features: HOG

Step 3 : Calculate Histogram of Gradients in 8×8 cells



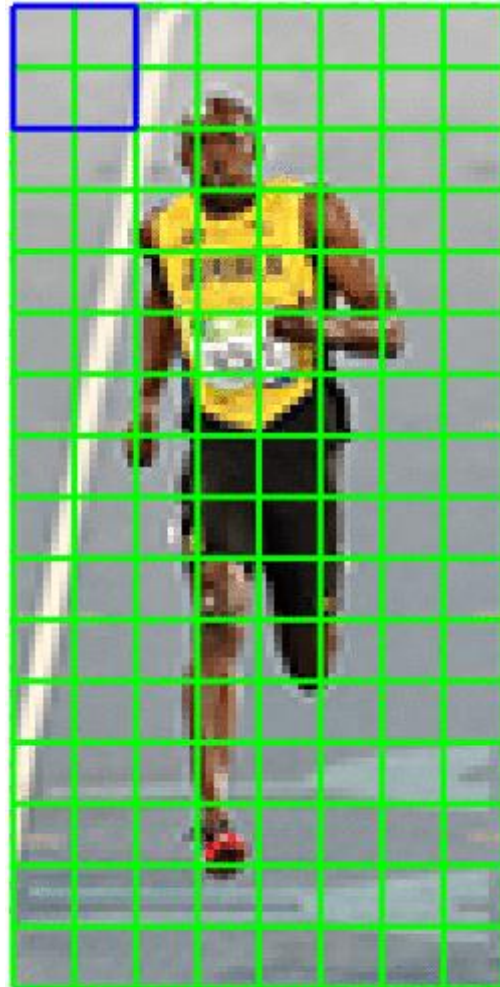
Histogram-based Features: HOG

Step 3 : Calculate Histogram of Gradients in 8×8 cells



Histogram-based Features: HOG

Step 4 : 16×16 Block Normalization



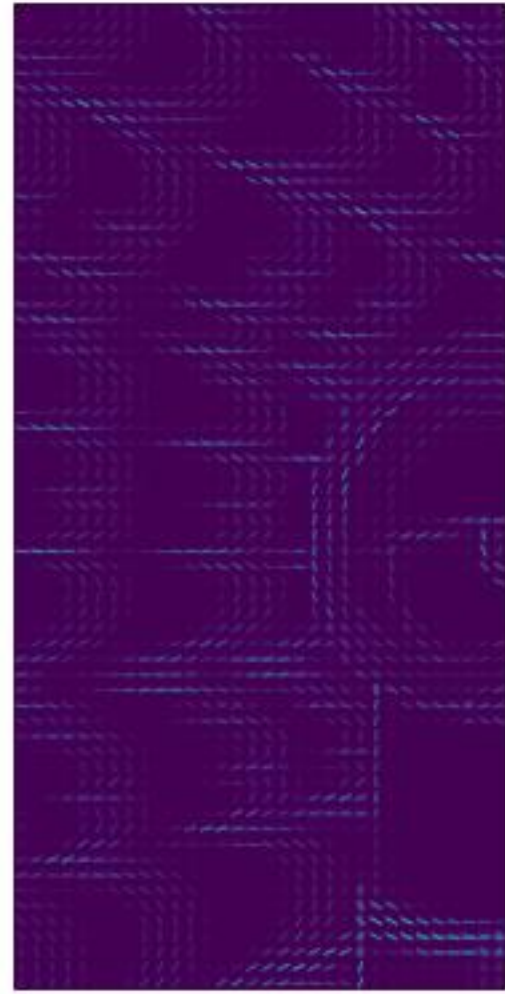
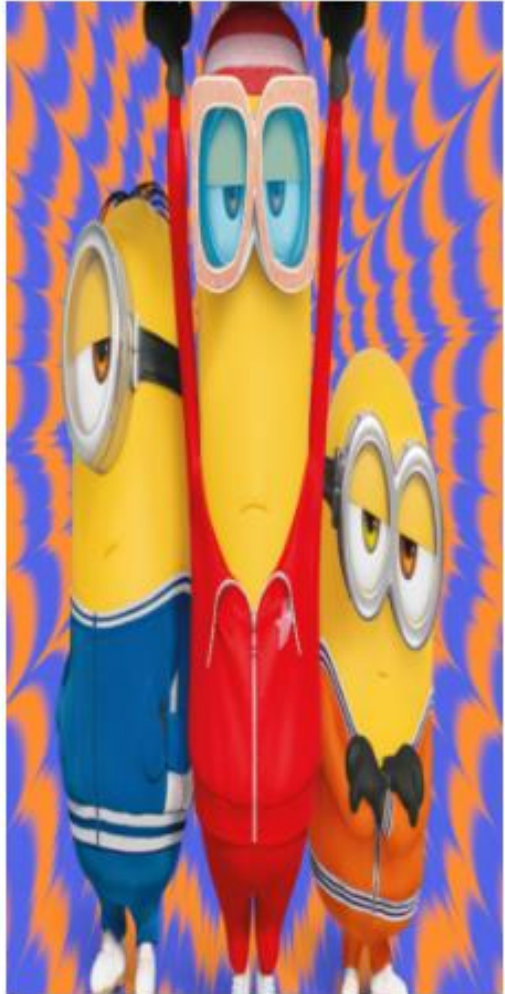
Histogram-based Features: HOG

Step 5 : Calculate the Histogram of Oriented Gradients feature vector

How many positions of the 16×16 blocks do we have ? There are 7 horizontal and 15 vertical positions making a total of $7 \times 15 = 105$ positions.

Each 16×16 block is represented by a 36×1 vector. So when we concatenate them all into one gaint vector we obtain a $36 \times 105 = \mathbf{3780}$ dimensional vector.

Histogram-based Features: HOG



Histogram-based Features: HOG

#orientations = 8



Image Features

- Pixel Values
- Colour/Greyscale Histogram
- Edge Detection
- Corner Detection
- Blob Detection
- Scale invariant feature transformation (SIFT)
- Histogram of Oriented Gradients (HOG)
- Principal Component Analysis (PCA)
- Deep-Learning features (**Later in the course**)

Image Features: pixel values



Image Features: edge detection

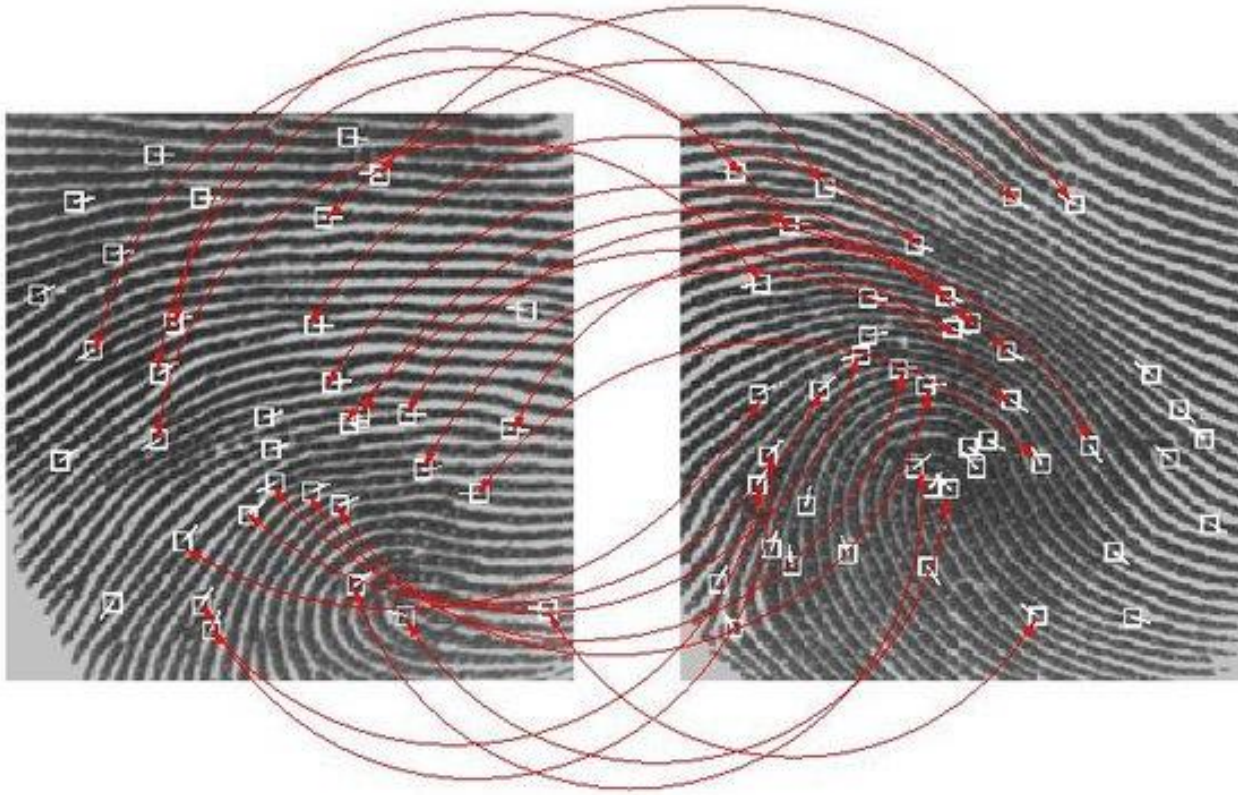


Image Features: Canny edge detection

- Low error rate: Meaning a good detection of only existent edges.
- Good localization: The distance between edge pixels detected and real edge pixels have to be minimized.
- Minimal response: Only one detector response per edge.

Image Features: Canny edge detection

Filter out any noise

Gaussian kernel of size=5

$$K = \frac{1}{159} \begin{bmatrix} 2 & 4 & 5 & 4 & 2 \\ 4 & 9 & 12 & 9 & 4 \\ 5 & 12 & 15 & 12 & 5 \\ 4 & 9 & 12 & 9 & 4 \\ 2 & 4 & 5 & 4 & 2 \end{bmatrix}$$

Image Features: Canny edge detection

Filter out any noise



Image Features: Canny edge detection

Find the intensity gradient of the image.

Apply a pair of convolution masks
(in x and y directions)

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

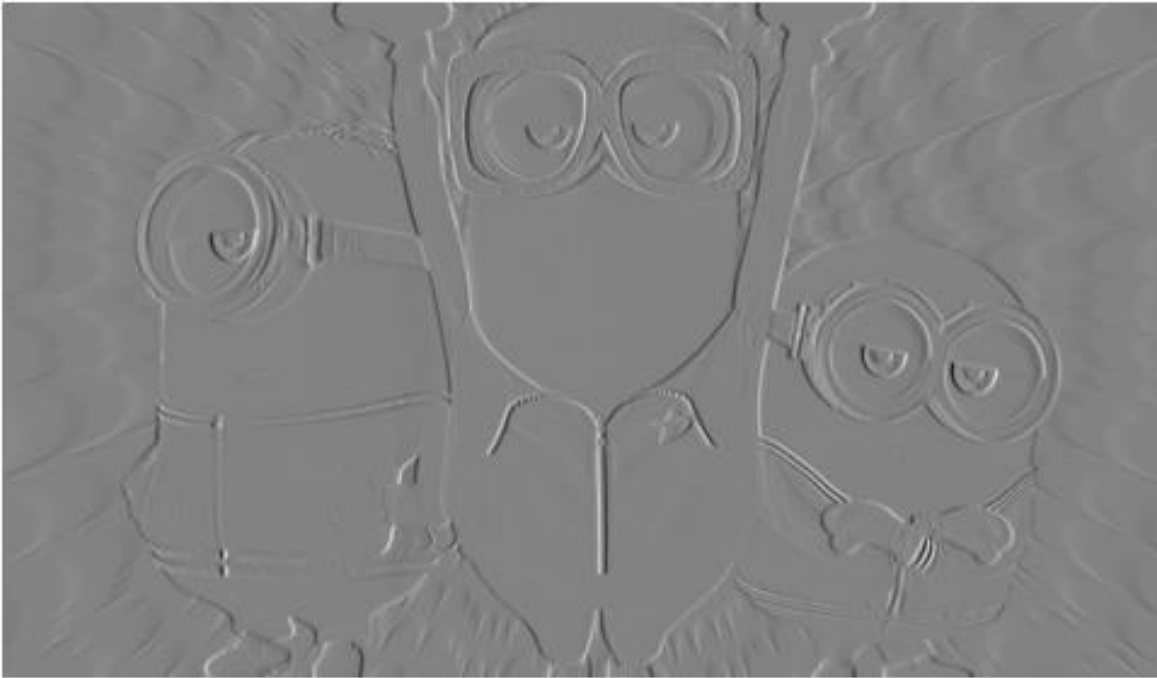
$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Find the gradient strength
and direction with

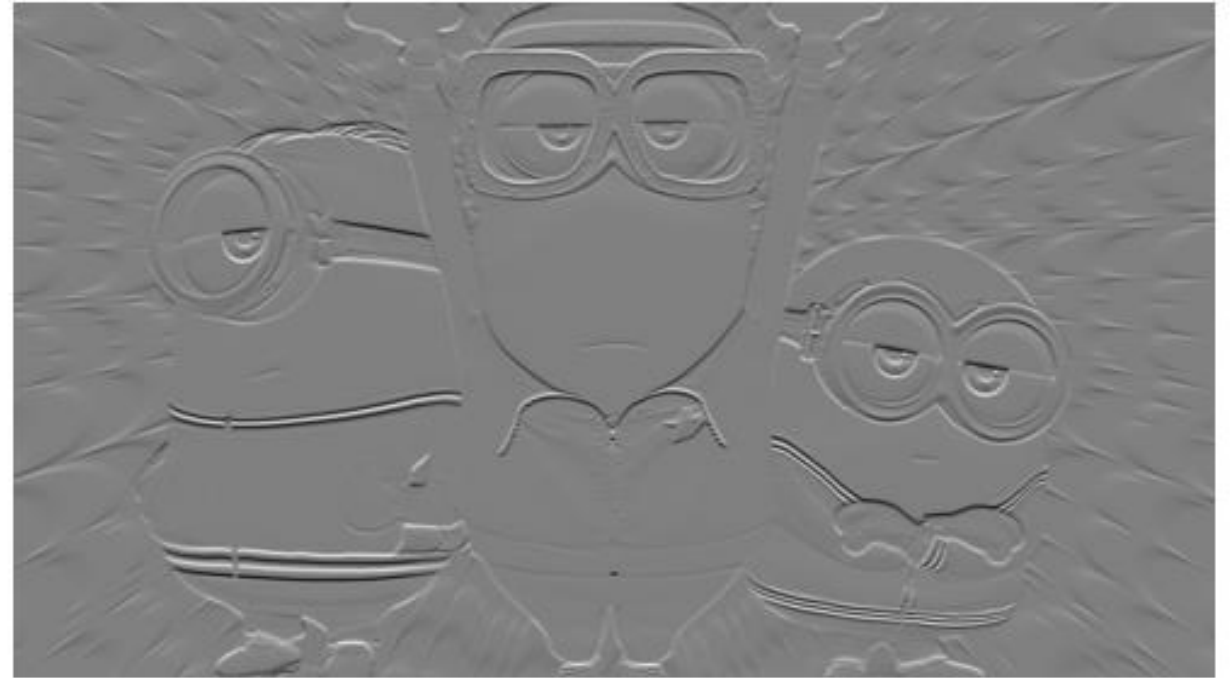
$$G = \sqrt{G_x^2 + G_y^2}$$

$$\theta = \arctan\left(\frac{G_y}{G_x}\right)$$

Image Features: Canny edge detection



$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$



$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Image Features: Canny edge detection



$$G = \sqrt{G_x^2 + G_y^2}$$

$$\theta = \arctan\left(\frac{G_y}{G_x}\right)$$

Image Features: Canny edge detection

Non-maximum suppression is applied. This removes pixels that are not considered to be part of an edge. Hence, only thin lines (candidate edges) will remain.

Image Features: Canny edge detection

Hysteresis: The final step. Canny does use two thresholds (upper and lower):

- If a pixel gradient is higher than the upper threshold, the pixel is accepted as an edge
- If a pixel gradient value is below the lower threshold, then it is rejected.
- If the pixel gradient is between the two thresholds, then it will be accepted only if it is connected to a pixel that is above the upper threshold.

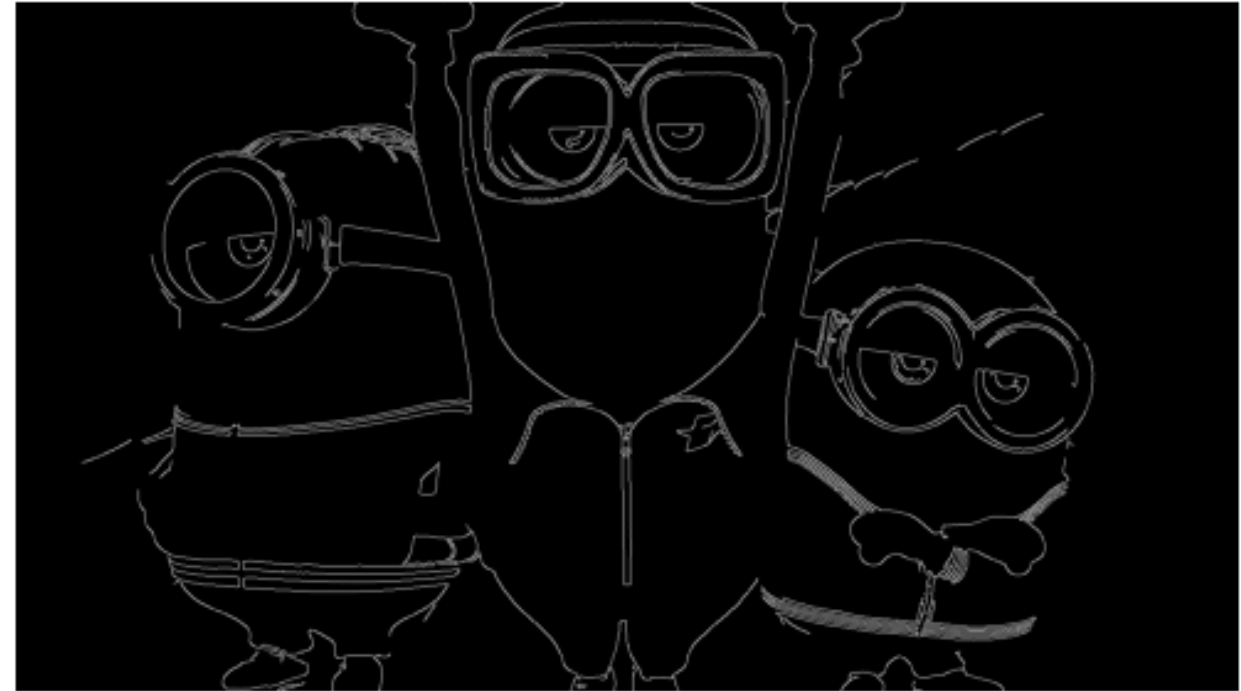
Canny recommended a upper:lower ratio between 2:1 and 3:1.

Image Features: Canny Edge detection

original



Canny Edge Detection



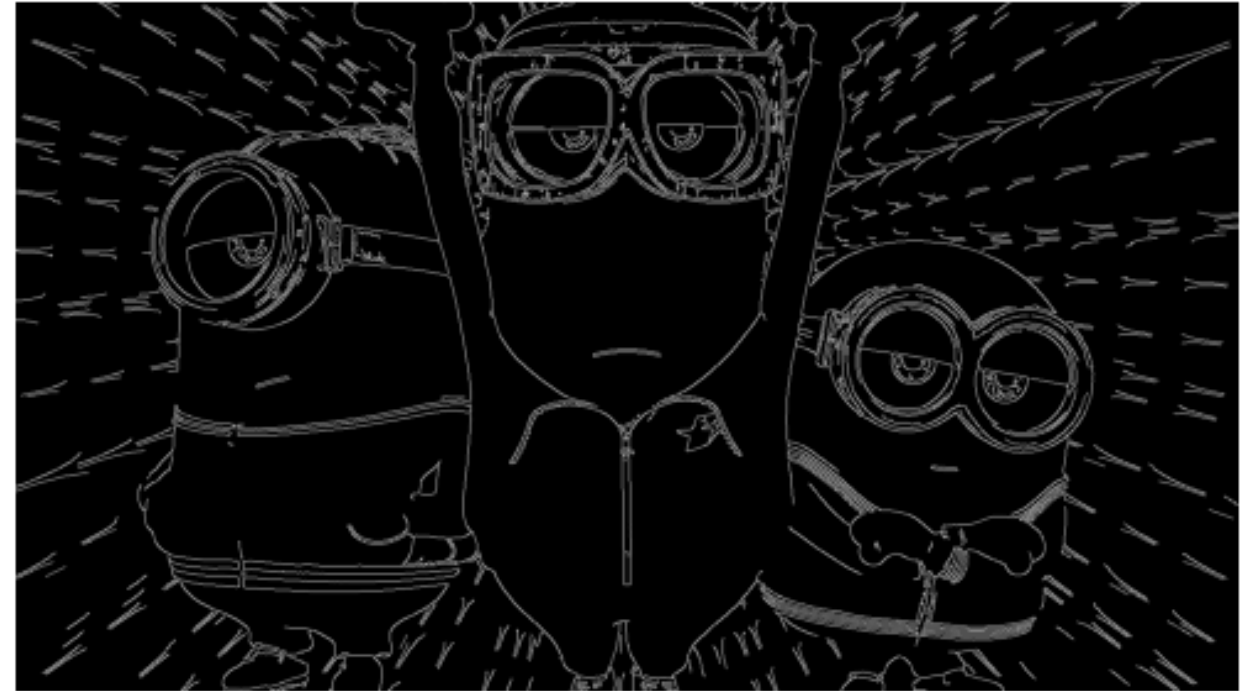
```
cv2.Canny(image=img_blur, threshold1=100, threshold2=200)
```


Image Features: Canny Edge detection

original



Canny Edge Detection



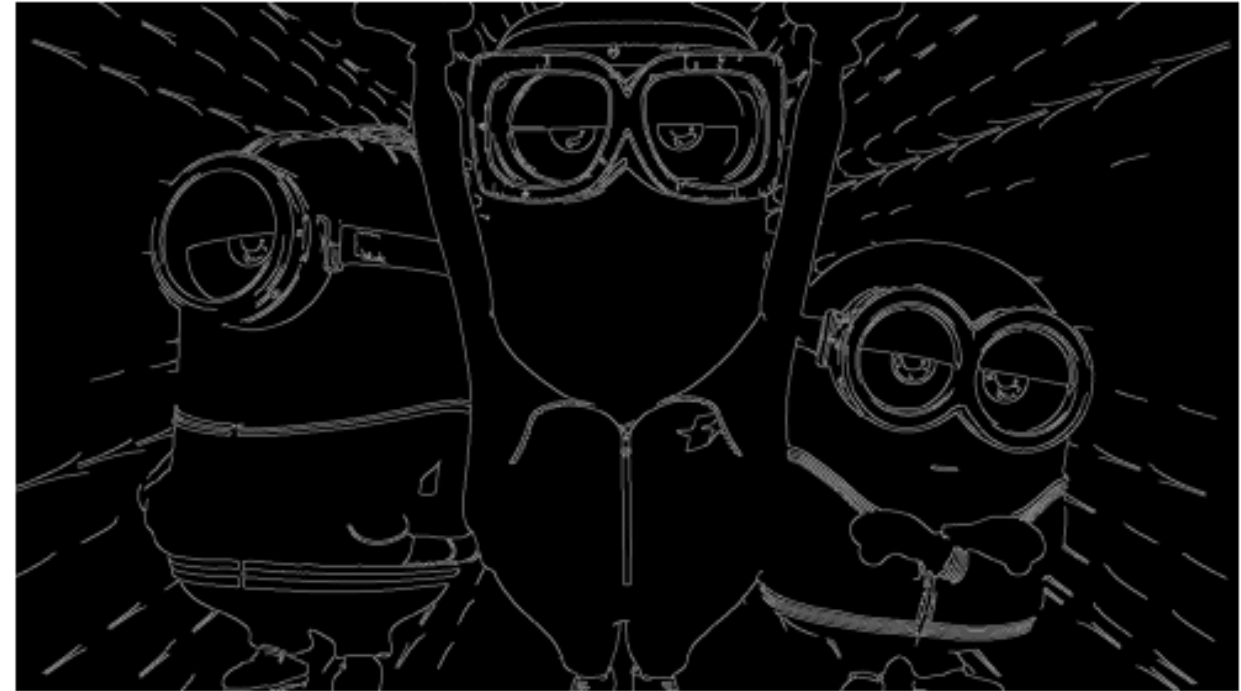
```
cv2.Canny(image=img_blur, threshold1=50, threshold2=100)
```


Image Features: Canny Edge detection

original



Canny Edge Detection



```
cv2.Canny(image=img_blur, threshold1=50, threshold2=150)
```

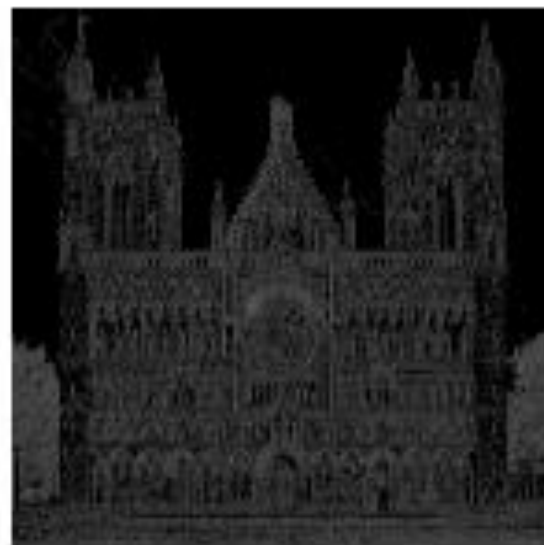
Original



Blurred image



Canny Edge
 $t1 = 10$ $t2 = 20$



Canny Edge
 $t1 = 20$ $t2 = 40$



Canny Edge
 $t1 = 50$ $t2 = 100$



Canny Edge
 $t1 = 100$ $t2 = 200$



Image Features: Sobel edge detection

Sobel Edge Detection is one of the most widely used algorithms for edge detection. The Sobel Operator detects edges marked by sudden changes in pixel intensity

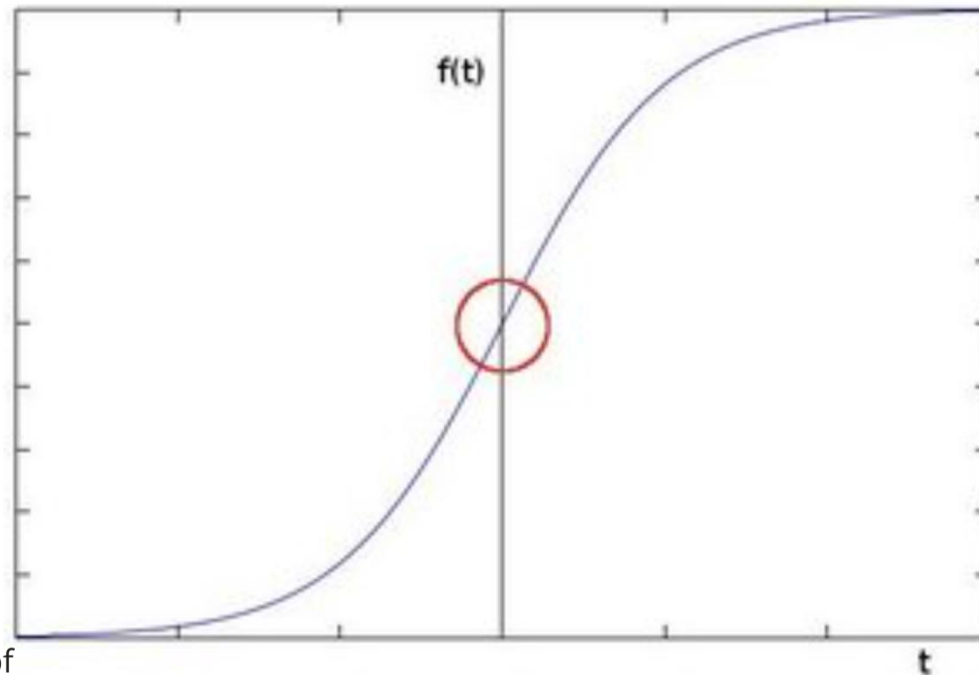


Image Features: Sobel edge detection

The edge "jump" can be seen more easily if we take the first derivative

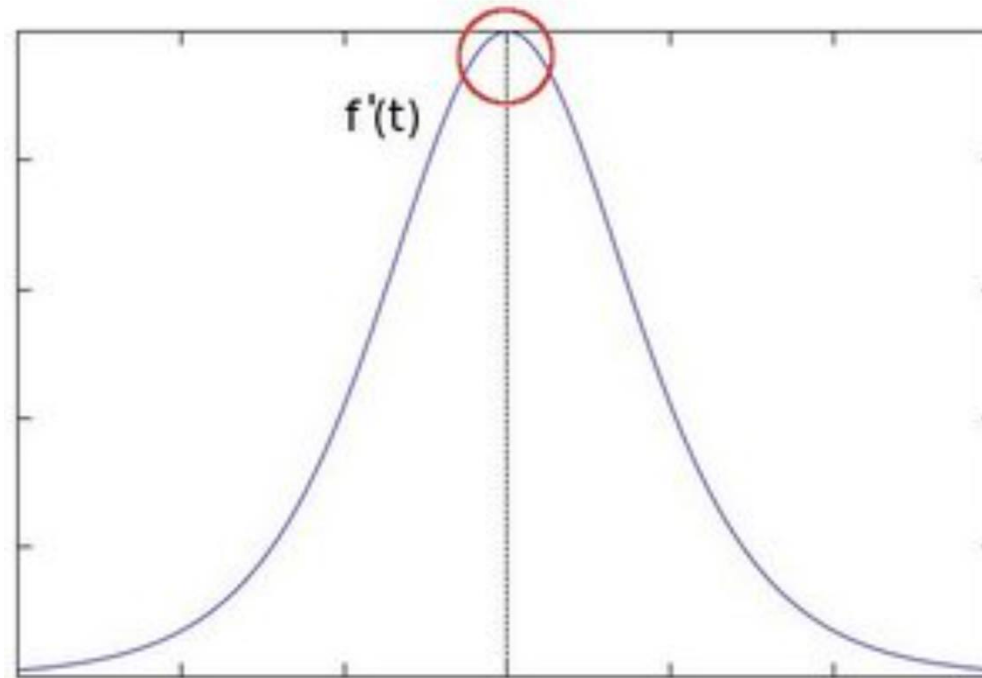


Image Features: Sobel edge detection

Find the intensity gradient of the image.

Apply a pair of convolution masks
(in x and y directions)

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

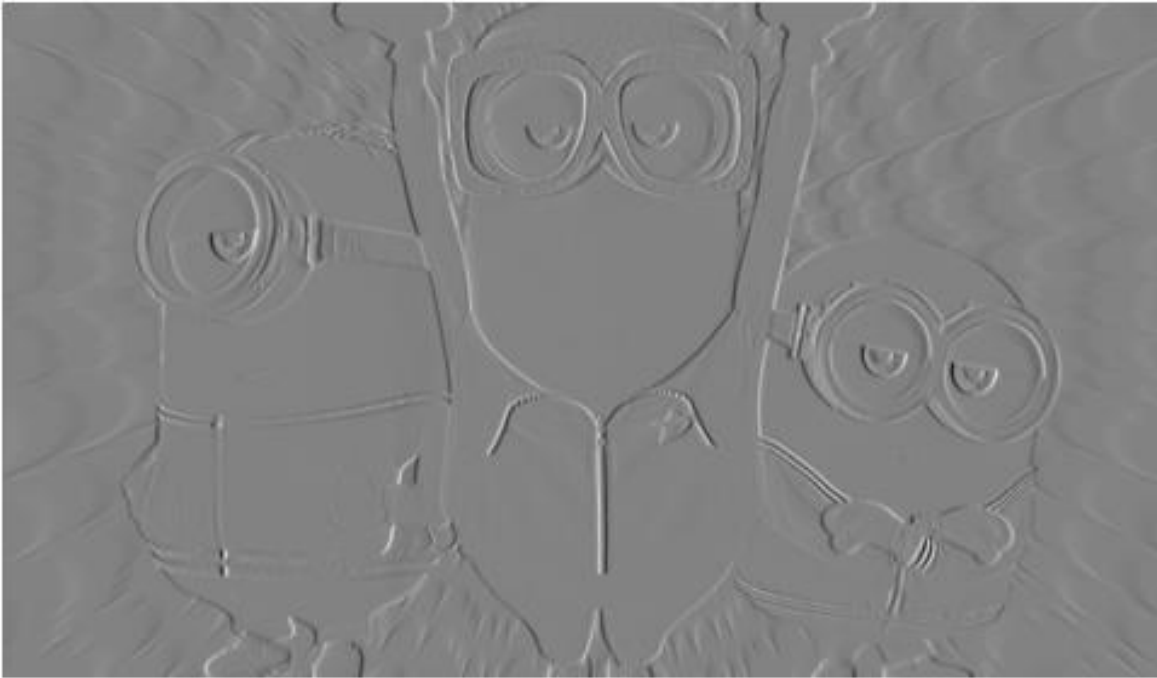
$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Find the gradient strength
and direction with

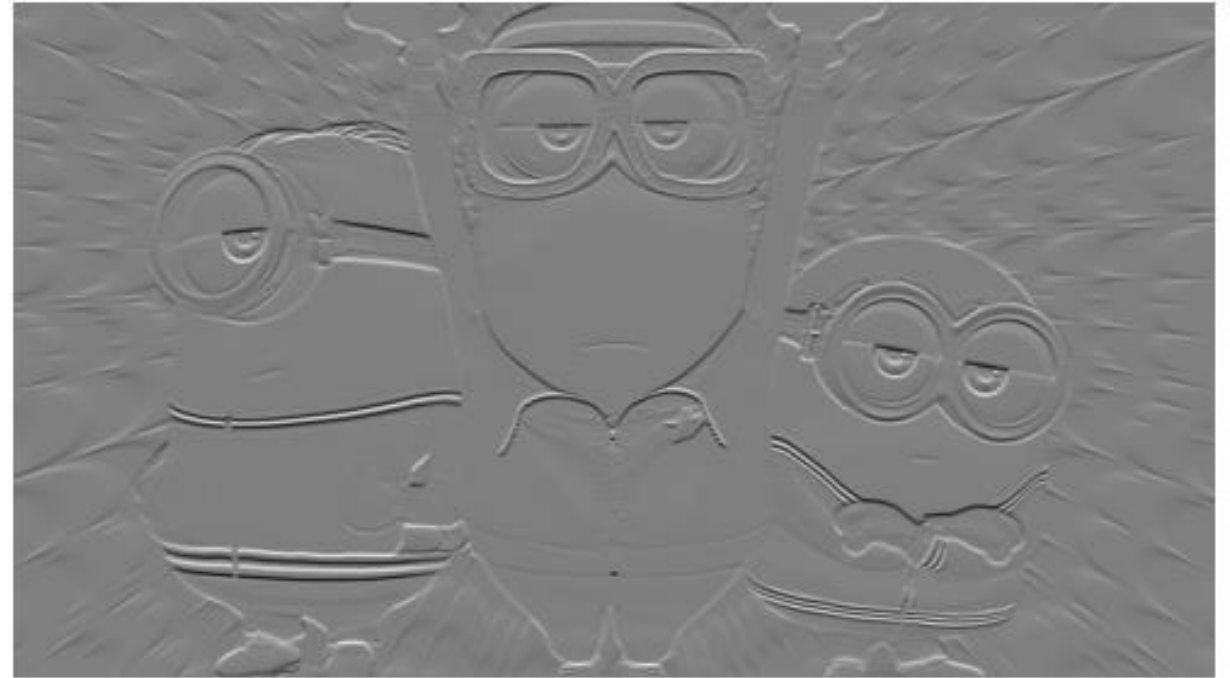
$$G = \sqrt{G_x^2 + G_y^2}$$

$$G = |G_x| + |G_y|$$

Image Features: Sobel edge detection



$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$



$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Image Features: Sobel edge detection



$$G = \sqrt{G_x^2 + G_y^2}$$

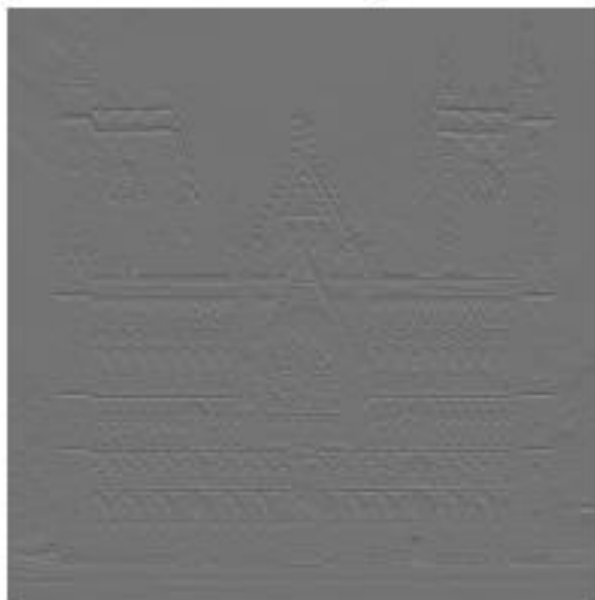
GrayScale Original



sobel x



sobel y



Sobel X Y

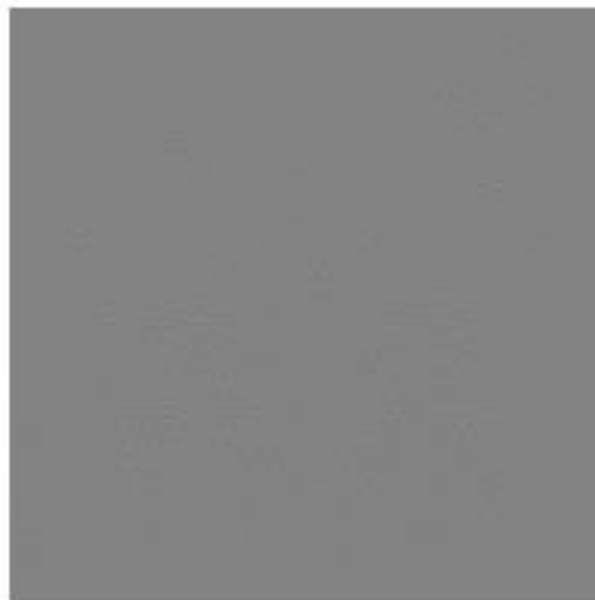


Image Features: Corner Detection

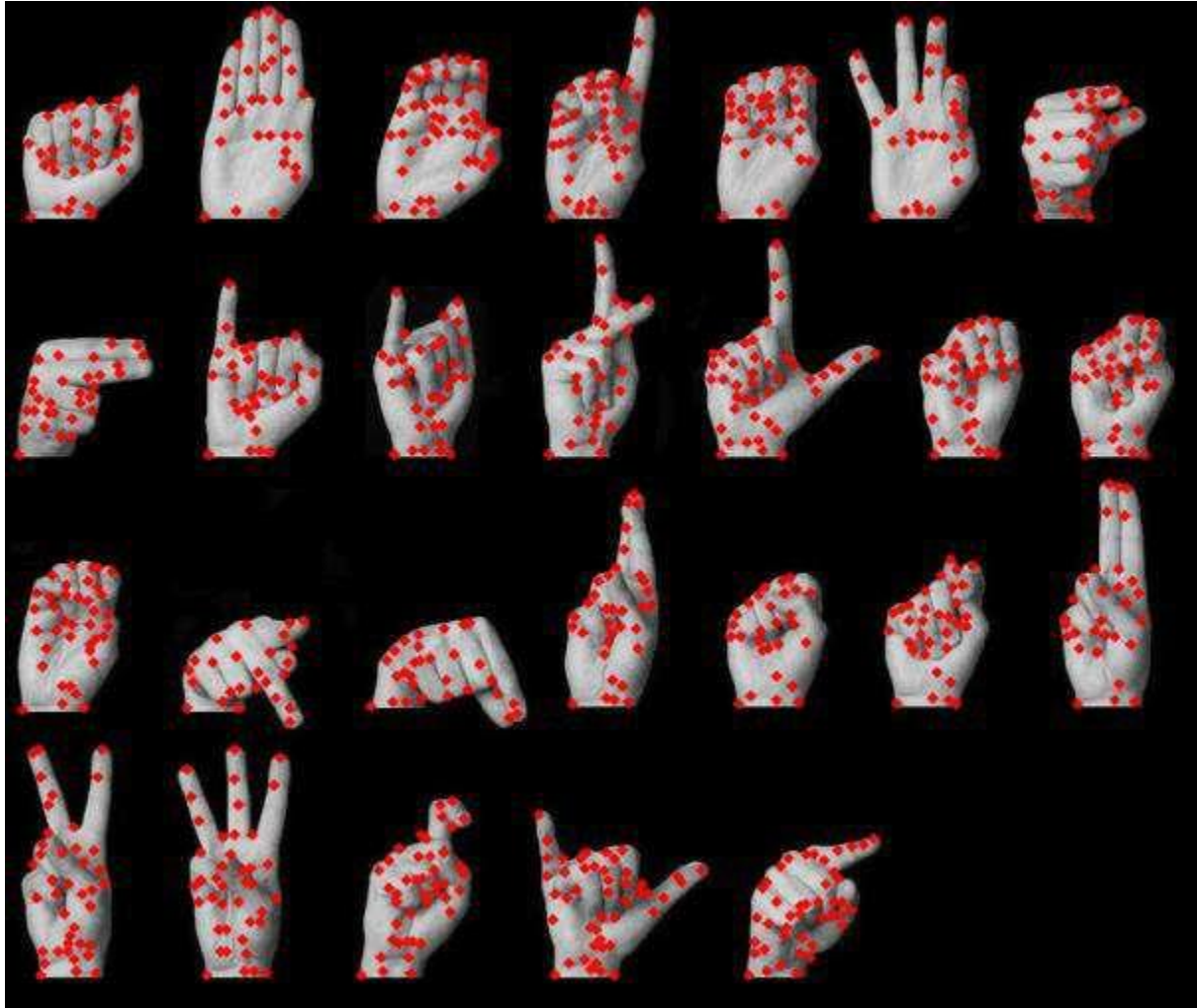


Image Features: Corner Detection

- Corners are regions in the image with large variation in intensity in all the directions.
- Harris Corner Detector
- It basically finds the difference in intensity for a displacement of (u,v) in all directions

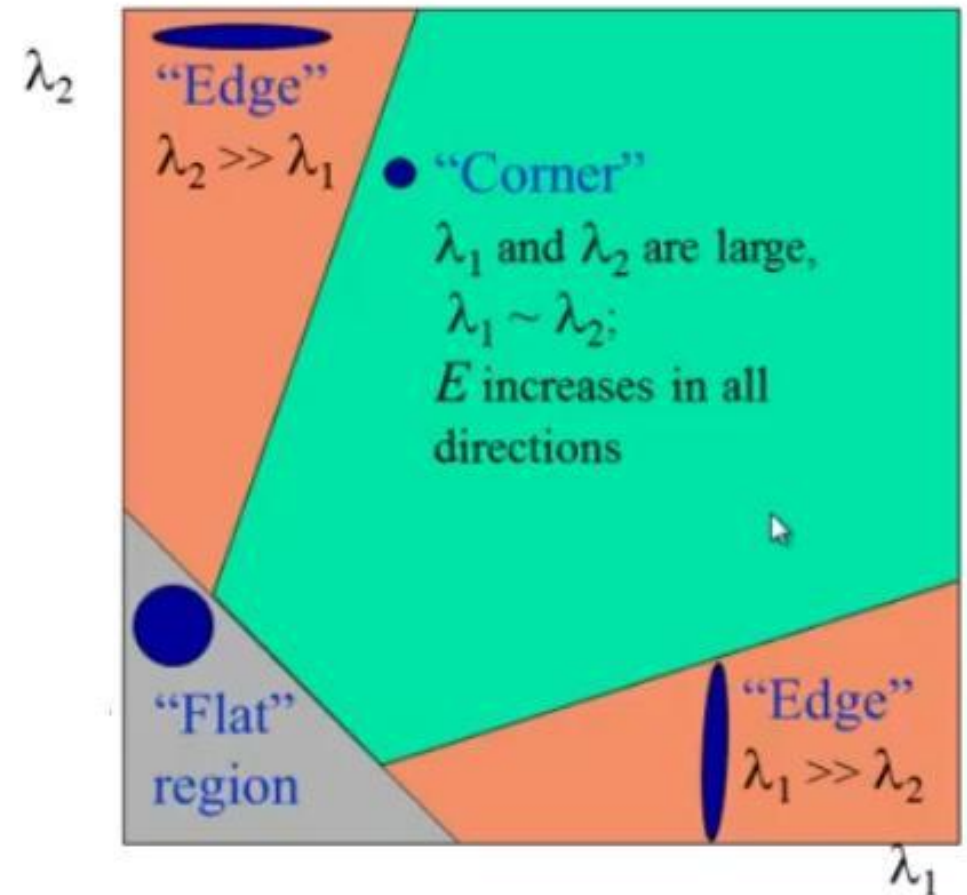


Image Features: Corner Detection

$$E(u, v) = \sum_{x,y} \underbrace{w(x, y)}_{\text{window function}} \underbrace{[I(x + u, y + v) - I(x, y)]}_{\text{shifted intensity}}^2 \underbrace{1}_{\text{intensity}}$$

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = \sum_{x,y} w(x, y) \begin{bmatrix} I_x I_x & I_x I_y \\ I_x I_y & I_y I_y \end{bmatrix}$$

λ_1 and λ_2 are the eigenvalues of M

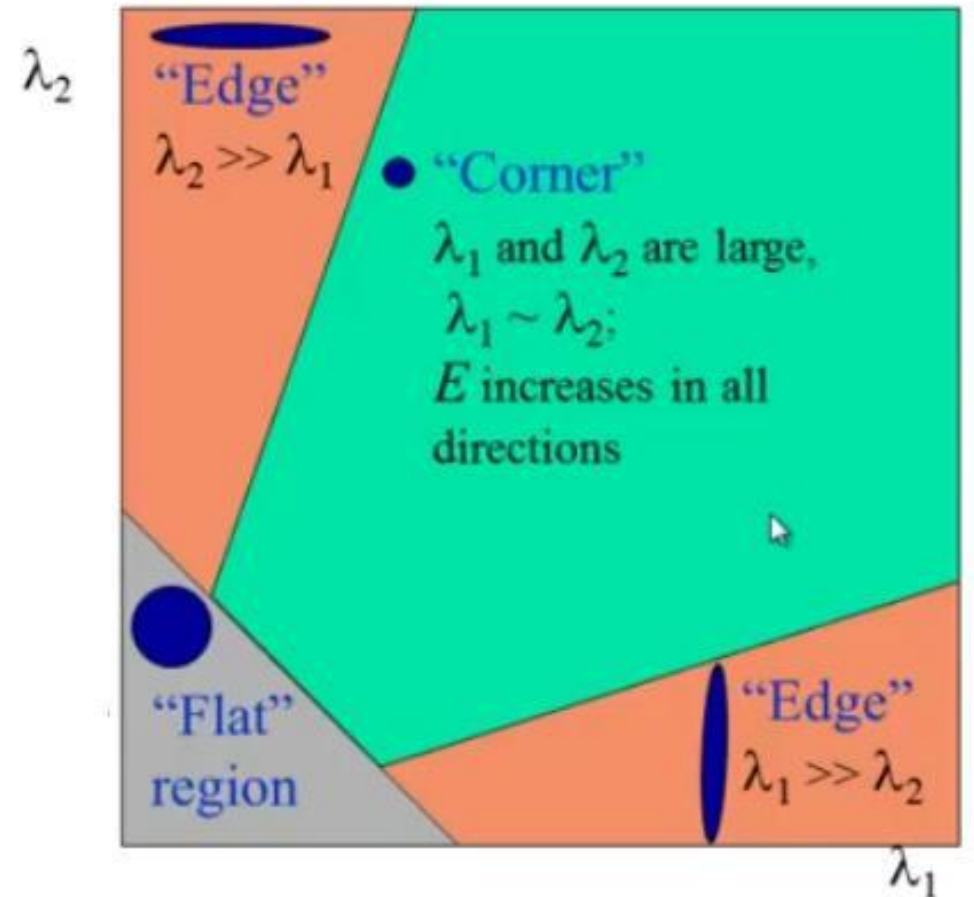


Image Features: Corner Detection

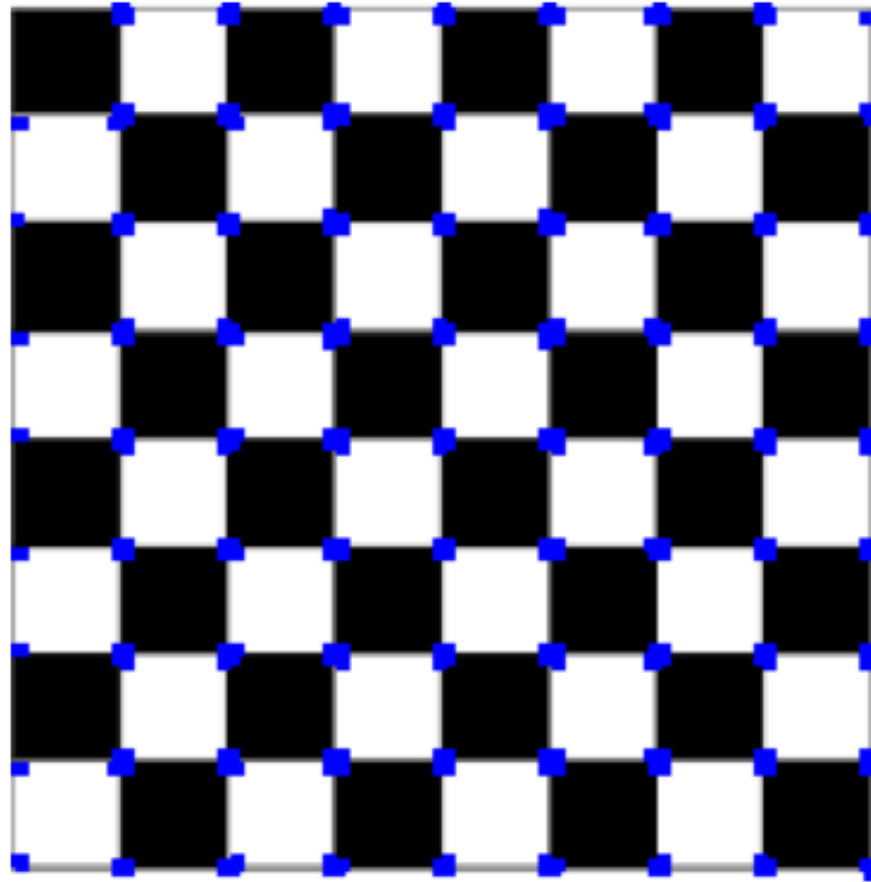


Image Features: Blob detection

- A Blob is a group of connected pixels in an image that share some common property (E.g, grayscale value). In the image above, the dark connected regions are blobs, and blob detection aims to identify and mark these regions.

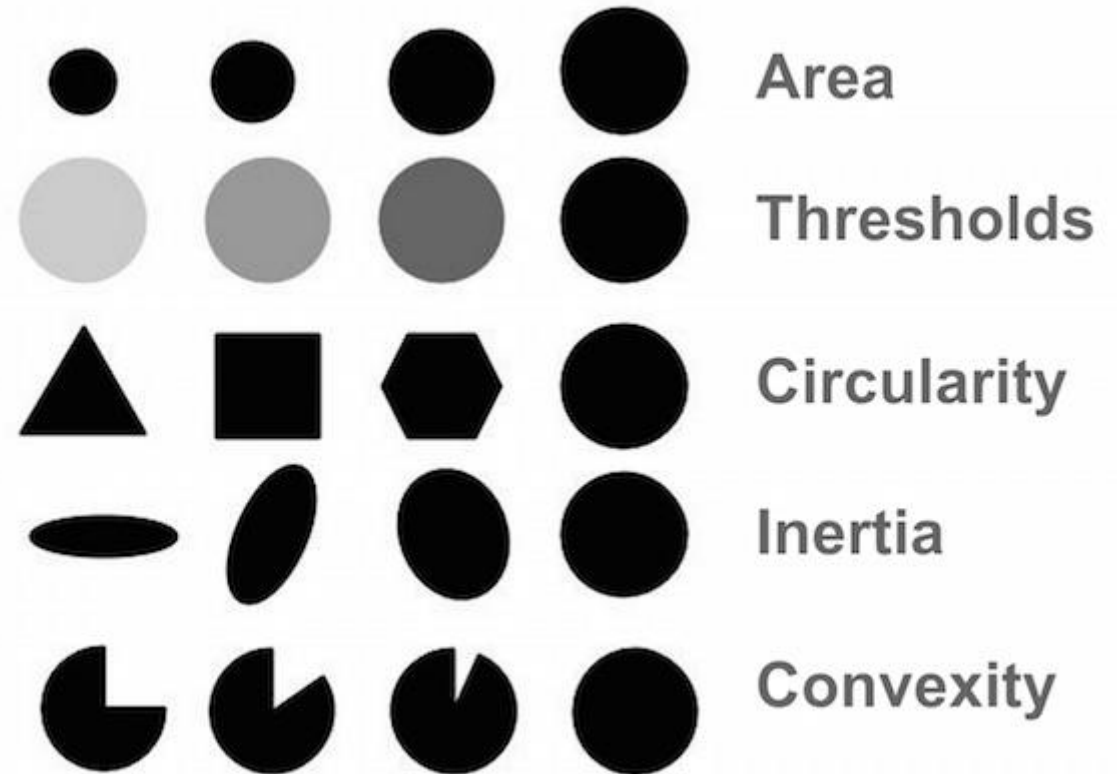


Image Features: Blob detection

- **Thresholding** : Convert the source images to several binary images by thresholding the source image with thresholds starting at minThreshold. These thresholds are incremented by thresholdStep until maxThreshold. So the first threshold is minThreshold, the second is minThreshold + thresholdStep, the third is minThreshold + 2 x thresholdStep, and so on.
- **Grouping** : In each binary image, connected white pixels are grouped. Let's call these binary blobs.
- **Merging** : The centers of the binary blobs in the binary images are computed, and blobs located closer than minDistBetweenBlobs are merged.
- **Center & Radius Calculation**: The centers and radii of the newly merged blobs are computed and returned.

Image Features: Blob detection

- Colour
- Circularity
- Convexity
- Inertia Ratio

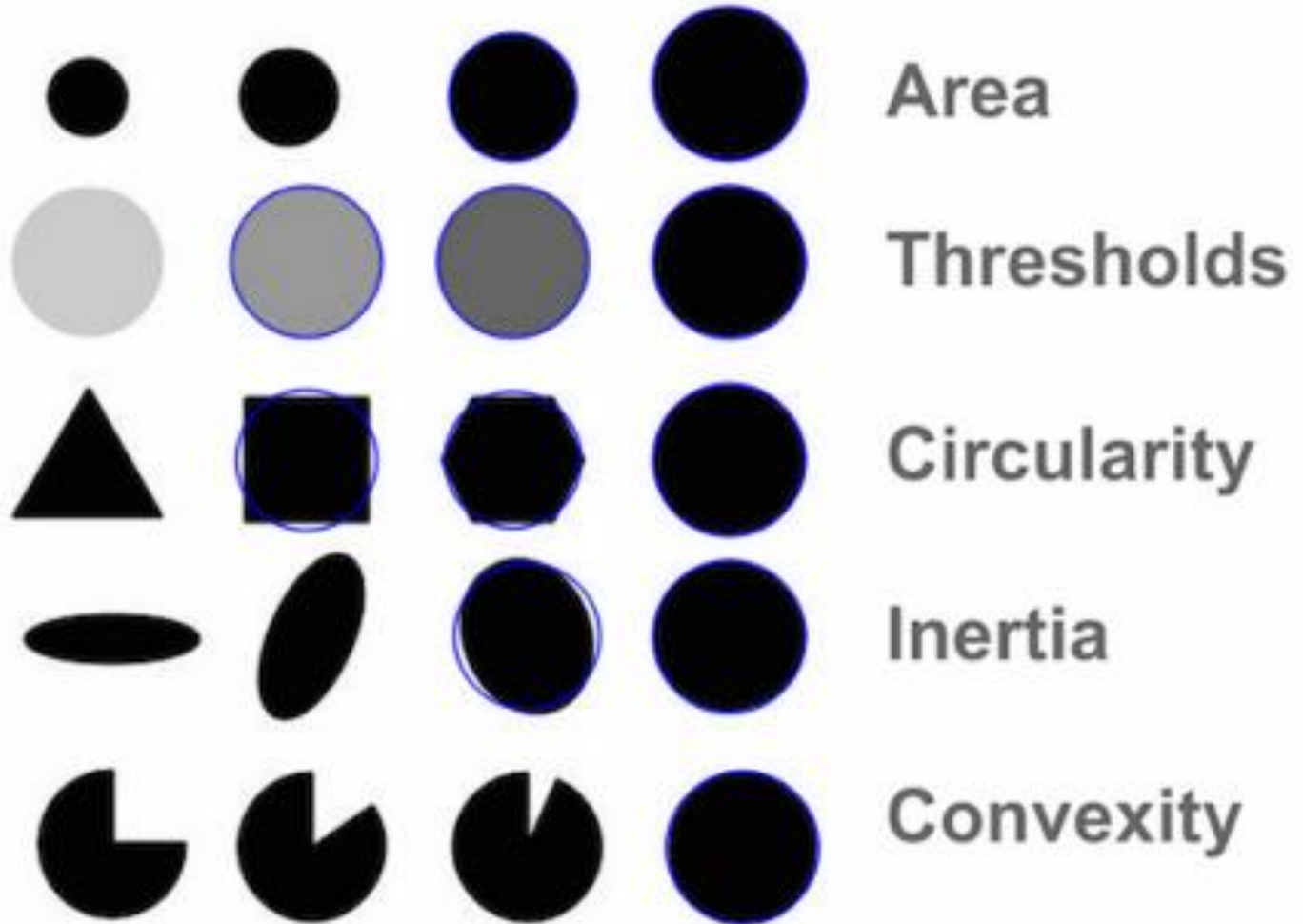


Image Features: Contour detection



(c)



(b)



(d)



(d)

Image Features: Contour detection

- Convert the image to grayscale format. Converting the image to grayscale is very important as it prepares the image for the next step
- Apply binary thresholding or Canny edge detection to the grayscale image.
- Use the `findContours()` function to detect the contours in the image.
- Once contours have been identified, use the `drawContours()` function to overlay the contours on the original RGB image.

Image Features: Contour detection using binary image

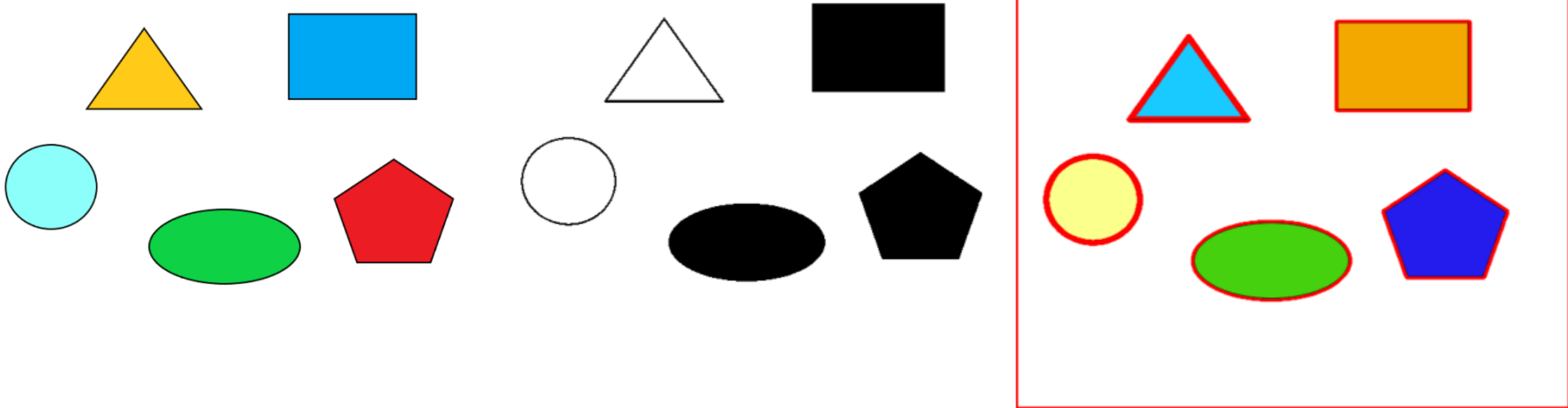
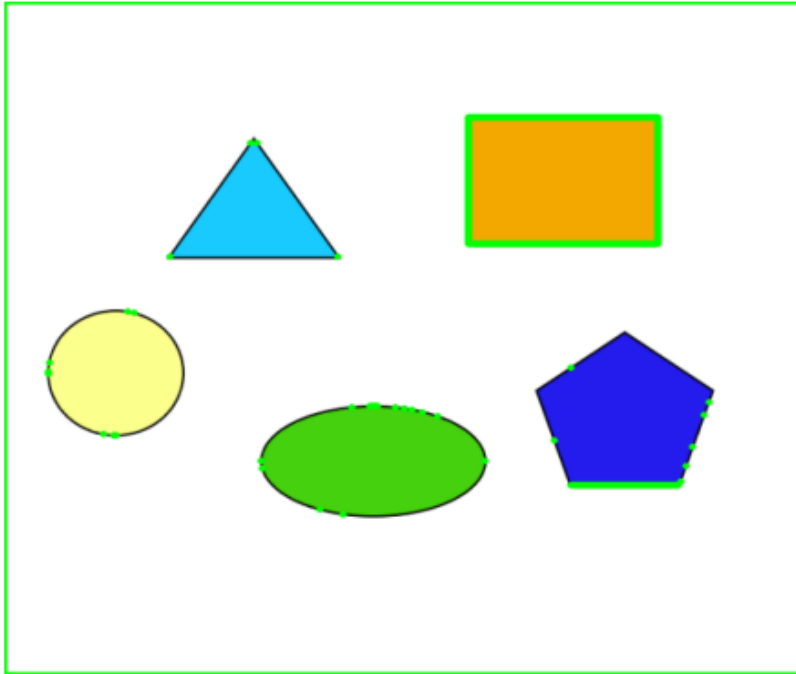
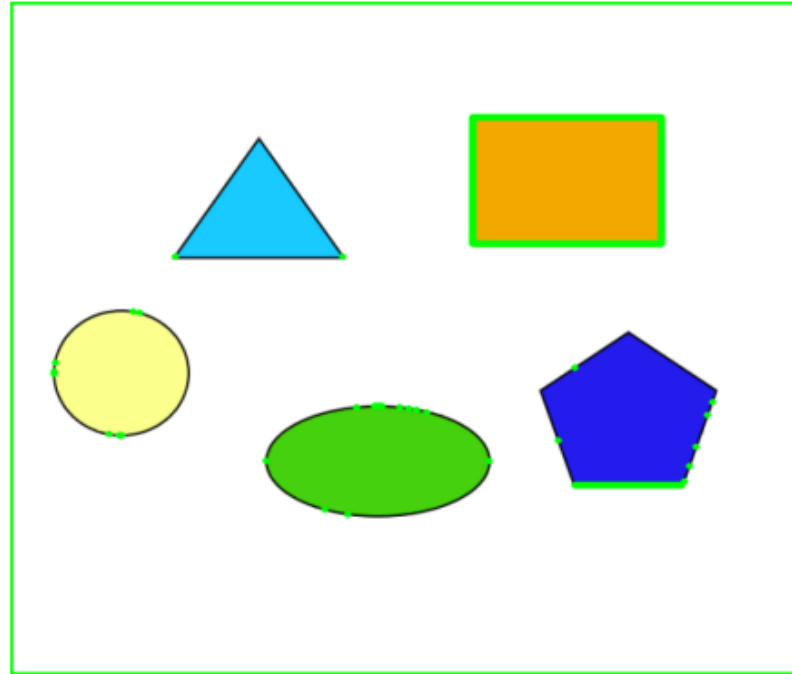


Image Features: Contour detection using three channels

Contour detection using blue channels only



Contour detection using green channels only



Contour detection using red channels only

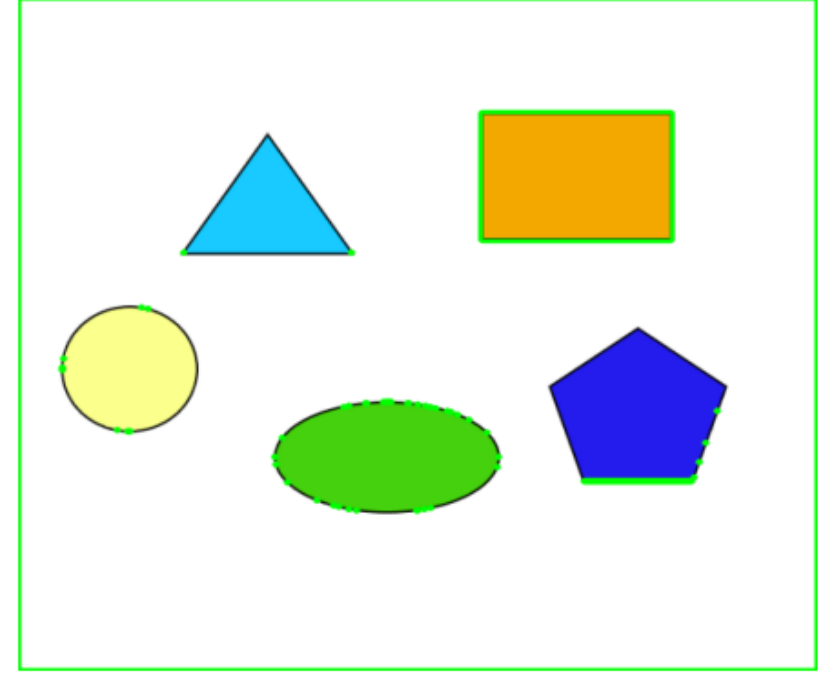
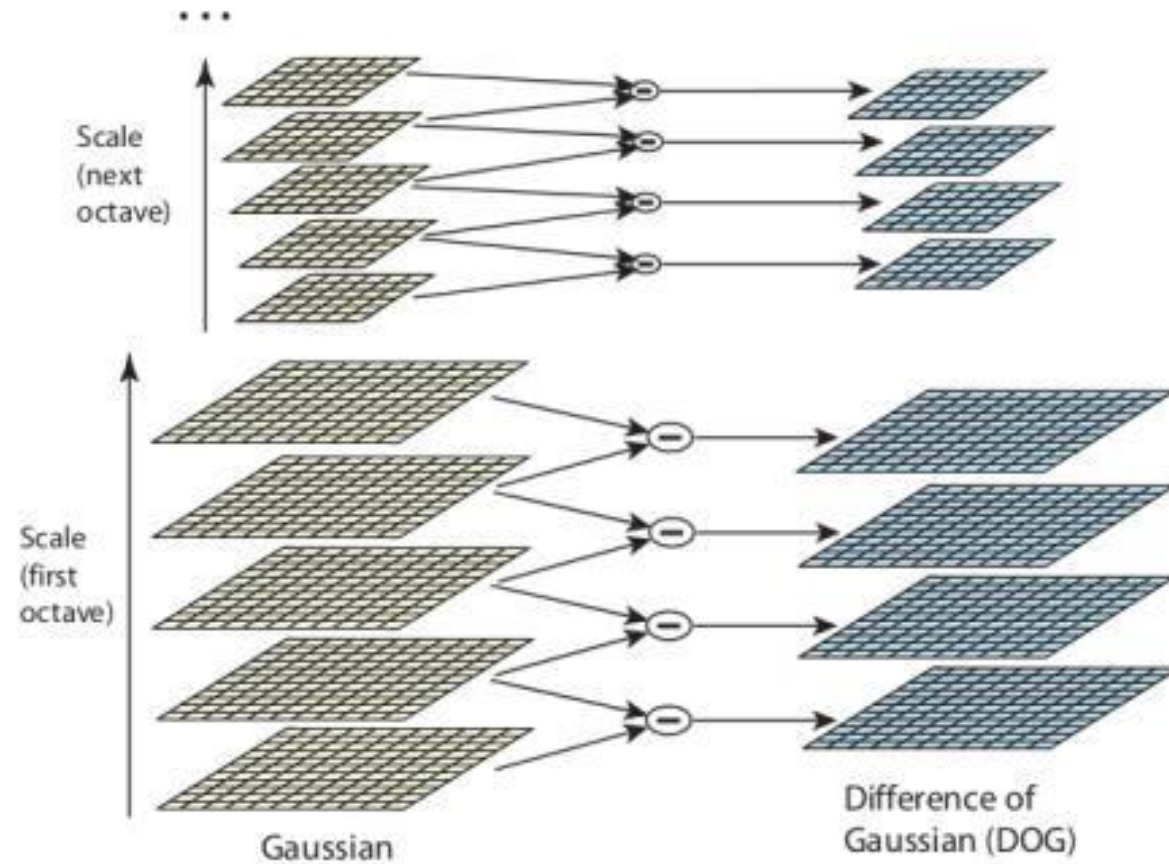
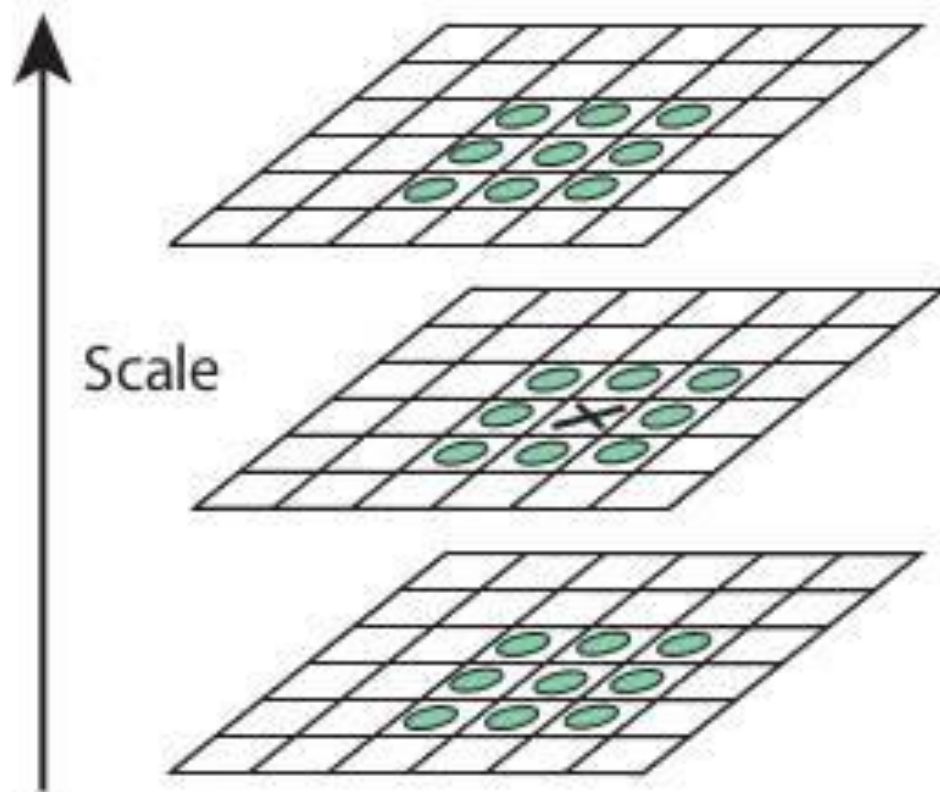


Image Features: SIFT



Difference of Gaussian is obtained as the difference of Gaussian blurring of an image with two different σ , let it be σ and $k\sigma$

Image Features: SIFT



Once this DoG are found, images are searched for local extrema over scale and space. If it is a local extrema, it is a potential keypoint. It means that keypoint is best represented in that scale.

Image Features: SIFT

- Once potential keypoints locations are found, they have to be refined to get more accurate results.
- Contrast and edge thresholds
- Eliminate any low-contrast keypoints and edge keypoints and what remains is strong interest points.

Image Features: SIFT

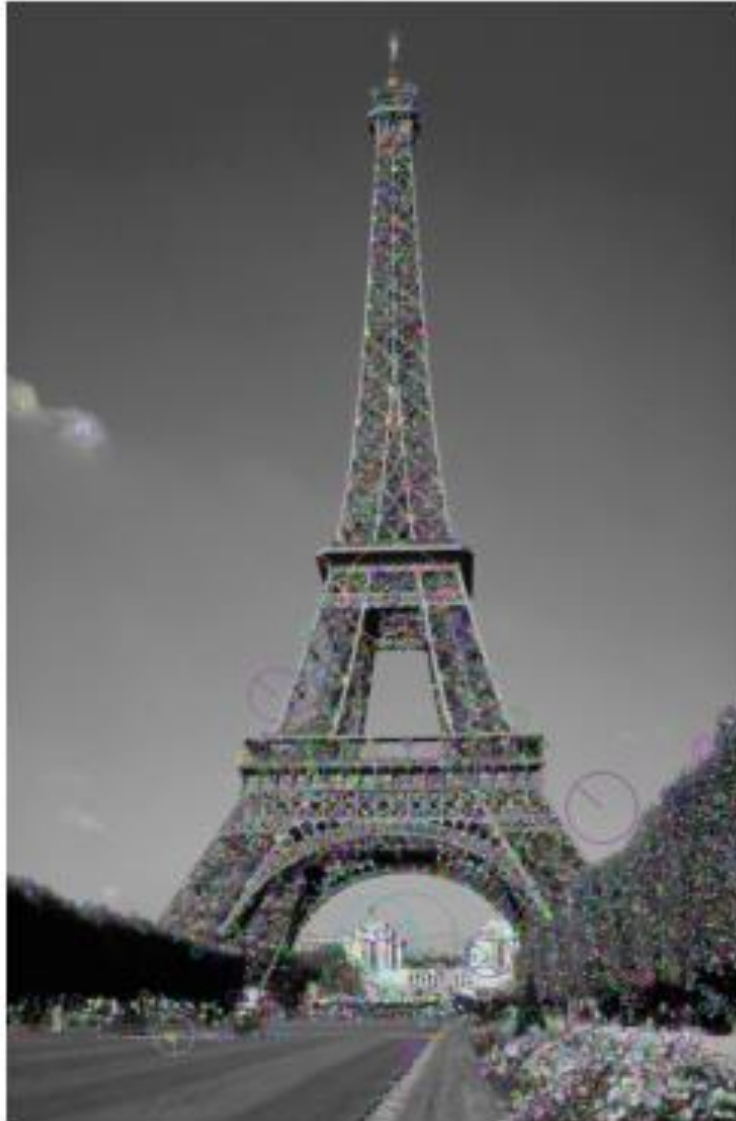


Image Features: SIFT

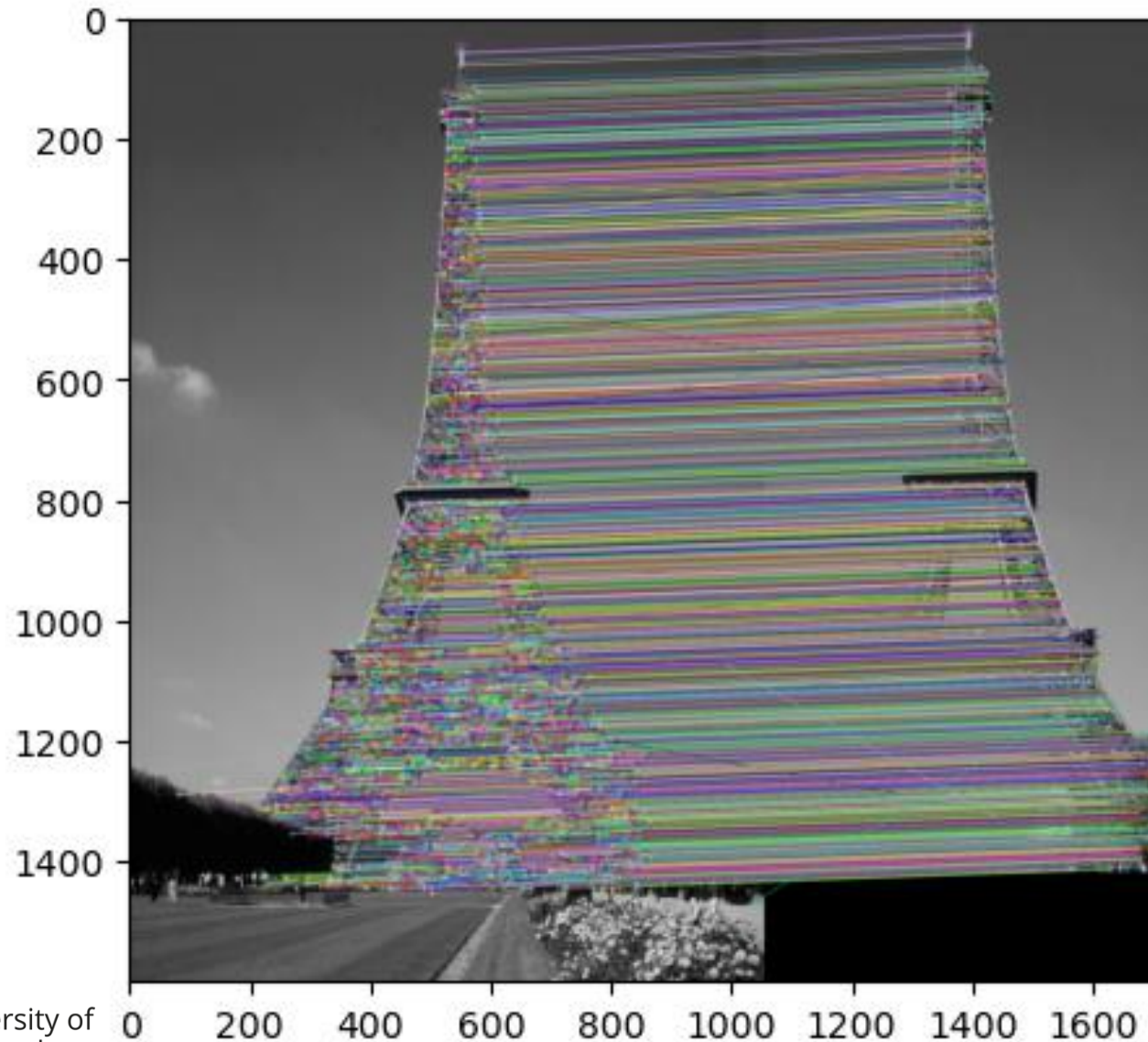


Image Features: SIFT

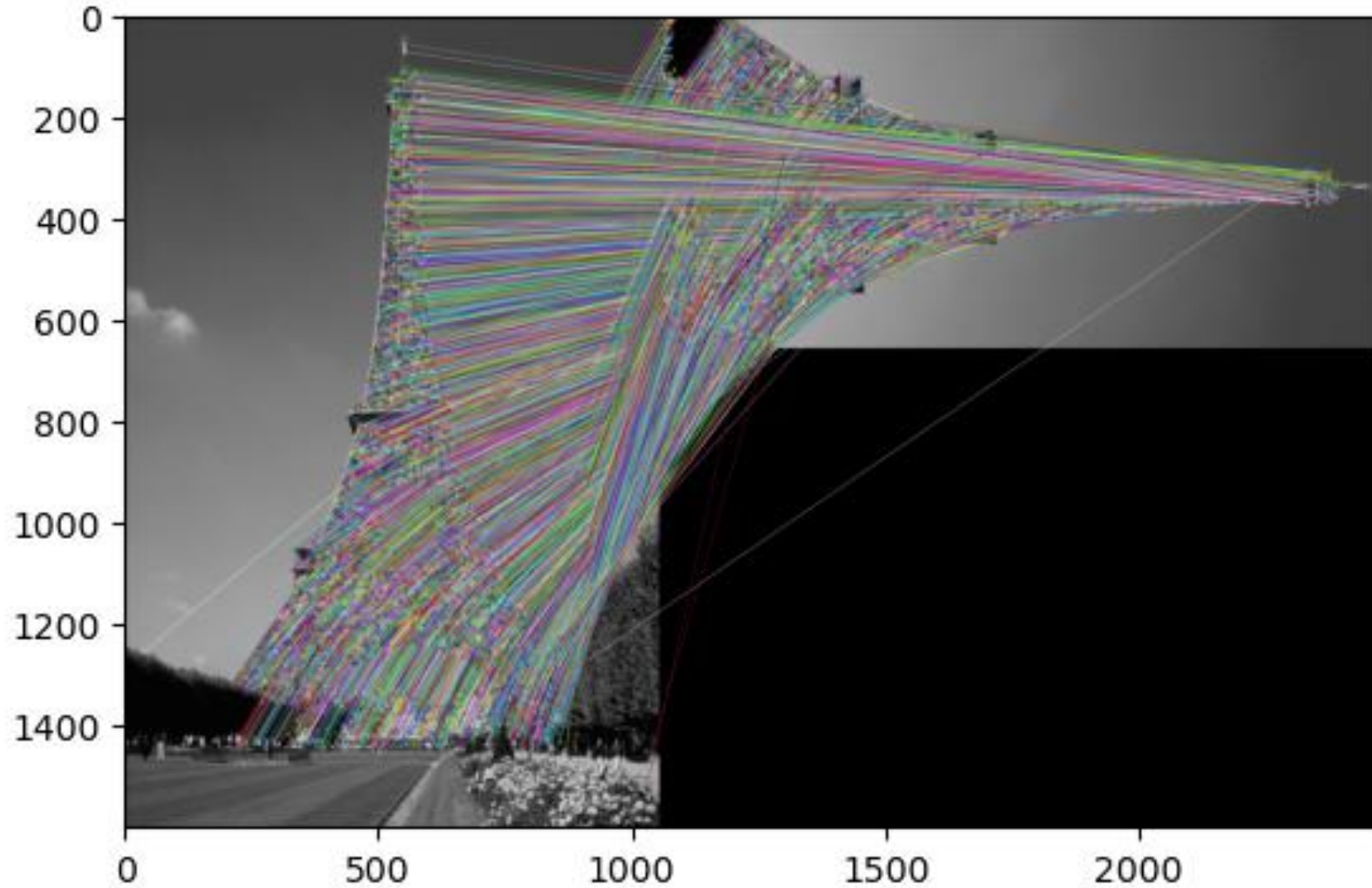


Image Features: PCA

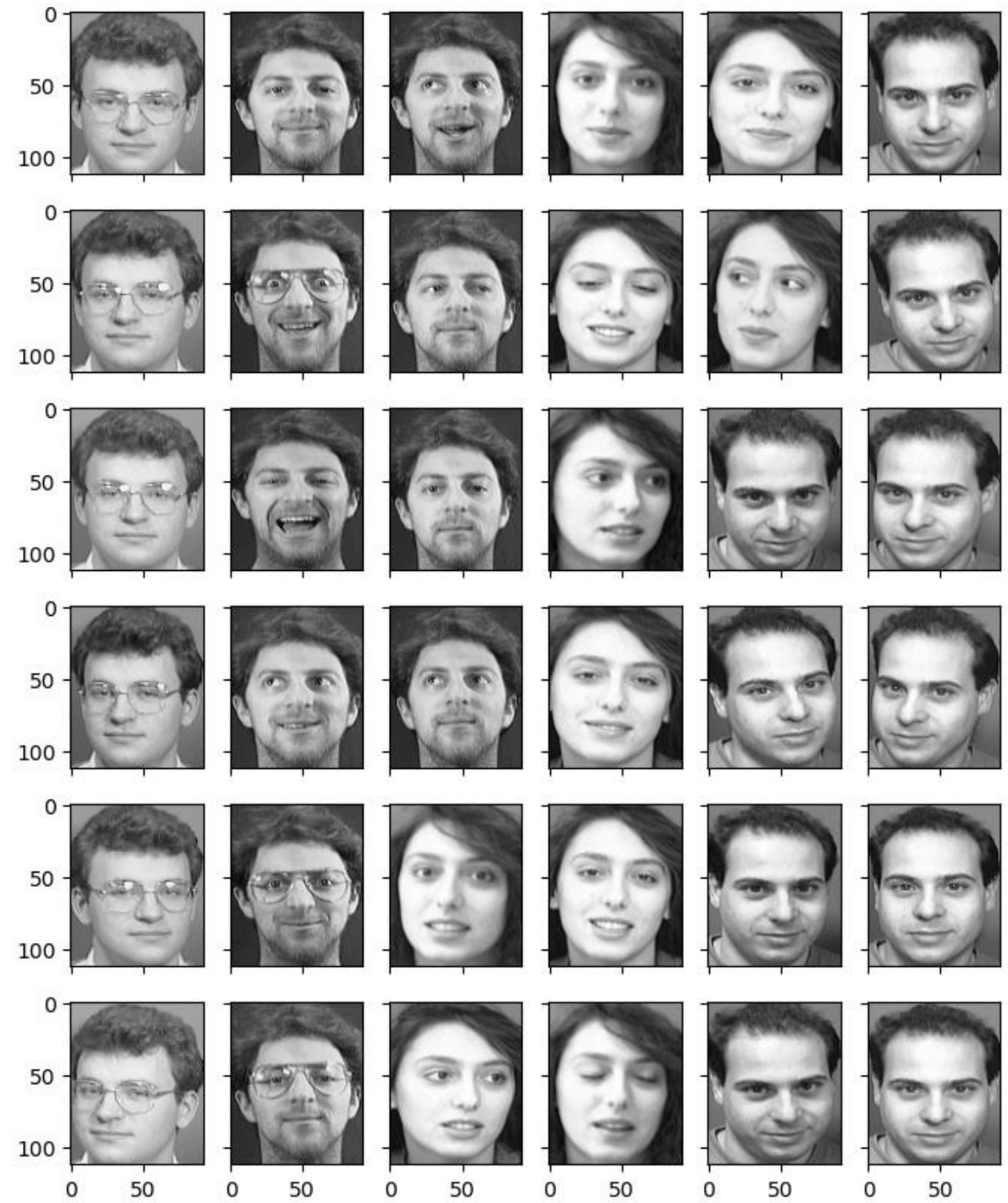


Image Features: PCA

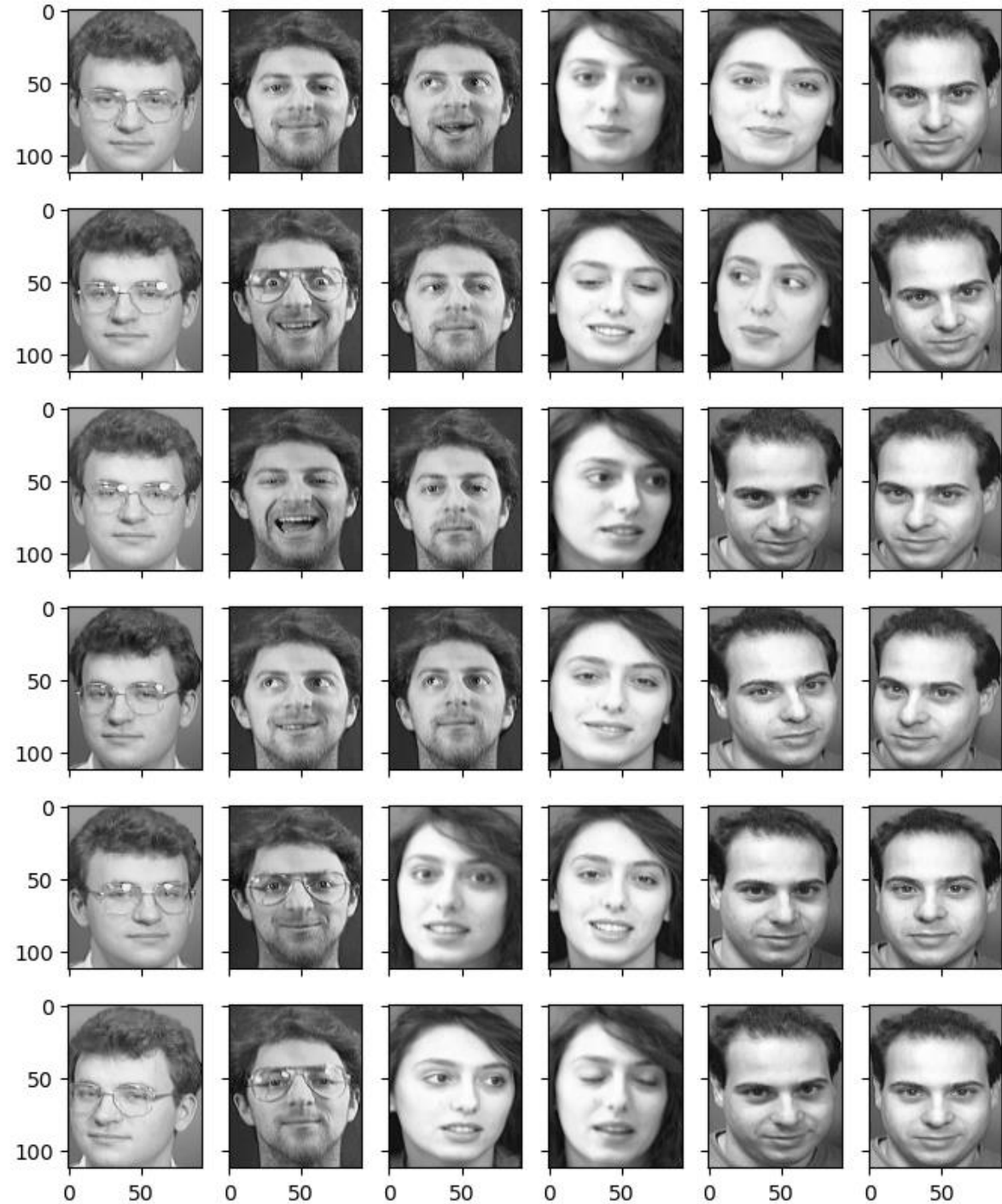
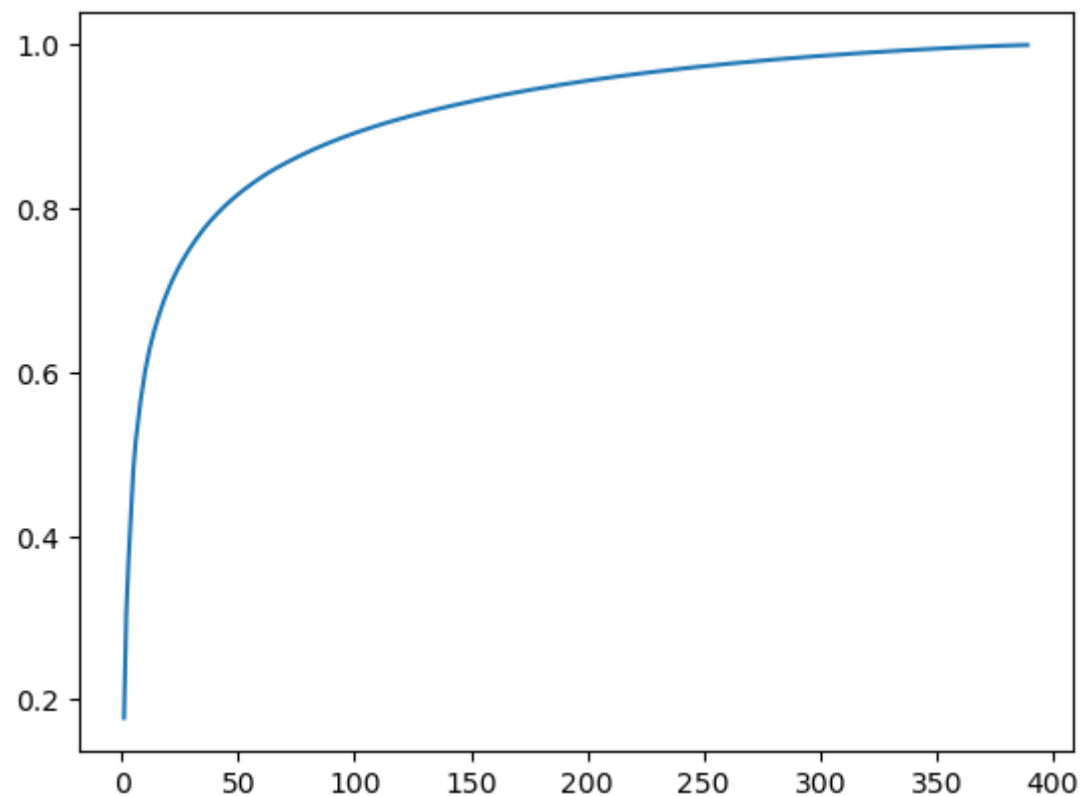
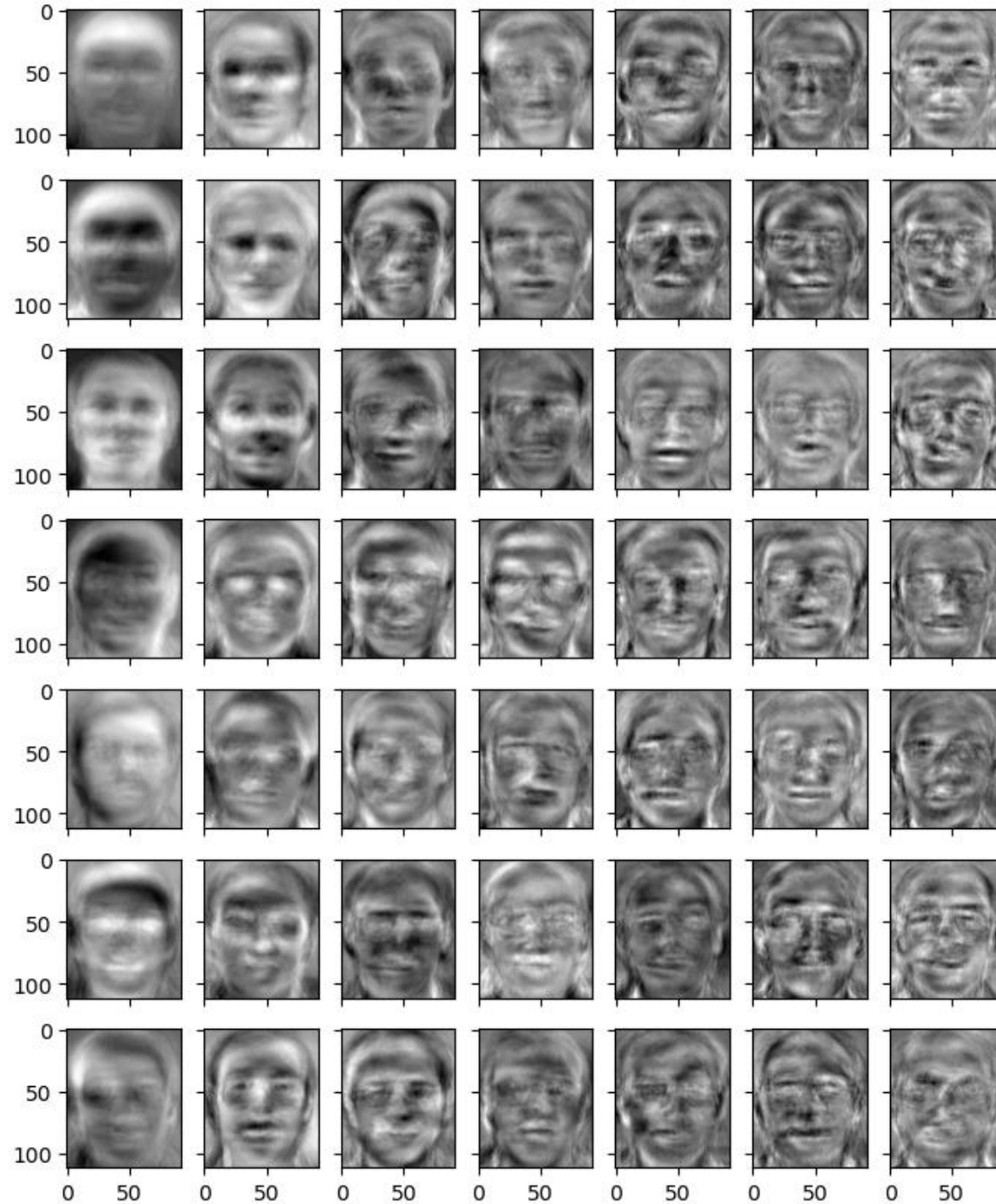
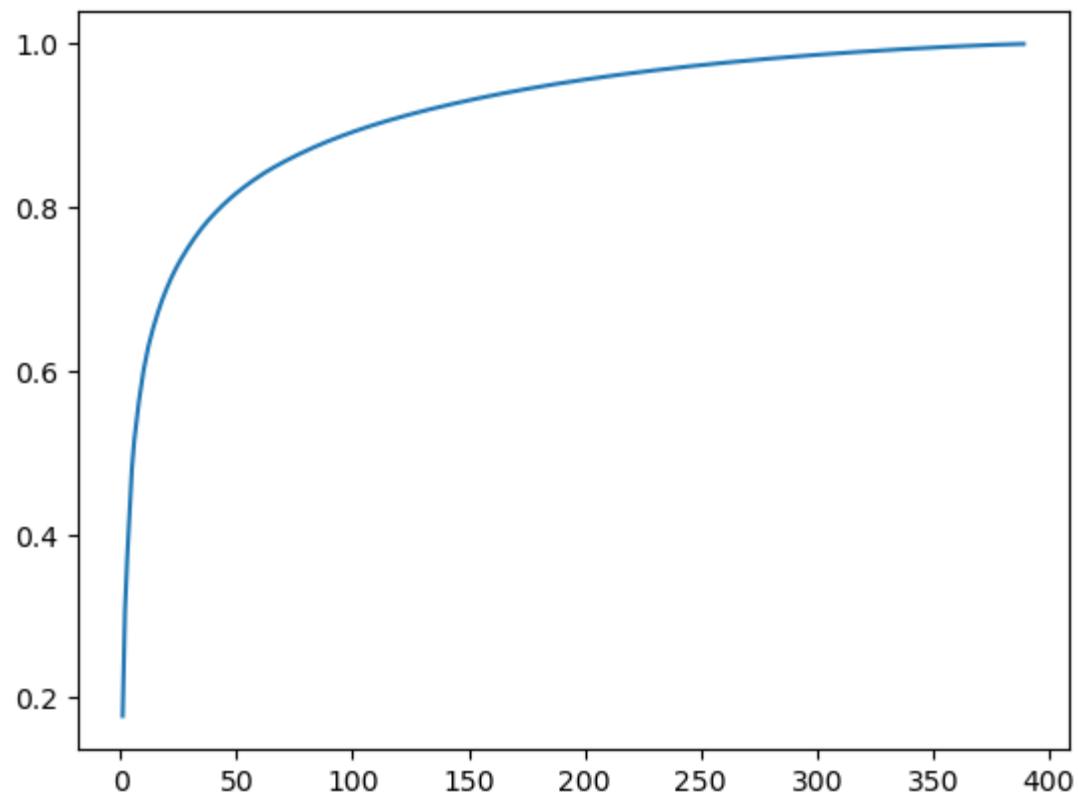


Image Features: PCA



Thank you !!
Questions ??